
Table of Contents

Introduction	1.1
Preface	1.2
Concepts & Tools	1.3
Complex Adaptive Systems	1.3.1
Emergence	1.3.1.1
Bali Irrigation	1.3.1.2
Example Models	1.3.1.3
Summary & Suggested Readings	1.3.1.4
Introduction to Modeling	1.3.2
Epistemology: From Ideas to a Conceptual Model	1.3.2.1
Models	1.3.2.2
Deductive and Inductive Reasoning	1.3.2.3
Dynamics and Scale	1.3.2.4
The Modeling Process	1.3.2.5
Summary & References	1.3.2.6
The Logic of Programming	1.3.3
Functions	1.3.3.1
Programs	1.3.3.2
Components of Algorithms	1.3.3.3
Object-oriented Programming	1.3.3.4
The Art of Programming	1.3.3.5
Summary	1.3.3.6
Introduction to NetLogo	1.3.4
Model Structures	1.3.4.1
First Steps with NetLogo	1.3.4.2
Programming with NetLogo(1)	1.3.4.3
Programming with NetLogo(2)	1.3.4.4
Summary & NetLogo Primitives	1.3.4.5
Randomness	1.3.5
Random Numbers in NetLogo	1.3.5.1

Accidental Millionaire	1.3.5.2
Random Movements	1.3.5.3
Importance of the Sequence of Updating	1.3.5.4
Different Distributions	1.3.5.5
How to use BehaviorSpace	1.3.5.6
Summary, References and NetLogo Primitives	1.3.5.7
Agents & Resources	1.4
Population Dynamics	1.4.1
Introduction	1.4.1.1
Basic Model	1.4.1.2
Density Dependent Resource Growth	1.4.1.3
The Effects of Mating	1.4.1.4
Summary and NetLogo Primitives	1.4.1.5
Foraging	1.4.2
Introduction	1.4.2.1
Creating a Landscape	1.4.2.2
Foraging	1.4.2.3
Memory	1.4.2.4
Summary and NetLogo Primitives	1.4.2.5
Evolution	1.4.3
Introduction	1.4.3.1
Bugs	1.4.3.2
Bug-bird Co-evolution	1.4.3.3
Genetic Drift	1.4.3.4
Summary and NetLogo Primitives	1.4.3.5
Tragedy of the Commons	1.4.4
Introduction	1.4.4.1
Modeling the Tragedy (1)	1.4.4.2
Modeling the Tragedy (2)	1.4.4.3
Summary, Reference and NetLogo Primitives	1.4.4.4
Financial Resources	1.4.5
Introduction	1.4.5.1
Banking Rules	1.4.5.2
Financial Bubbles	1.4.5.3

Summary and NetLogo Primitives	1.4.5.4
Agents & Their Friends	1.5
Networks	1.5.1
Introduction	1.5.1.1
Basics of Networks	1.5.1.2
Random Networks	1.5.1.3
Small World Networks	1.5.1.4
Scale Free Network	1.5.1.5
Summary, Additional Reading, and NetLogo Primitives	1.5.1.6
Diffusion of Innovation	1.5.2
Introduction	1.5.2.1
Diffusion of Innovations	1.5.2.2
Simple Diffusion Model	1.5.2.3
Diffusion in Social Network	1.5.2.4
Lock-in of Products	1.5.2.5
Summary, Literature, and NetLogo Primitives	1.5.2.6
Epidemics	1.5.3
Introduction	1.5.3.1
SIR Model	1.5.3.2
SIR Model on a Network	1.5.3.3
Policy Intervention	1.5.3.4
Summary and NetLogo Primitives	1.5.3.5
Fads and Fashion	1.5.4
Introduction	1.5.4.1
Consumer Behavior	1.5.4.2
A Model of Consumer Behavior	1.5.4.3
Including Social Influence	1.5.4.4
Summary, References and Recommended Readings	1.5.4.5
Collective Action	1.5.5
Introduction	1.5.5.1
A Threshold Model of Collective Action	1.5.5.2
Threshold Model Applied to Networks	1.5.5.3
Pick Your Friends	1.5.5.4

Introduction to Agent-Based Modeling

An introduction to agent-based modeling concepts and practices hosted at [GitBook](#).

Preface

This textbook is based on a collection of lecture notes that I prepared during the last few years in teaching an undergraduate course on agent-based modeling at Arizona State University. This course was given to students from a mixture of backgrounds in the life and social sciences as well as mathematics and computer science. Currently there is no suitable textbook on agent-based modeling for the undergraduate level. Books available focus on the computer science aspects, but do not relate it to the social and life sciences. In my lecture notes I try to link the material to relevant topics in social and life sciences and have a more basic introduction to the computer science concepts than in regular text books.

I focus especially on agent-based models that study how agents form collective behavior through simple rules, how agent interact in social structures and how information is spread from agent to agent. As such we can develop models for many social phenomena from the herding dynamics on financial markets to the development of friendship networks.

The book introduces you to the method of agent-based modeling, but will also use these models to discuss general concepts such as population dynamics, evolution and diffusion processes. At the end of the book you will have a basic understanding of the concepts of complex adaptive systems, the principles of agent-based models, have derived basic experience with programming in NetLogo. If you want to continue to advance your training in agent-based modeling, you can use more advanced textbooks like the one of [Steven Railsback and Volker Grimm](#) which is more focused on a graduate level audience.

The book starts with five chapters on concepts and tools. The concepts from complex adaptive systems such as complexity and emergence and are core to the kind of models we will discuss in this book. Chapter 2 discusses the art of modeling, formal and conceptual, and shows the importance of how we view the world affect decisions how we model problems at hand. Agent-based models are largely based on algorithms describing the rules of agents, and in chapter 3 we introduce basic logic that is fundamental for programming agent-based models. The modeling package NetLogo is introduced in Chapter 4 which is the tool that used throughout the book. In each subsequent chapter we will introduce additional primitives and commands of NetLogo. The first part of the book is closed with a chapter on randomness. Since agent-based models are stochastic it is important to understand what this means for interpreting model results. We will also introduce the tool BehaviorSpace in this chapter to analyze Netlogo models in an efficient way.

Part 2 of the book contains five chapters on agents and resources. Chapter 6 introduces how to model in NetLogo tradition population dynamic models with regrowth, birth and death. We give agents vision and memory in Chapter 7 so they can forage for resources. In

Chapter 8 we show how to model principles of evolution so that attributes of agents may evolve over generations. Resources are constrained, and in Chapter 9 we see how a tragedy of the commons can be avoided if agents have more cooperative norms, which can be stimulated by the use of enforcement. Herding behavior and the consequences for financial resources are discussed in Chapter 10.

Part 3 focuses on agents in social networks. Chapter 11 presents the concept of networks and different types of network structures. We see how network structure affects diffusion of innovations and viruses in Chapters 12 and 13. In Chapter 14 we see how inclusion of psychological based rules for decision making lead to fads and fashions in populations of agents. The book ends with a chapter on collective action and discusses under which conditions agents will cooperate.

I like this opportunity to thank the students from the various classes I taught to provide helpful feedback on earlier versions of the material. I also like to thank Nathan Rollins, Yunhwan Kim, Sean Bergin and Allen Lee for their technical support putting the material online, and Jennifer Fraser for editing the text.

Enjoy the material and let me know your experiences.

Marco Janssen

Concepts & Tools

Complex Adaptive Systems

Key Concepts

By the end of this chapter you will:

- Know the definitions of complex adaptive systems, emergence and stigmergy
- Know examples of these concepts
- Have seen examples of simulation of emergent phenomena

Introduction

Figure 1: Ants creating a bridge.



A little ant is walking on the sandy soil of Arizona looking for food. The food is meant for feeding the brood that is being taken care of by other ants in the colony. Different ants have different tasks in the colony, and the ant we are following is charged with leaving the nest every day in search of food. The amazing aspect of ant colonies is that such a complex organizational structure exists that is not controlled by the queen or a small group of

bureaucratic ants. There is also no plan or to-do list that the ants are following. No, the complexity of the ant colony emerges out of the local interactions among the ants. The ant we observe is following a trail of pheromones, indicating that other ants of the colony have found food nearby and dropped the pheromones on their way back to the nest. The use of pheromones is therefore a way in which ants communicate to others “follow my trail and you may find food.” The pheromone trail will evaporate at a certain speed, and will therefore only be of limited use. If other ants do not follow this trail, bring back food and add their own pheromones to the trail, the trail will disappear. But when the trail is enhanced by successful use by others, a highway of ants may emerge. On such a highway, we see one lane of ants empty handed following the pheromone signal, and the other lane of ants bringing food back to the nest.

There is an enormous diversity of ant-species, all of which have some variation in their social organization. Some ant-species produce ants with different physical characteristics that distinguish their role in the colony: workers, foragers, soldiers, etc. In other ant-species, all of the colony’s ants are physically similar and can switch roles when needed. For example, when foraging ants are killed by an ant-eater, the colony will experience a reduction of food being delivered, which then signals other ants to exchange their caring-for-the-brood role for a foraging role.

Ants change the environment, which subsequently changes the behavior of other ants. This indirect influence of agents via the change of the environment is called stigmergy ([link is external](#)). Another example of stigmergy is the digital trails we develop when we interact with websites. By buying books, renting dvds or listening to music files we leave information “pheromones.” We get recommendations of other books, dvds or music that people “like you” also bought, rented or listen to. These stigmergic interactions can lead to a reinforcement of choices. Popular movies featured on YouTube tend to get even more viewers. If we want to understand how certain books, movies or songs become so popular, we need to look into the various ways choices are influenced and reinforced by others.

Emergence

Emergence is a macro-level phenomenon, like the functioning of an ant colony, as a result of local-level interactions. Like the cartoon in Figure 2, understanding of macro level phenomena can only be derived by studying the consequences of the behavior of micro level agents.

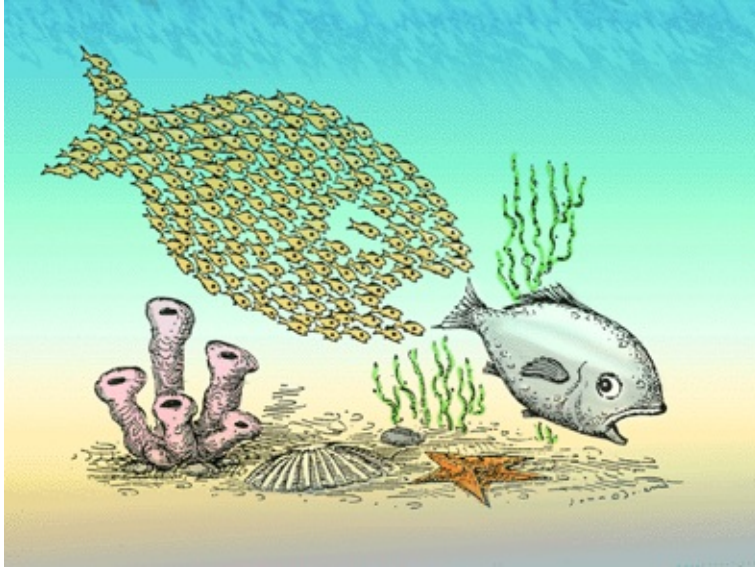


Figure 2: Macro level phenomena emerge from behavior of agents on the micro level

Other examples of emergence are:

- **Path Formation**

Paved paths are not always the most desirable routes going from point A to point B. This may lead pedestrians to take short-cuts. Initially pedestrians walk over green grass. Subsequent people tend to use the stamped grass path instead of the pristine grass, and after many pedestrians an unpaved path is formed without any top-down design.

Look around at your campus and you will find examples of paths that have emerged from a bottom-up design (see example of Figure 3).

Figure 3: Official paths and emerging trails on a university campus.



- **Traffic Jams**

Have you ever been stuck in a traffic jam only to reach the front of the slow-down to discover that there was no obvious reason for it? Sometimes it happens that traffic jams, defined by slow moving cars, develop rapidly in the opposite direction of the flow of traffic. As a consequence you may enter in a traffic jam that was formed many miles ahead. Traffic jams can also form without direct causes like accidents. When drivers have different preferred speed levels, they need to slow down as they approach a slower car in front of them. As a consequence other cars behind this car will need to slow down too. As we will see in later chapters, we can simulate the emergence of traffic jams by simply assuming differences in the desired speed of the drivers.

- **Mexican Waves and Standing Ovations**

In a Mexican wave people stand up and sit down from their seats in succession, creating a human wave. In a standing ovation people stand up to applaud the performance at the end of an excellent play. Both are examples of emergence in stadiums and theaters. No single individual in the audience is able to create a Mexican wave or a standing ovation. So, how do they emerge? When a few people start others may follow. How many need to start to trigger a wave and does it matter where they are sitting?



Figure 4: Mexican wave.

• Flocking Behavior

A flock of birds or a school of fish seems to move in unity (as in Figure 2) but no single fish or bird is controlling or directing the group. What individual behaviors can lead to this flocking behavior?

These emergent phenomena can be studied by [complex adaptive systems](#). With complex adaptive systems we refer to a group of (locally) interacting agents, who constant act and react to actions of other agents. The coherent emergent behavior that might occur in a system arises from the local interactions of the agents. In the case of a school of fish, we can explain the flocking behavior as a result of a few simple rules such as avoiding local crowding, steering toward the average heading of local fishes, and steering toward the average position of local fishes.

Other examples of complex adaptive systems are:

- **stock markets:** Many traders make decisions on the information known to them and their individual expectations about future movements of the market. They may start selling when they see the prices are going down (because other traders are selling). Such herding behavior can lead to high volatility on stock markets.
- **immune systems:** Immune systems consist of various mechanisms, including a large population of lymphocytes that detect and destroy pathogens and other intruders in the body. The immune systems needs to be able to detect new pathogens for the host to survive and therefore needs to be able to adapt.
- **brains:** The neural system in the brain consists of many neurons that are exchanging information. The interactions of many neurons make it possible for me to write this sentence and ponder the meaning of life.
- **ecosystems:** Ecosystems consist of many species that interact by eating other species, distributing nutrients, and pollinating plants. Ecosystems can be seen as complex food webs that are able to cope with changes in the number of certain species, and adapt – to a certain extent – to changes in climate.

- **human societies:** When you buy this new iPhone that is manufactured in China, with materials derived from African soils, and with software developed by programmers from India, you need to realize that those actions are made by autonomous organizations, firms and individuals. These many individual actions are guided by rules and agreements we have developed, but there is no ruler who can control these interactions.

Complex adaptive systems and emergence are determined by what we call bottom up processes. The opposite would be a system of top-down control. In a top-down controlled system all individual components obey the rules of a central commander or a blue print. Examples are armies and bureaucratic organizations, which are trained to execute orders from higher echelons. They have precise instructions on what to do in order to meet the goals of the organization as a whole. Taking individual initiatives are not appreciated since it may disrupt the well-trained order in the system.

Bali Irrigation



Figure 5: Rice fields at Bali.

An interesting example of emergence and complex adaptive systems in social systems is irrigation in Bali, Indonesia. Farmers who irrigate their fields in Bali have to solve a complex coordination problem that emerges at the intersection of two separate issues: pest outbreaks and water shortage. On one hand, control of pests is most effective when all rice fields in a watershed have the same schedule of planting rice. The reason is that rice is harvested at the same time and no food for the pests exists in a larger area to survive. If harvest of rice was in small fields only, pest can survive after a harvest by moving to a neighboring field where rice is still on the field. On the other hand, the terraces are hydrologically interdependent, with long and fragile systems of tunnels, canals, and aqueducts. Therefore, to avoid water shortage, the irrigators should not plant rice all at the same time.

To balance the need for coordinated fallow periods and use of water, a complex ritual system has been developed that details what rituals and actions should be done on each specific date in each organized group of farmers—called a [subak](#). These actions are related to offerings at temples, ranging from the little temples at the rice terrace level to the temples at the regional level and all the way up to the temple of the high priest Jero Gde, the human representative of the Goddess of the Temple of the Crater Lake. Crater Lake feeds the groundwater system, which is the main source of water for irrigating in the entire watershed. These offerings were collected as a counter gift for the use of water that belonged to the gods.

There is no person who has control over the whole irrigation system. At the temple level, subak leaders come together regularly to exchange information on their irrigation experiences in their subaks. When a particular subak makes a decision that has unfavorable effects on neighboring subaks, leaders of the local temple community come together to discuss these matters. This may lead to various actions, including the threat of cutting-off the water supply to the disrupting subak. When seasonal rainfall is different than expected, subaks leaders may come together to discuss alternative cropping pattern to avoid unfavorable circumstances.

The function and power of the water temples were invisible to the planners involved in promoting the [Green Revolution](#) during the 1960s. They regarded agriculture as a purely technical process. Farmers were forced to switch to the miracle rice varieties, which were predicted to lead to three harvests a year, instead of the two harvests of traditional varieties. Farmers were motivated by governmental programs that subsidized the use of fertilizers and pesticides. After the governmental incentive program was started, the farmers continued performing their water temple rituals, but they no longer coincided with the timing of rice-farming activities. Soon after the introduction of the miracle rice, a plague of plant-hoppers caused huge damage to the rice crop. A new variety was introduced, but then a new pest plague hit the farmers. Furthermore, there were problems of water shortage.



Figure 6: Irrigation network of subaks and temples, as studied by Steve Lansing (from Lansing and Kremer, 1994).

During the 1980s, an increasing number of farmers wanted to switch back to the old system, but the engineers interpreted this as religious conservatism and resistance to change. It was anthropologist [Steve Lansing](#) who unraveled the function of the water temples, and was able to convince the financiers of the Green Revolution project on Bali that the irrigation was best

coordinated at the level of the subaks with their water temples. Lansing built an agent-based model of the interactions of subak management strategies and the ecosystem, and the local adaptation of subaks to strategies of neighboring subaks, and showed that for different levels of coordination, from farmer level up to central control, the temple level was the level of scale where decisions could be made to maximize the production of rice. He also showed how the coordination might have been evolved as a result of local interactions. In his agent-based model, agents, representing subaks, make a decision at the end of the year which cropping pattern to use for the next year. They will look at their neighboring subaks to determine whether one of them is doing better than themselves. If this is the case, the subak copies the strategy of the better performing subak. Within a few years, the average simulated yield increases to a level that is close to the maximum yield. The result is that there are pockets of subaks who have the same cropping pattern, solving the pest problem. These pockets of subaks are small enough to avoid serious water shortages. Steve Lansing therefore shows that a complex irrigation network is able to organize themselves by pure local interactions.

Example Models

In this book we will use models to study complex adaptive systems. You will learn how to develop such models in Netlogo. Netlogo is a modeling platform developed by [Uri Wilensky](#) and his team at Northwestern University, and can be downloaded for free at <http://ccl.northwestern.edu/netlogo/>. We will use Netlogo version 5.0 for the examples in this book.

To get an idea of the types of models we are looking at during the course you can explore the large Netlogo Model library. There are many examples of various topics. Before you learn the programming, you can play with the existing models. We discuss now two examples in more detail.

In Netlogo Model library under the heading Biology there is a model titled Ants that simulates the foraging of a colony of ants. The ant colony in the center has 3 lumps of resources in the neighborhood. In Figure 7 the purple circle is the colony, and the blue circles are food piles. The ants initially move randomly around until they find a food resource. If an ant finds a food resource, they return back to their colony leaving a trail of pheromones behind them. Ants walking randomly around may encounter pheromone trails, and they follow this in the direction away from the colony. The three food piles are at different distances from the colony. You will experience that the ants consume first the closest food pile and end with the most distant food pile. One can explore the dynamics of the foraging behavior by changing the number of ants, the rate at which an ant leave pheromones behind them, and the rate the pheromones evaporate.

NETLOGO EXAMPLE: ANT MODEL

Model: Right click on the Link and Save it. Open Netlogo and Run it with NetLogo.

Another example of emergence is flocking behavior. In the biology section of the model library of Netlogo you will find the model titled Flocking. The agents monitor the movements of other agents in their neighborhood. If they are too close (minimum separation) to their neighbors they turn their direction (max-separate turn) to increase the separation. If an agent is not very close to its neighbors it turns its direction in line with the neighbors (max-align turn) to decrease the separation (max-cohere turn). The result is that groups of agents emerge who move around in a group. The values of the sliders determine whether a large group of agents or smaller groups of agents emerge.

NETLOGO EXAMPLE: FLOCKING MODEL

Model: Right click on the Link and Save it. Open Netlogo and Run it with NetLogo.

Summary & Suggested Readings

Summary Complex adaptive systems are systems that are defined by a group of (locally) interacting agents, who constant act and react to actions of other agents. Examples of such systems are ant-colonies, immune systems, ecosystems, financial markets, etc.

One of the phenomena of complex adaptive systems is the occurrence of emergence, which is a macro-level phenomenon as a result of local-level interactions. Examples are ant-trails, immune system response to a virus, flocking behavior of birds and bubbles in financial markets.

One of the mechanisms in complex adaptive systems that can cause emergent phenomena is stigmergy. Individuals change the environment, which, in turn, affects the behavior of other individuals.

In this course we will use agent-based models to study complex adaptive systems. We will also use computer simulation to find out how simple rules of local interactions of agents lead to emergent phenomena.

Suggested Reading

Gordon, D. (1999) *Ants at work: How an insect society is organized*. New York/London: W.W. Norton & Company.

Holland, J.H. (1998) *Emergence from chaos to order*. New York: Oxford University Press.

Lansing, J.S. (2006) *Perfect order: Recognizing complexity in Bali*. Princeton, NJ: Princeton University Press.

Lansing, J.S. and J.N. Kremer (1994) Emergent properties of Balinese water temples. *American Anthropologist* 95 (1): 97-114.

Mitchell, M. (2009) *Complexity: A guided tour*. New York: Oxford University Press.

Waldrop, M.M. (1992) *Complexity: The emerging science at the edge of order and chaos*. New York: Simon & Schuster.

Introduction to Modeling

Key Concepts

In the chapter you will

- Get an introduction to models and modeling
- Learn that models are a simplification of reality
- Notice that there are different ways to simplify reality in a meaningful way
- See that you have to make assumptions, but that you will be able to test the effects of assumptions you make.

Introduction

In this chapter many concepts are discussed that might not be familiar to you. We will have some philosophical discussions on how we know what we know and how there are different ways of knowing. By the end of the chapter you may not understand all the concepts, but in the weeks to come you will recognize some of the concepts we discuss. In fact, one of the main principles of learning to model is to do modeling, to struggle with the process and thereby learn how to do it better. Although modeling is a scientific method, to do modeling is sometimes an art.

Epistemology: From Ideas to a Conceptual Model

Epistemology is a branch of philosophy that deals with knowing and the methods of obtaining knowledge. What we consider in this chapter are the different ways that scholars reason about their topics of interest as far as this is relevant for agent-based modeling. Since agent-based modeling, like other modeling approaches in social simulation, combines knowledge, insights and tools from various disciplines, it is important to understand that disciplines differ in how they derive knowledge.

Models

Since our focus is on models, let us first discuss what we mean by a model. In fact, there are many kinds of models. A general classification is the distinction between formal mathematical models and non-formal models such as stories and cartoons. Sometimes people serve as role models for how one might live, such as [Nelson Mandela](#) or [Claudia Schiffer](#). In this course we will only deal with formal models. The advantage of these formal models is that they are explicit and many of them are computer models that can be used to do repeated experiments.

Models are simplifications of reality. To make models we have to make decisions what to leave out of the model. What are the most important characteristics of the problem we like to describe? In making the simplifications we make assumptions how to simplify. We start to rely on theory and use theory to define what we need into our model. But there are different theories and different approaches to make assumptions. We can develop formal models by using different approaches. Each modeling approach, in a broader context, involves its own set of theories, concepts, mathematical techniques, and accepted procedures for constructing and testing models. We can distinguish between deterministic and stochastic models, simulation and optimization models, reductionistic and integrated models, linear and non-linear models, one-agent and multi-agents models, and so on.

Instead of discussing all kinds of possible approaches, we want to discuss the difference between the reductionistic Newtonian approach and the complex adaptive systems approach. Since our course uses a complex adaptive system approach it is good to know how this differs from another frequently used approach. The basic difference is that a Newtonian approach assumes that a system, like the solar system, can be described as a clock. That is, it is mechanical and we can fully predict what will happen if we just have the right model. Complex adaptive models assume that there is no predictability of the full trajectory of the system. The system does not work like a clock, but like a flock of birds, constantly adjusting to changing contexts.



Figure 1: Issac Newton.

Mathematical modeling has long been influenced by physical science, which has developed a mechanistic, reversible, reductionistic and equilibrium-based explanation of the world. This proved to be very successful in calculating trajectories of moving objects (e.g., cannon balls) and predicting the positions of celestial bodies. The work of [Isaac Newton](#), culminating in the [Principia Mathematica Philosophiae Naturalis](#) in the late 17th century, was and still is very influential. In fact, Isaac Newton described the world as a machine for which we can predict its development if we understand all the mechanisms. Newton showed that he could explain and predict correctly the movement of planets around the sun. At least correct to an extent given the measurements they had at that moment in time.

The associated rational and mathematical way of describing the world around us was also applied during the last 100 years in the social sciences, economics and biology. Despite the fact that later developments in the natural sciences seriously constrained the applicability of the mechanistic paradigm, its relative simplicity had a great appeal to scientists from various disciplines working with models. However, despite the widespread use of this approach, the mechanistic paradigm is increasingly criticized. The foundations of the mechanistic view—reversibility, reductionism, and equilibrium-based and controllable experiments—have faded away in light of a number of “new” scientific insights. Basically we have to consider whether many of the systems in the life and social sciences can be described as a predictable machine.

First, the discovery of the [Second Law of Thermodynamics](#) brought down the notion of reversibility. The Second Law states that the entropy of a closed system is increasing. This means that heat flows from hot to cold, so that less useful energy remains. One of the consequences of the Second Law is the irreversibility of system behavior and the single direction of time. History matters!! Changes within systems cannot reverse back just like that (irreversible). This is in contrast to many mechanistic models, in which time can easily be reversed to calculate previous conditions.

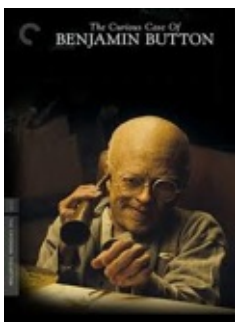


Figure 2: Benjamin Button.

If you let a glass fall it may break. It costs more energy to repair the glass than it cost to break it. When people become older parts of the body will degenerate. Our body can heal but scars of time will remain. Except in the case of the main character in the fantasy movie

[Benjamin Button](#) played by Brad Pitt. Here a baby was born looking old, bald and wrinkled, and became more youthful over time.

Second, the equilibrium view of species was brought down by [Charles Darwin's](#) book *On the Origin of Species* during the mid-19th century. The static concept of unchanging species was replaced by a dynamic concept of evolution through natural selection and adaptation, thereby fundamentally changing our view of nature. Natural systems are in continuous disequilibrium, being interdependent and constantly adapting to changing circumstances.

Third, the theories of [quantum mechanics](#) have confronted us with a fundamental uncertainty regarding knowledge about systems, especially on the level of atoms and particles. The [uncertainty principle](#) of [Werner Heisenberg](#) is well known, stating that it is impossible to simultaneously measure the position in space and momentum (mass times velocity) of any particle. The statement by [Pierre-Simon Laplace](#) in the early 19th century that if every position of every atom was known, the future might be predicted exactly, became therefore an illusion. Moreover, the notion of fundamental uncertainty implied that fully controlled experiments are strictly speaking not possible.

Notwithstanding the fact that these developments in the natural sciences changed our perception of the world, (mathematical) models are still frequently based on a mechanistic view of systems. Since the 19th century, scholars have studied systems that are irreversible, stochastic, out-of-equilibrium and holistic. Yet, since the 1960s, many examples have been found in nature, such as pendulum movement, chemical reactions, weather dynamics, and population ecology, which could be better explained by the new tools from complex systems. Complex system research focuses on finding simple underlying rules of complicated dynamics. Complex adaptive system research is a sub section of complex system research that focuses on systems of various interacting components that lead to emergent phenomena. Instead of the use of elegant analytical mathematics, as in the Newtonian approach to research, complex adaptive systems research is more dependent on the use of computer simulation. The reason for this is that we are not able to solve many of the models of the more complex non-linear systems by analytical equations and thus we approximate these models by simulations.

Deductive and Inductive Reasoning

An important topic in understanding different ways we derive knowledge is the difference between inductive reasoning versus deductive reasoning. With inductive reasoning we refer to a process where the starting point is the observation of a phenomenon. Specific patterns and regularities might be found leading to some tentative hypotheses. In the end these hypotheses may become the basis of more generalized theories. Such an approach is called a “bottom-up” approach. For example, an ethnographer observes a tribe in the Amazon for a number of years and finds certain patterns. Later she may compare her findings with other hunter-gather communities, which may lead to a more generalized theory.

Observation → pattern → tentative hypothesis → theory

Many social and life scientists use the inductive approach. It focuses on observation in situations where one does not know what to look for. A trained social and life scientist can observe interesting and surprising behaviors that need explanation.

The deductive reasoning approach follows the opposite direction. One starts with a theory about the topic of interest, and defines hypotheses in order to test the theory. Controlled experiments may be defined and executed so that one can derive observations that one can compare with the predicted behavior. This may lead to confirmation or falsification of the original theory. An example of deductive reasoning is seen with an experimental social scientist who tests the prediction that humans will take into account the welfare of others in making decisions. An experiment is developed to test the theory in a controlled setting where the subjects have to make decisions. The results may confirm or falsify the original theory, and one can compare which theories best explain the observations.

Theory → hypothesis → observation → confirmation

Both approaches are equally valid methods of deriving knowledge although some disciplines are focused on induction (anthropology) while others are focused more on deduction (economics). It is very reasonable to assume one needs to use both approaches to understand complex social processes.

Also from a modeling perspective we can observe the two different approaches. Some scholars develop models that have a certain data set taken from a case study as their starting point. One first derives statistics from the empirical case and then develops a model

that may fit the data. When they have developed a model that fits the observations, this model may serve as the first step toward a theory that one can test in future case studies. The Bali irrigation case of chapter one is an example of this approach.

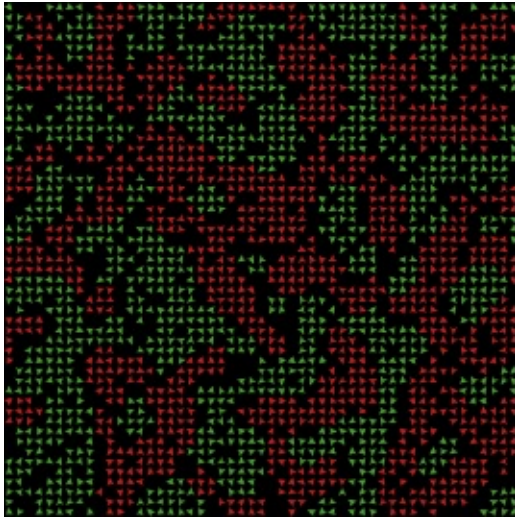


Figure 3. Screenshot of segregation model with a threshold of 50%.

The other approaches start with a general abstract model using rules based on existing rules. The question being asked with this kind of approach is what kind of patterns will evolve when the model is run. Sometimes we derive new insights by testing existing theories, or adjustments of existing theories, in new contexts as we can do with simulation models. For example, Thomas Schelling showed with a model of pennies and dimes that segregation of groups was possible even if agents were tolerant of living among agents of the other type. In the social sciences component of the NetLogo library you find a model of segregation based on [Thomas Schelling's](#) pencil and paper model. Red and green agents are randomly placed on empty cells of the screen. Not all cells will be occupied which makes it possible for agents to move to another empty cell. The idea derived from the model is as follows. Agents are happy to stay in the cell they are in if at least a fraction T_{min} of the agents in the eight surrounding cells are of the same color. Hence, the agents still tolerate agents of a different color, but may not want to be completely isolated from their own color. If an agent is not happy, it will move to a nearby empty cell, which is randomly drawn. The model starts with agents randomly placed on the screen. Every time step all agents evaluate in a random order their situation, and the model stops until all agents are satisfied.

The surprising finding of this model is that agents who tolerate diversity starts to become segregated. In Figure 3 we see an example with T_{min} equal to 50%. At the end of the simulation 86% of the neighbors of agents have the same color.

This model shows possible mechanisms that can explain observed segregation in the real world. Segregation is possible without a planned discrimination of certain agent types. It is not proof that actual segregation happens in this way. The model points to certain questions

researchers may want to take into account when they do study segregation. Can you point to some limitation of the model that may affect the insights that it derives?

Dynamics and Scale

The two hardest concepts in social science, maybe science in general, are dynamics and scale. In fact, we don't even have very clear descriptions of these concepts. If we look at a car, we can measure the speed of the car. In the world of Newtonian mechanics and objects that follow the laws Newton discovered, we can measure speed and acceleration with high accuracy. But in social systems change happens in people's minds. The change of beliefs, expectations, understandings or preferences all affect our decision making. Such changes can only be measured indirectly, for example via experiments and surveys. It does not help that our subjects of interest, in contrast to the atoms of Newton, reflect on their behavior. If we ask the same person to fill in the same survey one week apart, we likely get differences in responses, even if the respondent is filling in the survey truthfully.



Figure 4: Prediction is difficult, especially about the future.

What we can measure are census of citizens, sales statistics, email patterns, traffic flows, etc. as indirect information about the decisions people make. Using statistical models we can make predictions if people continue the behavior they did in the past. Statistical models cannot well capture what happens if behavior is changing. For example, will people react to information about a possible new pandemic in the same way they reacted to the news of the most recent Swine flu ([link is external](#))? In many models in social simulation we describe the decision process itself. This makes it possible to capture a larger diversity of behavior. We would be able to predict the future better if people did not change their behavior. Statistical models are superior for doing just that. But humans are able to anticipate possible behavior and change their behavior, making statistical models obsolete during periods of change. For social simulation scholars studying financial markets the recent economic crisis was not a surprise since they had observed that type of behavior—herding of decision makers leading to bubbles in markets—regularly in their simulation models.

Another question to consider when we study social systems is at what level do we describe the entities of the system? One can look at individuals, couples, families, firms, organizations, communities, nations, etc. Although these seem clear categories, in practice boundaries can be fluid. For all these levels we can look at entities that interact with each other. Of course actual systems consist of entities at different scales of organization. To

make it more complicated, a person might be a member of a family, a firm, an organization, a community and a nation, and have different roles within those entities. Suppose with two persons A and B, person A might be the chairperson of a sport organization in which person B participates, but person B is person A's boss in a firm.

Studying social systems is complicated since it is difficult to follow the changing behavior of a system at different levels of scale. When we develop models we will sometimes consider different scales, such as individuals and firms, but it will be a challenge to make models that include agents of different levels of scale.

The Modeling Process

How do you create a formal model? You don't start by sitting at a computer writing a program. Rather, it is useful to follow a set of actions that provides you with a more reasoned approach to the model design.

First you need to define the scope of your research. What is the question you want to address? You may brainstorm in a group or by yourself about what the future model needs to be able to do? Is it used to test a specific hypothesis, or is it used as a planning tool in collaboration with stakeholders? Do you want the model to be interactive with visual output, or to run the model many times for many different assumptions?

Suppose you are interested in studying why a movie becomes a hit. The question then becomes how to make a model that can predict whether a movie will become a hit based on the characteristics of that movie. Such a model can be developed from available statistics, and a statistical model seems to be an appropriate approach. However, if one wants to understand the “ecology” of movies, why one movie attracts lots of consumers while a similar movie does not, an agent-based model might be a more suitable approach.

One of the interesting statistics about movies is that most movies derive the highest earnings in their first week. After a few weeks the movie stops attracting viewers. If people go to a movie because they read good reviews of the movie, you would expect that with successful movies there would be an increase in visitors over the weeks. However, this is a rare pattern in the data. In Figure 5 we depict the U.S. returns of two movies that were released in the middle of November 2008. The James Bond movie [Quantum of Solace](#) earned half of the total earnings in the first week after its release (82 million dollars). The returns fell rapidly after the first week. This is in contrast to the Oscar-winning movie [Slumdog Millionaire](#) which started with very modest earnings. It gained momentum during the Christmas season and got an extra boost when it was nominated for 10 Oscars. The highest earnings were derived the week after the movie won the Oscar for best motion picture.

The pattern of [Slumdog Millionaire](#) is a very rare pattern in movie ecology. Most movies follow the pattern of [Quantum of Solace](#). Most people want to see a movie when it first becomes available. The same phenomenon, highest sales at the moment of release, is also found in other information products like books (consider the sales of the Harry Potter books).

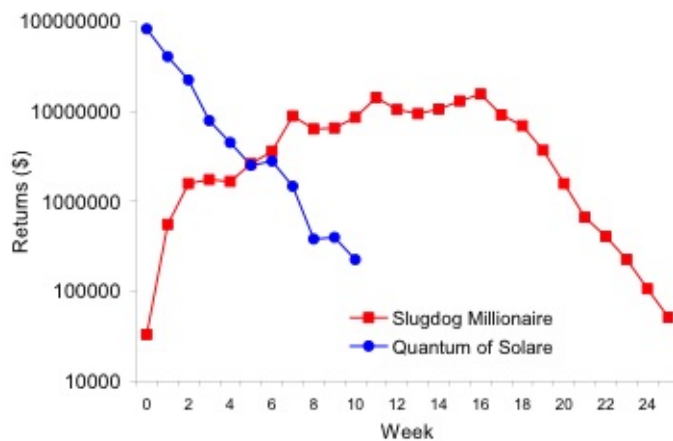


Figure 5. The returns in dollars on the U.S. market for each week in the cinema.

When you get some ideas of what you want to work through you can start to define the components of the model. It is important to “keep it simple, stupid.” So, try to identify only the most important components and processes. What are the agents? What are their attributes and their interaction with other agents and their environment?

For the movie example, one may consider at least individuals as potential moviegoers, movies, and producers of movies as agents. Individuals have a variety of preferences, limited time and resources, friends and family with whom they like to go to the movies, a distance from nearby cinemas, etc. Movies have different characteristics such as genre, actors, and a budget for movie and advertisement. Producers of movies need to make decisions on which kind of movies to produce and when to release them to derive a profit. You can add more and more details such as cinemas, actors, media and advertisement. The challenge is to find the right balance between creating a simple model and having enough content to address the questions you have. This will be an iterative process. You may think you have a simple model, but find out that the results of your model are difficult to understand, and you need to rethink how to simplify it even more.

After these steps, it is time to look at the literature. Have other people studied similar question to the one you want to address? You can use [Google Scholar](#), “[Web of Knowledge](#),” or look at specific simulation models to find relevant models. If you find a number of useful papers, find out what the pros and cons are of these other papers. What can you learn from these papers? What would you like to do differently? Now you can start to define more precisely the hypotheses you want to explore.

On the movie model there is a substantial literature across different disciplines such as economics (De Vany, 2003), and marketing (Elberse and Eliashberg, 2003). There are also a number of agent-based models developed already such as Delre et al., (2008)

Note that we use a deductive approach in our model design. An inductive approach is possible for social simulation in the following sense. You formulate a number of agents and their interactions and let the model run, observing what happens. Sometimes an explorative

approach is used, for example, by changing existing models and adding new processes to explore the consequences. Since the model process that accords the deductive approach is stricter, we will use this approach to discuss the modeling process.

Now you can become more specific on the model components. You may start developing a flow chart of the conceptual model, by which we mean a set of boxes representing variables, and arrows representing relationships. What are the specific attributes of the agents, the cells and other components of the model? What kind of learning model do you like to use? Do the agents derive information without observation error? Etc.

For the movie market model we need to start thinking about the interactions between the individuals, movies and producers (Figure 6). Potential moviegoers can be influenced by advertisements, the attributes of the movie and by the opinions and behaviors of other people in their social network. The producer needs to decide what kind of movie to make, how much to spend on advertisement and when to release it. If the producer has a good portfolio, will it survive the competition? Do producers specialize in certain movies or maintain a wide portfolio? What can explain the fact that after the first week of its release the movie will have attracted the majority of its audience?

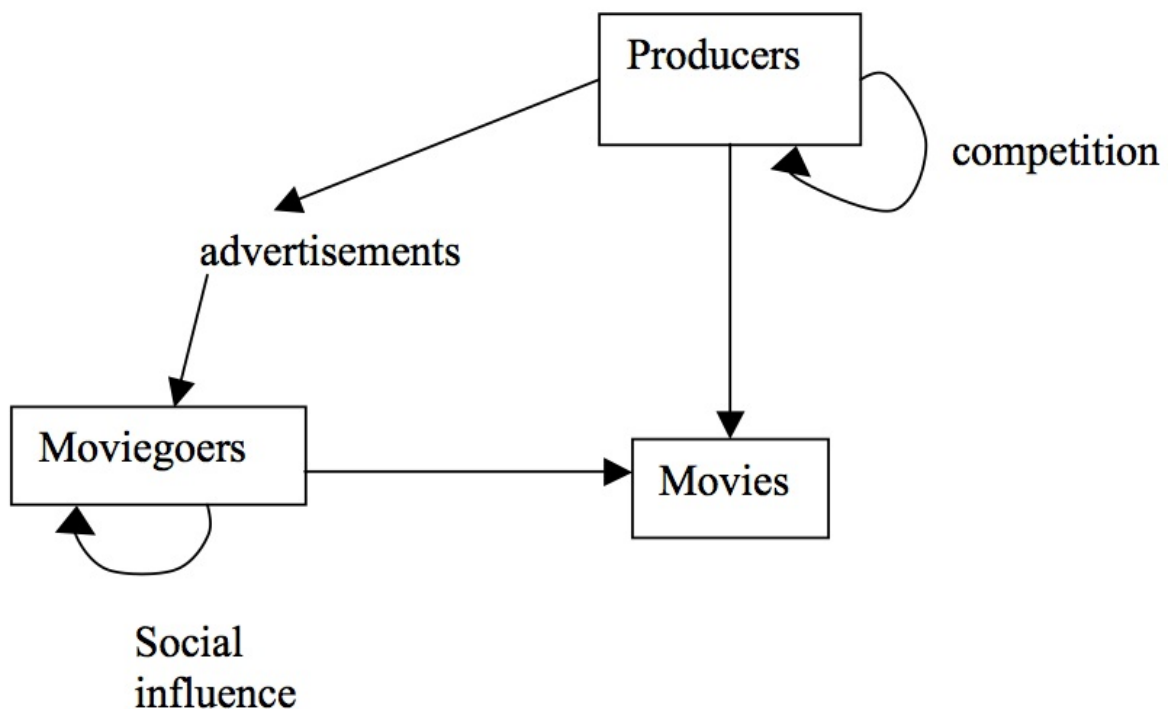


Figure 6. Flow diagram of a possible model of movies.

Developing flow diagrams leads to a detailed plan of action, and you can finally start the coding of the model. What platform do you want to use? The choice of platform depends on you and your collaborators' familiarity with the platform, the technical requirements of your model (visualization, connection with Geographic Information Systems), and the audience (students, stakeholders, other scientists). When you have decided on the platform, it is

useful to see if there are models available that use similar processes as your model. You can use such a model as your starting point. When you have implemented the model, it is important to test the code (verification). For example, you simulate situations for which you know the result, and confirm that your model produces the correct results.

When you have coded the model, it is important to write down the model specification on paper, including all the assumptions, relevant literature and parameter values you are using. This will be important when you start writing a paper about your model analysis later.

The main use of a computational model is the analysis of its behavior for different assumptions. In a mathematical model we can describe frequently the behavior of the system in closed form. This means that just a few equations describe all the possible situations of the system. This is not possible with simulation models. We can implement the behavior of a number of agents with just a few rules, but to understand what the consequences are we have to do simulations for quite a number of different parameter settings. We need to do a sufficient number of simulations to get a good grasp of the behavior of the system. We will discuss later how to use statistical tools to describe how many simulations and ways there are to explore the parameter space.

Summary & References

Summary

Models are simplifications of real systems, and to simplify we have to make assumptions. There are different ways to make these assumptions, and such decisions are made based on the training and discipline of the modeler. Models should not be seen as holy grails that predict the future but as tools to systematically explore different assumptions.

References

Delre, Sebastiano A., Thijs L.J. Broekhuizen and Wander Jager (2008). The effect of social influence on market inequalities in the motion picture industry. *Advances in Complex Systems* 11(2): 273-287.

De Vany, Arthur (2003). *Hollywood economics: How extreme uncertainty shapes the film industry*. London; New York: Routledge.

Elberse, Anita and Jehoshua Eliashberg (2003). Demand and supply dynamics for sequentially released products in international markets: The case of motion pictures. *Marketing Science* 22(3): 329-354.

The Logic of Programming

Key Concepts

In the chapter you will

- Be introduced to algorithms, loops and programs
- Learn that random numbers are not random in a computer
- See that the order in which you give commands to a computer matters

Introduction

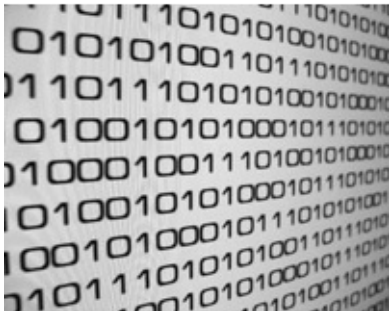


Figure 1. Computer programming.

A computer is basically a large number of electric switches that are either turned on or off. To program a computer we have to develop a way to communicate in binary code: 0 and 1. Computer languages like Java ([link is external](#)) or C++ ([link is external](#)) do this kind of translation for you. Nevertheless, it is helpful to know that a computer is just a large number of on/off statements. In order to get the computer to do what you want you have to be precise in the commands you give it.

This chapter provides a background to programming before we focus on NetLogo, the programming platform used in this book, in the next chapter.

Functions

Suppose we want to model the number of students at the university in order to investigate how many new classrooms need to be built in the coming years. We will need to predict how the number of students will change over time. Therefore, our main focus will be the number of students, which is represented as NoS. The number of students in 2000 is presented as NoS(2000). In 2001 some students graduated, others dropped out, and new students came to the university. To describe the number of students in 2001 we may have something like:

$$\text{NoS}(2001) = \text{NoS}(2000) - \text{Graduated} - \text{Dropped out} + \text{Transfers} + \text{New students.} * (1)$$

This relationship will be the same for each year. As such we can define a function for $\text{NoS}(t)$.

In more general term, most agent-based models describe how values of variables change during the iterations of the simulation. A *variable* is an attribute of a system that can change over time. With a system we refer to set of interacting or interdependent entities forming an integrated whole. With iterations we mean the execution of a set of instructions that are to be repeated.

Given is a variable of interest x , which might refer to the population size, level of financial capital or the level of natural resources. Such a variable is called a [state variable](#), and a dynamic model describes how the value of x changes over time. In the simulation models discussed in this course time is considered to be discrete. Hence, given a time step t for which $x(t)$ is defined, we need to define $x(t+1)$. To define the relationship between the current and the future state of the system, we formulate a difference equation:

$$x(t+1) = f(x(t)) \quad (2)$$

where $f()$ is a function of variable x . More specifically, the function $f()$ may consist of a number of [parameters](#) p , which determine the shape of the function:

$$x(t+1) = f(x(t), p) \quad (3)$$

Examples of parameters are the growth or death rate of a species, interest rate, etc.

Programs

In this course we will learn how to develop computer programs to simulate social phenomena. In this chapter we introduce some basic concepts of programming. A computer program consists of data and algorithms. Data refer to quantitative information of the problem, while an algorithm is a set of ordered steps to solve a problem. In fact, a program is like a recipe. Let's look at a recipe to prepare a [Reuben sandwich](#).



Figure 2. Reuben sandwich.

Ingredients

- ½ pound (225g) sliced corned beef
- ¼ cup (60ml) drained sauerkraut
- 2 tablespoons chopped sweet onion
- 2 tablespoons creamy Russian dressing
- 2 slices rye bread
- 1-3 slices Swiss cheese
- chopped parsley
- 1 tablespoon butter

Procedure

- Combine the sauerkraut, onion, and parsley
- Spread the dressing on two slices bread
- Pile layers of corned beef, cheese, and sauerkraut mixture on one slice
- Add second slice of bread
- Butter the outside of the bread
- Grill until lightly browned, roughly 5 minutes

The ingredients are like the input data for a program, while the procedure is the algorithm to make a Reuben sandwich given the input data. Additional data are used in the algorithm, such as 5 minutes in step 6 of the procedure, which is like a parameter value in a program. Although a recipe is like a program, different persons may generate different variations of the sandwich. With a computer program the instructions have to be clear enough so that each computer will produce exactly the same results. There is no room for different

interpretations. In practice this can make computer programming a time-consuming and frustrating experience, since you will get error messages or wrong results if you don't give the right commands.

Example of Algorithms: Sorting

Suppose you have the following list of numbers:

4 2 1 3

You are asked to develop an algorithm that sorts this list of numbers from small to large. This algorithm should also be potentially applied to any other list of numbers. What would such an algorithm be? A simple algorithm would be the so-called [bubble sort](#). This algorithm steps repeatedly through the list to be sorted comparing two items at a time, swapping them if they are in the wrong order. This process is repeated until a pass through the list does not lead to any more swaps. The list is then sorted.

****2 4** 1 3 swap**

2 **1 4 3 swap**

2 1 **3 4 swap**

****1 2** 3 4 swap**

1 **2 3 4**

1 2 **3 4**

****1 2** 3 4**

1 **2 3 4**

1 2 **3 4**

Bubble sort is a simple but very inefficient algorithm since it takes many repeats to go through the list to get it sorted. The duration of the algorithm grows exponentially with the length of the list. Another sorting algorithm invented by [John von Neumann](#), which is much more efficient, is [merge sort](#). It is based on the observation that short lists can be sorted quickly, and one may split up the list into smaller units, and then merge the smaller lists into a bigger list.

Thus for a list larger than 1 unit, divide up the list in two sub lists. Sort each sub list recursively by merge sort. As a consequence we first split a list into very small sub lists and then aggregate this up in steps to the original size of the list.

4 2 1 3

2 4 1 3

2 4 **1 3**

</dl> Now it compares the first of both lists.

****2** 4 **1** 3 →1 2**

****2** 4 1 **3** →1 2 3**

2 **4 1 **3** →1 2 3 4**

The computer had to do 9 comparisons with the bubble sort and only 5 with the Neumann algorithm. For larger lists the efficiency of Neumann's algorithm will become more prominent.

Components of Algorithms

We will now discuss a number of concepts that are used in developing algorithms.

If then else

In everyday situations you have many different choices to make. If the traffic light is green you cross the street, otherwise you wait. If you want to send a letter, you mail it with sufficient stamps, otherwise the letter will not reach the destination.

In programs we have many of these situations too, which are stated more formally as IF a condition is true THEN do this ELSE do that. This procedure is used to let algorithms do tasks that are dependent on the state of the system. For example, an agent will move randomly except when the agent sees food on the cell in front of itself. In this case the agent will move forward. In this example the situation is stated

IF food on next cell THEN move forward ELSE move randomly.

Another example shows how you can avoid dividing by zero. Given are variable x, and parameters a and b.

IF $a = 0$ THEN $x = b / a$ ELSE write "division by zero" Where $=$ means "not equal to."

For example, in bubble sort, you can have the following use of if then else:

```
IF a > b THEN
  c = b<br>
  b = a<br>
  a = c<br>
ELSE
  a = a
  b = b
```

where a and b are two items in the list that are swapped if $a > b$.

Boolean

This is a variable that is defined to be FALSE or TRUE.

A boolean variable has only two possible values, TRUE or FALSE. This means that if a variable x is not TRUE, it will be FALSE.

Suppose x is FALSE. What is the value of a in the following statement?

```
IF x THEN a=2 ELSE a=1
```

Since x is FALSE, a is equal to 1.

While loop

Sometimes you want to repeat a set of commands until a particular condition is met. For example, you are searching through a list of numbers until you find a particular value. You move to the next item in the list until you find the item you are looking for. You can implement this by using *while loops*.

Using the search example you can implement this as follows

```
position 1
WHILE a != seacha
{
    a = list [position]
    position = position + 1
```

where seacha is the value looked for. Recall that != means “unequal to.” One moves through the list, looking at the value of each item until seacha is found.

What will be the value of X in the following while loop?

```
A 1
WHILE A < 10
    X = A * 2
    A = A + 2
```

Answer, X will have the values 2, 6, 10, 14, 18 and then the while loop will be ended.

Recursion

Recursion implies a procedure that calls itself and it is commonly used in computer programs. A simple example is to calculate n! by the following procedure *factorial*

```
Factorial[N]
    IF N <=1 THEN fact = 1 ELSE fact = N * factorial[N-1]
```

Thus if we want to calculate 4!, we will start with Factorial[4]

Factorial[4] leads to fact = 4 * Factorial[3]

Factorial[3] leads to fact = 3 * Factorial[2]

Factorial[2] leads to fact = 2 * Factorial[1]

Factorial[1] leads to fact = 1

Then it jumps back to finish Factorial[2]: fact = 2 * 1 = 2

Then it jumps back to finish Factorial[3]: fact = 3 * 2 = 6

Finally it can finish Factorial[4]: fact = 4 * 6 = 24

Thus 4! = 24

Random numbers

Random numbers in simulation models are not truly random. Rather, they are generated by so-called [random number generators](#). In fact, the functions used to generate random numbers are deterministic and may therefore repeat the same sequence after tens of thousands of “random numbers.” An example is the [linear congruential generator](#) that generates sequences of numbers x by using the next equation:

$$x(t+1) = (A * x(t) + B) \bmod M$$

where the A , B and M are pre-selected constants. Note that M represents the upper level of the sequence of random numbers. A and B are defined in such a way that they meet desirable statistical properties. An example which is commonly used is $A = 1664525$, $B = 1013904223$, $M = 32$. Another number that needs to be specific is the seed, which is the value of x at the start of the sequence.

Suppose you start with $x = 1$, then the next step we get

$$x = (1664525 * 1 + 1013904223) \bmod 32$$

Since mod means modulo which finds the remainder of division of one number by another we get

$$x = 1015568748 \bmod 32 = 12$$

If we continue we get the sequence 1, 12, 27, 30, 5, 0, 31, 18, 9, 20, 3, ...

Any idea what the next number will be?

When you specify seeds in simulations, you can replicate the same results, even though you use random numbers. Sometimes the seeds are initialized using the computer's real time clock, which may provide enough randomness for simulation purposes, but the individual results can not be replicated.

The function `random()` in most programming languages produces a random number between 0 and 1 according to a uniform distribution. When the underlying sequence of random numbers falls between 0 and M , the random number is divided by M to produce a random number between 0 and 1.

In chapter 5 we will discuss randomness in more detail with various NetLogo models.

The order of commands

The order in which a program will execute the commands can be important. Suppose I have two agents. Each agent has a money account and the program gives one of the agents 1 dollar. What will happen in each of the following two algorithms?

Algorithm 1:

```
For Agent 1    [Account(t+1) = Account(t) + 1]
For Agent 2 [Account(t+1) = Account(t) + 0]
```

Algorithm 2:

```
Flip a coin. If heads [A = agent 1, B = agent 2] Else [A = agent 2, B = agent 1]

For A [Account(t+1) = Account(t) + 1]
For B [Account(t+1) = Account(t) + 0]
```

Object-oriented Programming

In this course we use NetLogo which is a modeling program specifically designed for agent-based modeling and is developed by using Java. Java is a so-called [object-oriented programming](#) language. NetLogo provides a user-friendly shell around Java. NetLogo is also an object-oriented programming language. To understand how to read and write programs, we need to discuss briefly what object-oriented programming means.

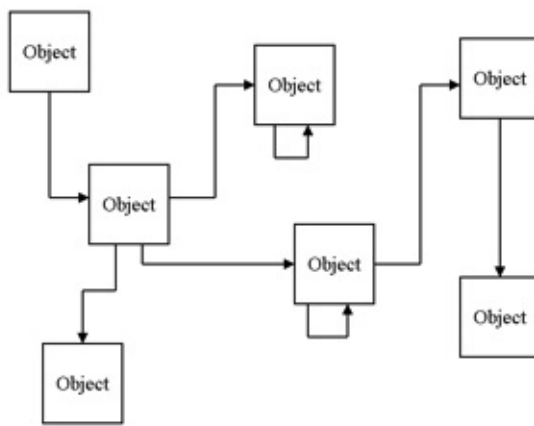


Figure 3: Flow diagram of objects in a computer program.

A program is build up by objects. These objects are little pieces of code that are activated during the simulation. For example, you make a model of ants walking around in an environment searching for food and bringing food back to the nest. Instead of making one long list of commands, you split it up into components such as: how to walk when searching, how to walk back to the nest, how to recognize food, how to drop pheromone, etc. These components are self sustained and can be called upon by different places in the program. In Figure 3 we give an example of a number of objects that are called upon in different places in the program. If the user starts the program, the object in the left top corner will be activated (in NetLogo the objects are called procedures, and the starting procedure is often called “go”). This object will call another object and based on what happens in the object, one of three routes is used. Some objects call themselves. This is an example of recursion.

In your NetLogo models you will see that you first need to activate “setup,” which is a set of calculations to clean up the memory and define the number of agents and cells, and the starting values of the variables. Only when this has happened, the procedure “go” can be activated, which will run the model through various iterations.

The Art of Programming

Programming can be a time consuming and frustrating process. Especially in the beginning the program never seems to do what you intended it to do. By practicing regularly you see that you make common mistakes (typos, forgetting a bracket [], etc.) and learn to debug your code better over time.

If you want to make a new program it will help to follow some best practices. It is important not to start programming immediately. First define the program and outline it in words or in a drawing. Once you have identified what components need to be included, then you can write your model in [pseudo code](#), which is a high-level description of your model. Once you have completed these steps, then you can start programming. If possible start with some component that you can test before you add another component. By the end of this process, your program should be work as you intend it to. A common understanding among programmers is that there are no programs without errors. Therefore it will be helpful to do a number of tests, such as running conditions for which you know the answer to see whether your model is accurate. You can also let somebody else work with your model. Another way to check you model is to write a clear documentation.

Summary

In this chapter we introduced some concepts that will we use during the rest of the course. Computer programs consist of algorithms. Algorithms are a set of commands you give to the computer. Computer programs are very strict since the commands are taken literally. It will take practice to learn to program. That is why we will do so many exercises in the coming weeks.

Introduction to NetLogo

Key concepts

In this chapter you will

- Be introduced to the programming language NetLogo
- Get practical examples to build your first models

Introduction

We use the NetLogo package to implement the agent-based models discussed in this course. This package can be downloaded for free from the [website](#). We use NetLogo version 5.0 in this course. The NetLogo website has a lot of information available for novices like tutorials and example models. We also advise you to check out the [YouTube tutorial](#).

In this section we discuss a few items to get you started.

Model Structures

A NetLogo model, like [other platforms for agent-based models](#), consists of algorithms that produce a sequence of commands to calculate changes of the attributes of the agents. First a model needs to be initialized, which means that all attributes of agents and environmental variables are defined and have initial values. In NetLogo, people typically use the procedure “setup” to do this. When you look at one of the many demo models in the NetLogo library, you see the button “setup” in most interfaces. Clicking on that button initializes all the relevant variables of the model.

When a model is initialized the model continues by calculating the updates of the attributes of the agents and the environment. This is typically started by clicking on the button “go” in the interface.

When you click on “go” you start a sequence of calculations. A model can be described as follows:

$$F(x, p)_t = F(x, p)_{t-1}$$

where F is a model, x the state variables of the model, and p the parameters of the model. The subscript t refers to the time step during the simulation, and $F(x, p)_0$ is the initialized version of the model.

Most models are built up on many different procedures. Procedure f may call procedure g and h . An important aspect of simulation models is the sequence in which procedures are called and thus the order in which variables of the model are updated. Suppose you have procedures *die*, *eat* and *metabolism*. Suppose you first call the procedure *die* to check whether the agent has sufficient energy. If not enough energy is available, the agent is removed from the model. Then the procedure *eat* is called, and the remaining agents derive more energy, before the procedure *metabolism* is called. Compare this sequence, with the case where first the procedure *eat* is called, then *die* and finally *metabolism*. You can tell that the order in which the agents are updated affect which agents remain in the model. Hence, the sequence in which you call the procedures is important.

The scheduling is not only important for the order in which you call procedures, but also the order in which you update the agents. Suppose you always update agents in the same order and they have to look for resources. Then the first agent has an advantage in finding available food, while the last agent may always experience a scarcity of resources. Such a

fixed order leads to artifacts in the model. Therefore you see that the order in which agents are updated is typically random. When in NetLogo you call `ask agents []`, agents are updated randomly.

First Steps with NetLogo

How do you learn to use NetLogo? First, you can follow the YouTube tutorial. Second, you can explore the large number of existing models in the NetLogo model library. You can explore the behavior of a model by running the model for different assumption, such as different values of parameters. Lets use the example of the Rabbits Grass Weeds model in the model library (File – Models Library – Biology).

In this model rabbits move freely around on the landscape and consume weeds and grass. The rabbits have a metabolism of say 0.5 energy units every time step. By eating grass and weeds the rabbits derive new energy. When the accumulated energy of an agent is above the birth-threshold, the agent will generate offspring. Generation of a child is implemented as adding a clone of the agent in a nearby cell, and providing half the energy of the parent to the cloned child. As a result the number of rabbits in the population evolves over time, as well as grass and weed, which reappear by a fixed chance on an empty cell.

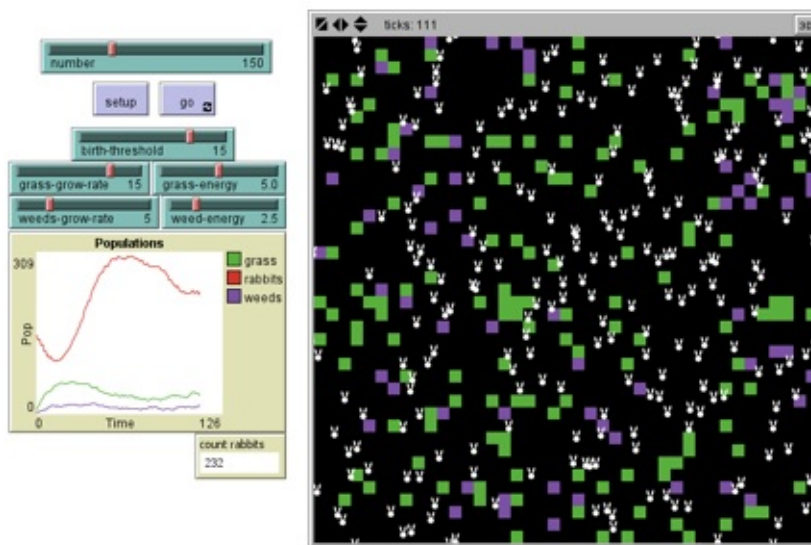


Figure 1: NetLogo Interface of the Rabbits Grass Weeds model

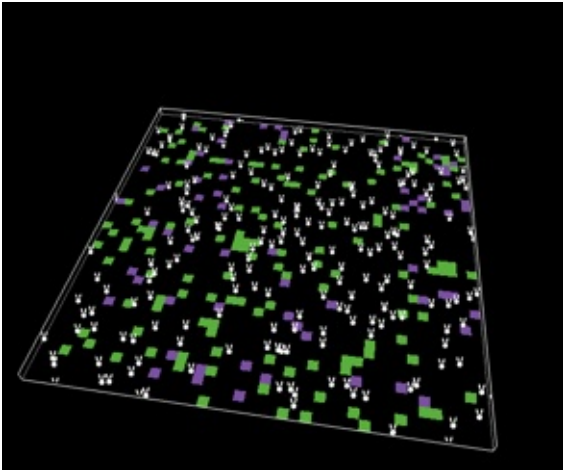


Figure 2: Example of a 3 dimensional view of the Rabbit Grass Weed model.

The basic user interface includes the “Interface” which gives the graphical output of the model (as depicted in Figure 1), the “Info” which provides the background information of the model, and “Code” which provides the source code of the model. The basic interface as depicted in Figure 1 shows more components. A few are important to mention: “File” provides the option to save the model, start a new model, or find a model in the model library. “Tools” include a number of helpful tools like “Shapes editor” to select a shape to represent an agent, “BehaviorSpace” to run a large number of simulations and store the results in a data file, and “3D view” which allows you to visualize the model output in 3D (Figure 2). “Help” includes a useful link to the user manual. In the detailed user manual there are discussions on a number of sample models, as well as helpful tutorials that go through the basics of NetLogo. The manual also includes detailed information on how to develop the interface as well as programming guidelines. Finally, the manual includes a number of tutorials on the use of features like “Shape Editor,” “Behavior Space,” “Sound” and “System Dynamics.”

To use the interface in Figure 1, you click on Setup, which will run the command to initialize the model. After this you can click on “go” to run the model. You can change the parameters during the simulation and see directly the effects.

Programming with NetLogo(1)

Learning to program in a NetLogo package is mainly a process by learning-by-doing. By working through tutorials and example models, you will become familiar with the basics of the language and where you have to look for solutions to the problems you encounter. A few basic elements of the NetLogo language are discussed in the following paragraphs. Agents are called turtles in NetLogo. NetLogo has been based on follow up languages of Logo, a procedural programming language developed in the 1960s. The idea in the development of [Logo](#) is to have a simple language to steer an agent or robot, the turtle. For example, a turtle with a pen strapped to it can be instructed to do simple things like move forward 100 spaces or turn around. The figure below shows you can create a square by having the following sequence of commands:

```
Forward 50  
Right 90  
Forward 50  
Right 90  
Forward 50  
Right 90  
Forward 50  
Right 90
```

Where Right 90 means that the turtle will turn right 90 degrees.

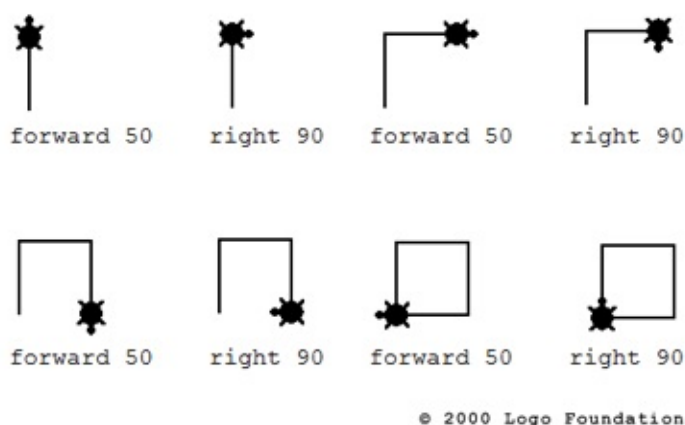


Figure 3. Logo.

From these building blocks you can build more complex shapes like squares, triangles, circles, etc. Packages like StarLogo and NetLogo simulate populations of turtles and provide them with many more commands. Turtles have coordinates (xcor, ycor), an orientation (heading), and they can move around in the environment of patches.

The cells of the spatial explicit environment are called patches. Patches have coordinates (pxcor, pycor), neighbors and a color. Patches can also include more information like whether it is empty, has an agent on it, or contains grass or weed (like in Figure 1).

There is also an observer perspective. From this perspective, you can define aggregates of the patches and turtles, and define the order in which certain commands need to be executed.

The program consists of a number of procedures which define the actions of the turtles and patches, and in which order they are executed. A procedure starts with `to` and ends with `end`. As an example we define below two example procedures `setup` and `go`.

```
to setup
  ca          ;; clear the world
  crt 10      ;; make 10 new turtles
end

to go
  ask turtles
  [ fd 1          ;; all turtles move forward one step
    rt random 10  ;; .. and turn their head a random amount
    lt random 10 ]
end
```

We recommend that you implement this model before you continue.

There are three main types of procedures in NetLogo:

Commands are a list of actions for the agents to carry out. A command always starts with `to` and ends with `end`, hence a typical command looks like:

```
to (commandname)

(actions)

end
```

In the following example, the name of the command is `draw-polygon`, and it uses values of global variables `num-sides` and `size`. The other actions (`pd`, `repeat`, `fd`, `rt`) are primitives, as discussed below. Look up the meaning of these primitives to understand what the command `draw-polygon` is doing.

```

to draw-polygon
  pd
  repeat num-sides
  [      fd size
    rt (360 / num-sides)]
end

```

Reporters are some specific commands. They report the value of a function with a specific input. Sometimes you have a set of actions that you use in different parts of the model. In such a case it is useful to define this sequence as a function. The general layout for a reporter is

```

to-report (namereporter) [ inputvariables ]

(actions)

end

```

See for example a function that defines the absolute value of a number,

```

to-report absolute-value [number]
  ifelse number >= 0
  [ report number]
  [report 0 - number]
end

```

Primitives are built in reporters or commands. The developers of NetLogo have defined a number of common reports and commands that many people use, and then build in directly into NetLogo. If you look at the primitives directory at <http://ccl.northwestern.edu/netlogo/docs/dictionary.html> you see a large set of primitives like `turtles`, `turtles-here`, `ask`, `clear-all`, etc. Learning which primitives are available and how to use them will take some practice. The best way to learn this is by looking at example models and see how other models have implemented certain problems.

Variables that are globally defined can be used everywhere in the program and are defined at top of the program

```

globals [ parameter ]

```

or are defined as a slider or switch in the interface.

Also the attributes of turtles and patches are defined globally:

```
turtles-own [attribute]
```

```
patches-own [attribute]
```

To update a variable you use the command `set`, for example

```
set year year + 1
```

You can also define a local variable within a procedure by

```
let variable 0
```

Note that a local variable can only be used within a procedure.

To update a variable for all turtles use the command `ask`.

You can update the color of all turtles

```
ask turtles [set color red]
```

or of all patches

```
ask patches [set pcolor red]
```

You can also change the color of the patches on which there are turtles

```
ask turtles [set pcolor red]
```

or a specific turtle

```
ask turtle 5 [set color green]
```

or

```
set color-of turtle 5 green
```

or patch

```
ask patch 2 3 [set pcolor green]
```

A more complicated procedure is

```
ask turtles
[
  if (attribute > threshold)
  [
    set attribute attribute - 1
    fd 1
  ]
]
```

In the procedure above the agent is not doing anything if the attribute is less than the threshold. In the following procedure we adjust the earlier procedure to include that agents make a different decision if the attribute is smaller or equal to the threshold. We also will visualize this in the graphical interface by giving the turtle different colors for each condition.

```
ask turtles
[
  ifelse (attribute > threshold)
  [
    set attribute attribute - 1
    set color white
    rt random-float 360
    fd 1
  ][
    set color red
    rt random-float 90
    fd 2
  ]
]
```

Agentsets are a subset of the agents, for example all red turtles

```
turtles with [color = red]
```

or all the red turtles on the current patch

```
turtles-here with [color = red]
```

Another example is all the patches at the right part of the screen

```
patches with [pxcor > 0]
```

From the perspective of the “caller,” a turtle of patch, you can define an agentset of all turtles within a certain radius

```
turtles in-radius 3
```

Defining four neighboring patches (north, south, east and west)

```
patches at-points [[1 0] [0 1] [-1 0] [0 -1]]
```

but this can also be defined easier with a primitive

```
neighbors4
```

What are agentsets for? Well you may want to update a specific set of agents, like

```
ask [...]
```

or check whether there is a specific agentset

```
show any?
```

or count how large the agentset is

```
show count
```

A specific example is the following where the turtle with the lowest fitness is removed

```
ask min-one-of turtles [ fitness ][ die ]
```

Special agentsets are breeds. In fact you define them typically in the setup of your model, like

```
breed [consumers consumer]
```

If you look into the primitive dictionary you see a number of specific primitives for breeds like

```
create-<breed>  
create-custom-<breed>  
<breed>-here  
<breed>-at
```

So that you can use it like

```
ask turtle 100 [ if breed = consumer [...]]
```

or, if you want to change the breed of a turtle

```
ask turtle 100 [ set breed consumer ]
```

As mentioned before, learning to use NetLogo is a matter of practicing, for example studying the example models we use, or reimplementing models discussed in papers.

Programming with NetLogo(2)

Let's start with the implementation of a model of termites making piles of wood chips. You start a new NetLogo model and first create the interface. You add to buttons "setup" and "go." The setup button is at the observer level and will be executed once. The go button is at the observer level and goes forever

Also add two sliders: "number" with a minimum of 1 and a maximum of 300 with a stepsize of 1, and "density" with a minimum of 0 and a maximum of 100 with a stepsize of 1.

Next, you will go to the procedures tab, and define setup as follows

```
to setup
  ca
  setup-chips
  setup-termite
end
```

Randomly strew yellow wood chips (patches) with given density

```
to setup-chips
  ask patches [ if random-float 100 < density
    [set pcolor yellow ] ]
end
```

Randomly position given number of white termites

```
to setup-termite
  create-turtles number
  ask turtles [set color white
    setxy random-pxcor random-pycor]
end
```

Before you continue, click on the setup button in the interface. Does it work?

Now we continue with the go command. When you press on go, the termites (turtles), will continuously execute three rules:

- look around for a wood chip and pick it up
- look around for a pile of wood chips
- look around for an empty spot in the pile and drop off the chip

```
to go
  ask turtles [
    pick-up-chip
    find-new-pile
    drop-off-chip
  ]
end
```

The pick-up-chip command is defined as follows,

```
to pick-up-chip
  while [pcolor != yellow]
    [explore]
  set pcolor black
  set color orange
end
```

where explore command is defined as

```
to explore
  fd 1
  rt random-float 50 - random-float 50
end
```

Find a new pile is defined as

```
to find-new-pile
  while [ pcolor != yellow]
    [ explore]
end
```

Finally, drop off chip is modeled like

```
to drop-off-chip
  while [ pcolor != black]
    [explore]
  set pcolor yellow
  set color white
  fd 20
end
```

Click on the “go” button in the interface. What is happening? If you click on the “Settings” and change the dimensions of number of patches on the screen to a larger number, say 100x100 patches. Will the same results appear?

Summary & NetLogo Primitives

Summary

This chapter provides an initial overview of NetLogo and instructions on how to start programming. In the video lecture of this week we will go through various examples in which you see how to create a model in NetLogo.

NetLogo Primitives

clear-all(ca)
create-turtles(crt)
forward(fd)
ask
right(rt)
left(lt)
pen-down(pd)
repeat
to-report
report
if
ifelse
turtles
turtles-here
globals
turtles-own
patches-own
set
let
color
pcolor
random-float
with
pxcor
pycor
in-radius
at-points

neighbors4

show

any?

count

min-one-of

breed

setxy

random-xcor

random-ycor

Randomness

Key Concepts

In this chapter you will:

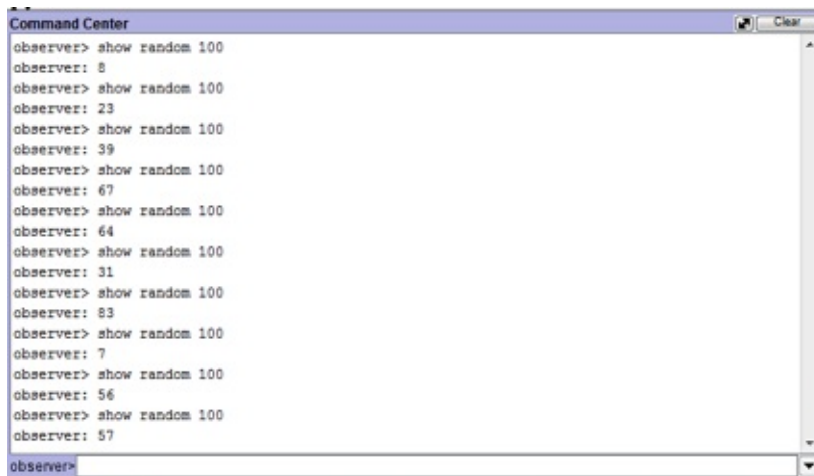
- Learn how to generate random numbers in NetLogo
- Experience the difference between random updating and updating in a fixed order
- Learn to use Behavior Space.

Introduction

In Chapter 3 we discussed how random numbers in computer programs are generated by a deterministic function. In this chapter we will look in detail at various examples of randomness in NetLogo, how to implement this in NetLogo and what the consequences are of different distributions.

Random Numbers in NetLogo

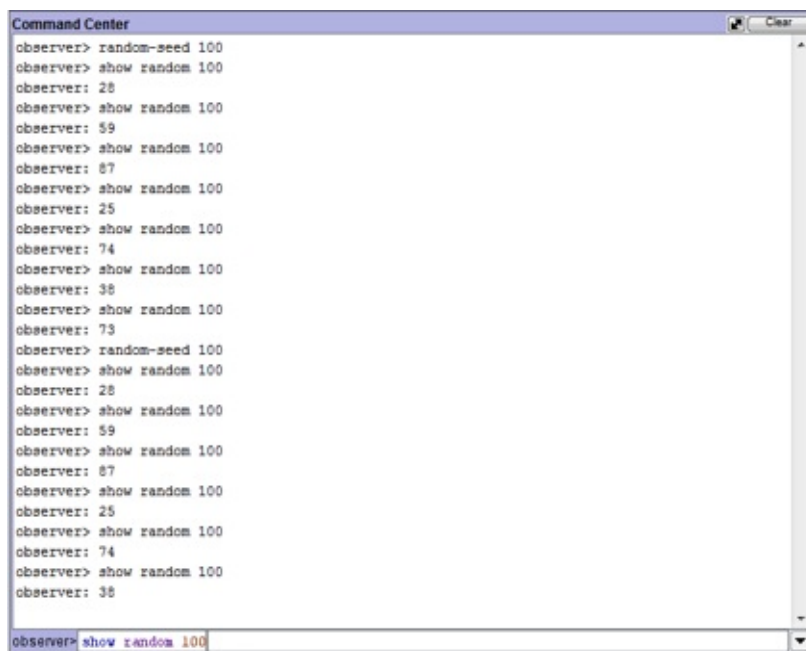
The basic command in NetLogo to generate random numbers is `random.` `Random 100` means that an integer value is randomly drawn from the series 0, 1, 2, ...97, 98, 99. If you type in `show random 100` then you will get a different value every time.



```
Command Center
observer> show random 100
observer: 8
observer> show random 100
observer: 23
observer> show random 100
observer: 39
observer> show random 100
observer: 67
observer> show random 100
observer: 64
observer> show random 100
observer: 31
observer> show random 100
observer: 83
observer> show random 100
observer: 7
observer> show random 100
observer: 56
observer> show random 100
observer: 57
observer>
```

Figure 1.

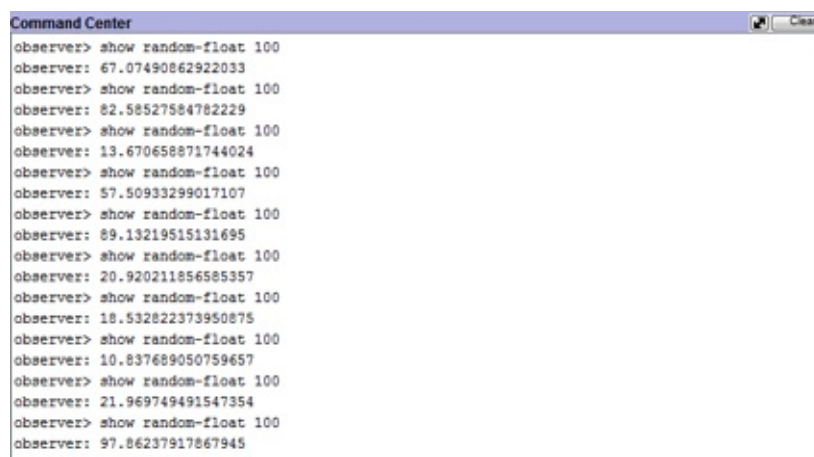
As we discussed in Chapter 3 a random number is an outcome from a deterministic function. If you start the function with the same number you will get the same sequence as we show here:



```
Command Center
observer> random-seed 100
observer> show random 100
observer: 28
observer> show random 100
observer: 59
observer> show random 100
observer: 87
observer> show random 100
observer: 25
observer> show random 100
observer: 74
observer> show random 100
observer: 38
observer> show random 100
observer: 73
observer> random-seed 100
observer> show random 100
observer: 28
observer> show random 100
observer: 59
observer> show random 100
observer: 87
observer> show random 100
observer: 25
observer> show random 100
observer: 74
observer> show random 100
observer: 38
observer> show random 100
```

Figure 2.

If you want to have real numbers instead of integer numbers, you have to use `random-float.` With `random-float 100` the numbers will be between 0 and 100 as shown below



```
Command Center
observer> show random-float 100
observer: 67.07490862922033
observer> show random-float 100
observer: 82.58527584782229
observer> show random-float 100
observer: 13.670658871744024
observer> show random-float 100
observer: 57.50933299017107
observer> show random-float 100
observer: 89.13219515131695
observer> show random-float 100
observer: 20.920211856585357
observer> show random-float 100
observer: 18.532822373950875
observer> show random-float 100
observer: 10.837689050759657
observer> show random-float 100
observer: 21.969749491547354
observer> show random-float 100
observer: 97.86237917867945
```

Figure 3.

Accidental Millionaire

Can you become a millionaire by chance? Here we show a simple model suggesting that it is possible. Suppose that there are 10,000 agents who make guesses whether the stocks will go up or down. Each round they make a guess by flipping a coin. The stock market goes up and down and its direction is also determined by flipping a coin.

When agents make a guess, they pay a price p to the pool x . The agents who correctly predicted the direction of the market will get an equal share of the pool.

How do we implement this model in NetLogo? Each tick the agents first pay a fee to the pool and determine their guess by flipping a coin. Flipping a coin is modeled by `random 2`, which provides values 0 and 1 with equal probabilities.

```
ask turtles [  
  set earnings earnings - price  
  set guess random 2  
]
```

The size of the pool is equal to the fee times the number of agents. Whether the stock market goes up is determined by flipping a coin.

```
let pool nr-agents * price  
set stockmarket random 2
```

Now for all agents we check whether their guess coincides with the direction of the stock market, if so, the agent has a correct prediction.

```
ask turtles [ifelse guess = stockmarket [set correct? 1][set correct? 0]]
```

Those agents who correctly predicted the direction of the market earn an equal share of the pool. We test whether the number of agents who made correct predictions is larger than zero to avoid the unlikely possibility that we divide by 0, which is not allowed.

```
let poolshare 0  
if (count turtles with [correct? = 1] > 0) [set poolshare pool / count turtles with [c  
orrect? = 1]]  
ask turtles [  
  if correct? = 1  
  [set earnings earnings + poolshare]  
]
```

So what is the outcome we can expect? On average the earnings will be zero. There is no money coming into the system. As we can see in the simulation of 10,000 agents after 100,000 ticks, the distribution looks like Gaussian distribution

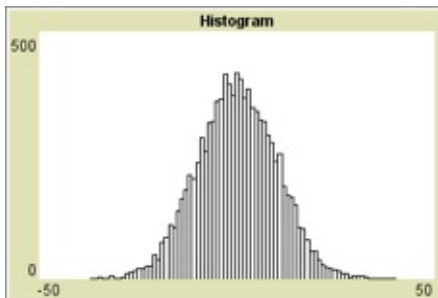


Figure 4.

However, if we look at the maximum earnings of agents in the population, we see that this maximum keeps going up. In fact it looks like one of the agents is very lucky and earns a high amount.

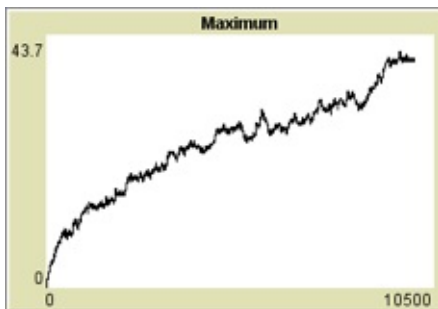


Figure 5.

NETLOGO EXAMPLE: RANDOMNESS

Model: Right click on the Link and Save it. Open Netlogo and Run it with NetLogo.

Random Movements

There are different ways to model movement of agents on a landscape. Hereby we present a simple movement model and show the consequences of different assumptions.

The model is as follows. An agent moves straight with probability ps and if it does not move straight it will turn right with a random number of degrees and it will turn left with a random number of degrees (D). Thus, there are 2 parameters that define the movement.

The basic code in NetLogo looks like.

```
ifelse random-float 1 < probstraight [
  fd 1
][
  rt random degrees
  lt random degrees
  fd 1
]
```

Different values of D and ps lead to very different behavior. In the table below you see 9 possible patterns. If D is 20 and ps the agent is mainly moving straight making small adjustments. If D is 360 and ps the agents is moving around randomly in a small area.

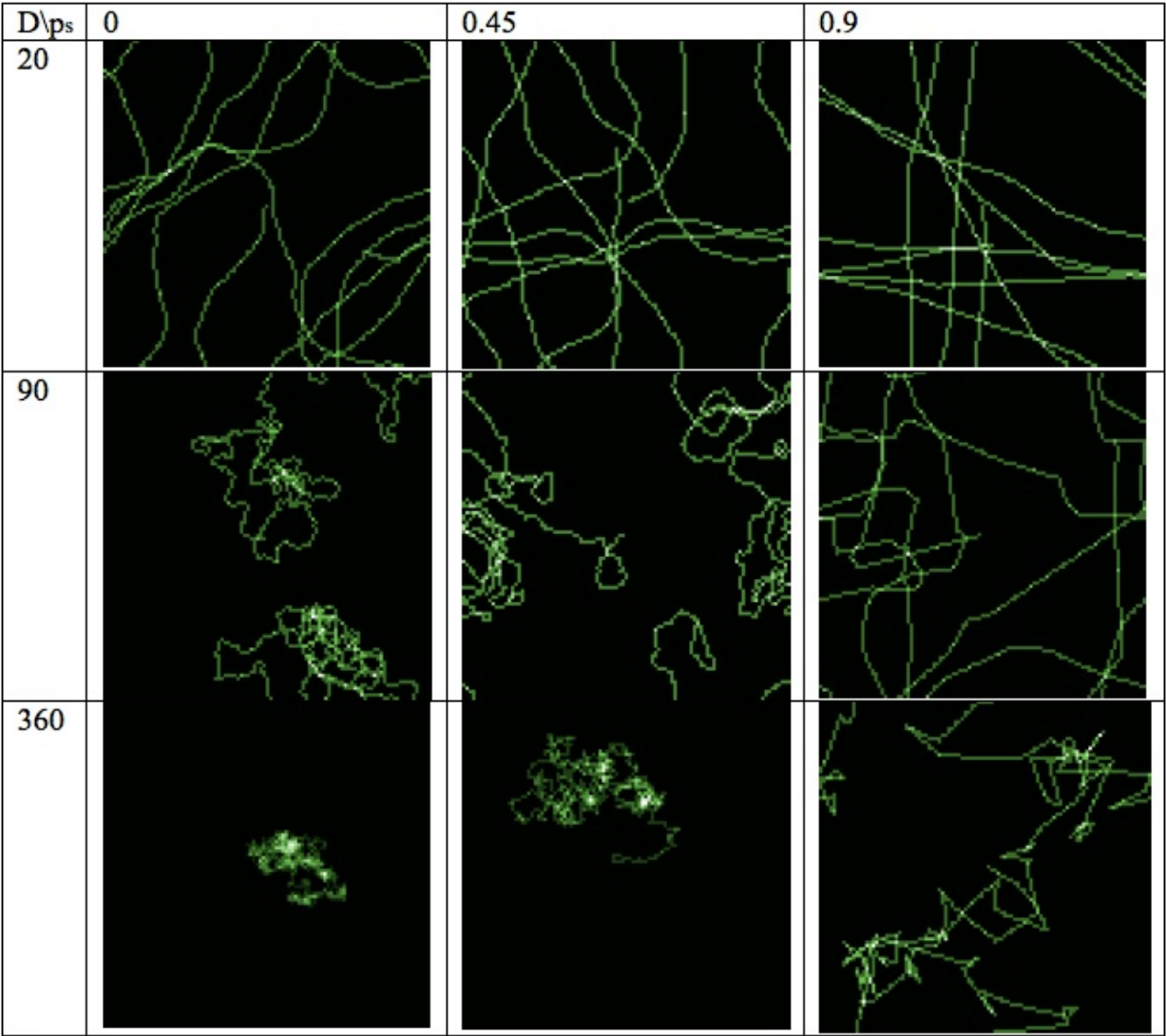


Figure 6.

NETLOGO EXAMPLE: RANDOM MOVEMENT

Model: Right click on the Link and Save it. Open Netlogo and Run it with NetLogo.

Importance of the Sequence of Updating

When you implement agent-based models, you need to specify the order in which the attributes of the agents are updated. There are different ways you can define the order, and the choice of the method for updating can have important implications on the results of the model. We will discuss the main updating techniques.

A simple approach for agent-based models is synchronous updating. Here all agents are calculated for time step t using the information of all the cells in time step $t-1$. If one has N agents, then the order in which these N agents are updated is not relevant since only information of the previous time step is used.

In most real systems updating of information is not synchronous. Instead it looks more like a continuous process where some agents make decision earlier than others. This is what we call asynchronous updating. Now the order in which the N agents are updated becomes important. Suppose you update the states of agent i and agent j . Since decisions of agent j depends on the states of agent i , and visa versa, one may derive different decisions when first agent i is updated and then agent j , or the other way around. Therefore, one distinguishes different orders.

One can use a fixed order of updating. For example, you may update agent 1 up to N always in the same order. This can lead to artifacts. Suppose agents are looking for resources, and agent 1 always has the first choice, and agent N the last. This will definitely favor agent 1.

Due to this artifact, which might be undesirable, a common approach is random asynchronous updating. Each time step, agents are updated in a random order. This leads to the case that sometimes agent 1 is updated before agent N , and sometimes agent N before agent 1. Each agent gets equal opportunities on average due to the random order of updating.

An example of the importance of updating is illustrated by the model of spatial games of Martin Nowak and Robert May (1992). They developed a spatial model where the environment is a 99×99 square lattice with fixed boundaries. Each cell represents an agent who plays rounds of a Prisoner's Dilemma game with its eight surrounding neighbors. Each round an agent has to decide to play C (cooperate) or D (defect) against all eight neighbors. The payoff of the Prisoners Dilemma is 1 when an agent and her neighbor cooperate, and $b > 1$ when the agent defects while the neighbor cooperates. In the other two cases, mutual defection and cooperation by the agent and defection by the neighbor, leads to a zero payoff. The sum of all games played with the neighbors is the total payoff for the agent. If agents chose only what is in their best interest they would always defect. But we will see that

in this spatial game there are situation in which cooperation emerges. The way the model updates the agents affect the results, and that is why it is a good example to show the importance of how agents are updated.

At the beginning of each step the agent will copy the best strategy of the eight neighboring agents. When one starts with all cooperators, except the agent in the center, you will derive interesting patterns of cooperators and defectors (Figure 7). Cooperation and defection can coexist in a spatial setting.

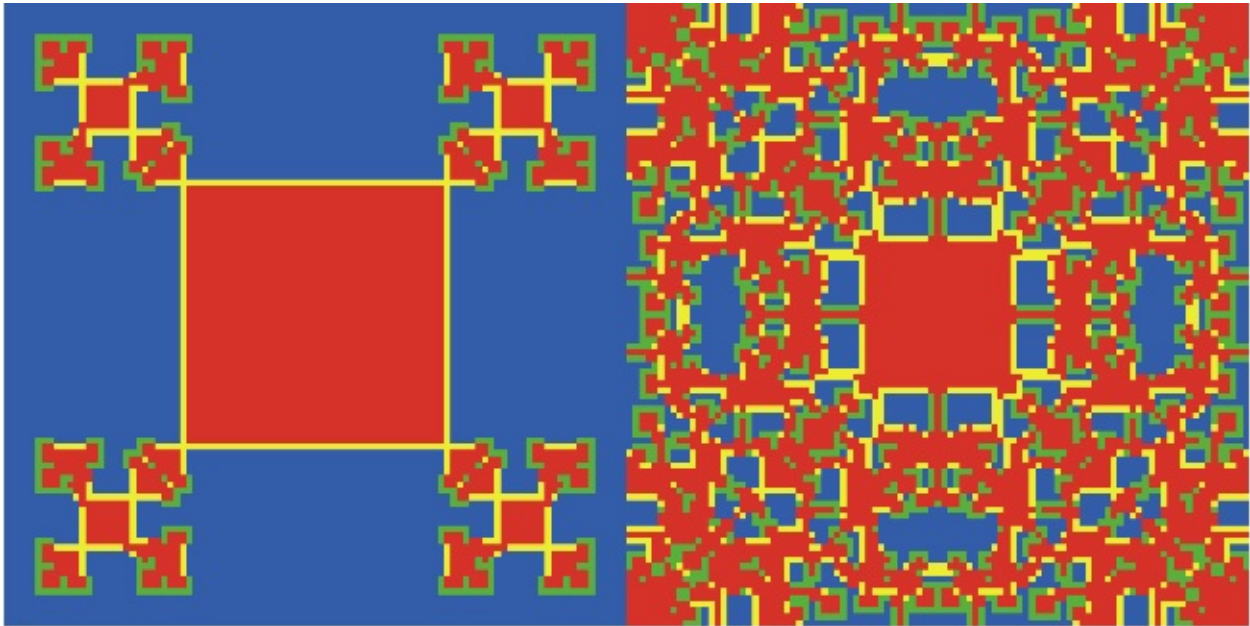


Figure 7: *Spatial patterns of cooperators and defectors. The color of the cell relates to strategy of the agent during the last 2 time steps. The cell is red when D follows a D, is blue when C follows a C, is green when D follows a C, and is yellow when C follows a D. The figures above are two snapshots of spatial patterns when the initial screen is all cooperative, except a defector in the center of the screen. (t about 20, and t about 100)*

Huberman and Glance (1993) criticized the findings of Nowak and May (1992) by showing that the results are sensitive to the way information is updated. When agents do not update their strategies all at the same time, but asynchronously, the defector strategy will spread rapidly (Figure 8). Hence changing the updating from synchronous to asynchronous leads to a stable solution of all defection.

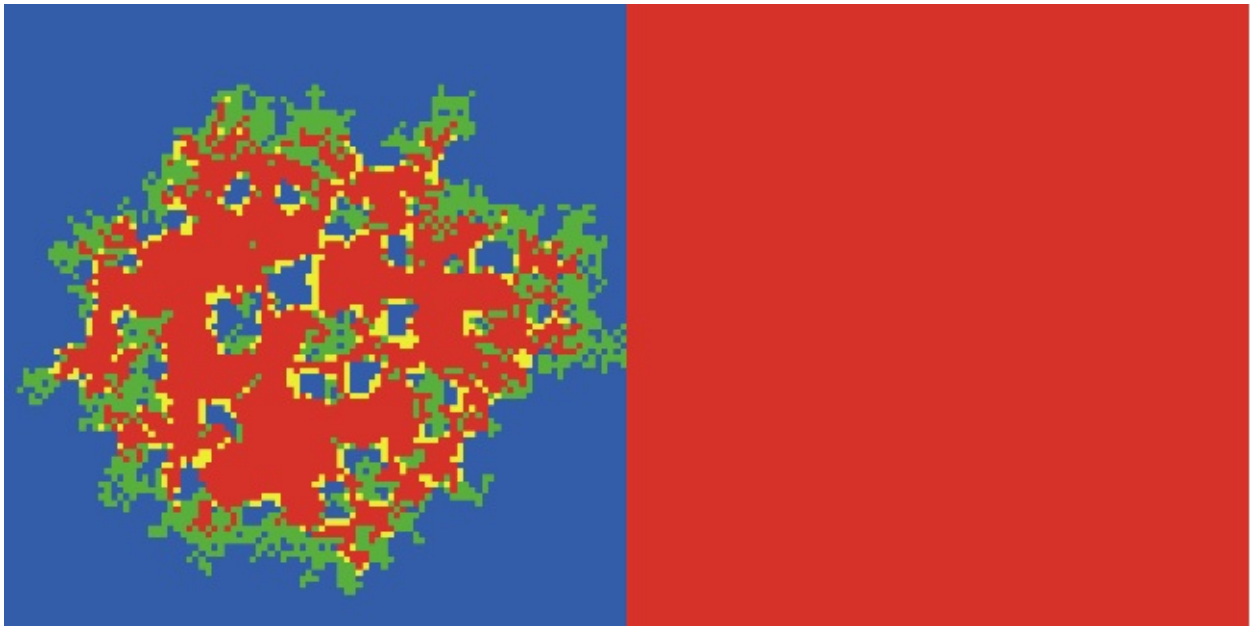


Figure 8: *Spatial patterns when updating is asynchronous.*

There are a number of other sensitivities we can show with this model. Nowak and May (1992) used a neighborhood of eight neighbors and the agent itself for determining the score and assumed that agents would also play a game against their own strategy. Thus the score is the results of 9 games. Playing a game against your own strategy is a questionable assumption. Suppose we assume that the score is only determined by 8 games against the neighbors, thus excluding a game against one's own strategy. In that case, defectors will start dominating too (Figure 9).

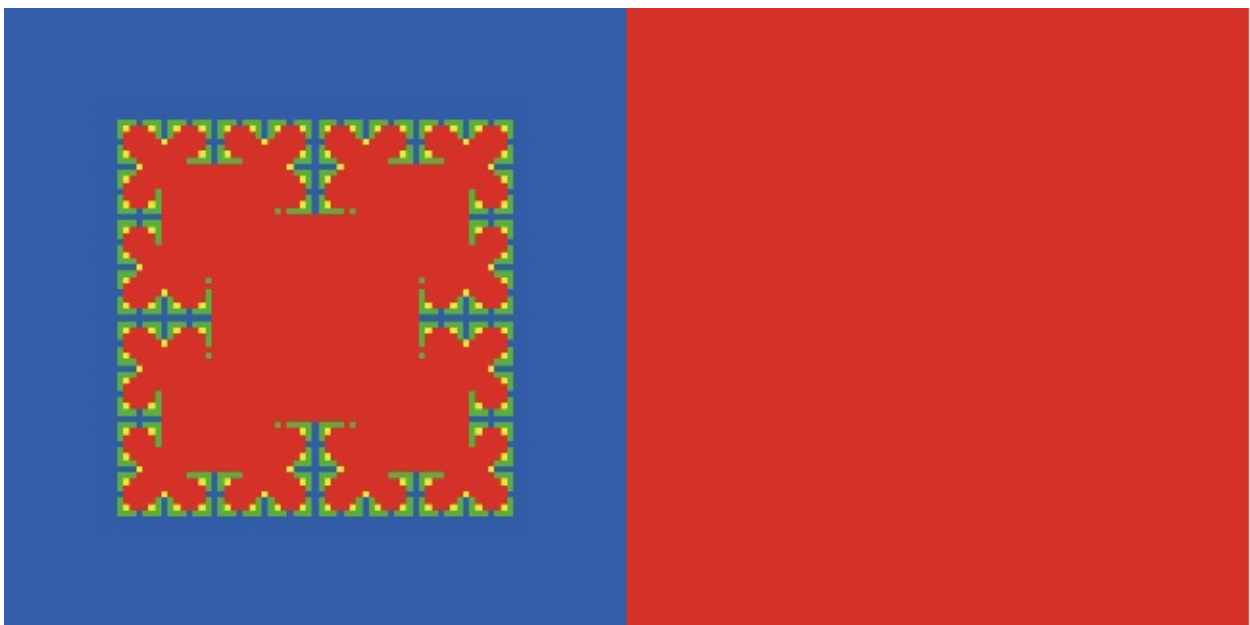


Figure 9: *Spatial patterns when updating is synchronous and agents do not play games against themselves.*

Finally, we show the results in the case where agents determine the score of the 9 games against neighbors and their own strategy, but they do copy the strategy of the best 8 neighbors, even when their own strategy is better. This leads to a co-existence of

cooperation and defection strategies (Figure 10).

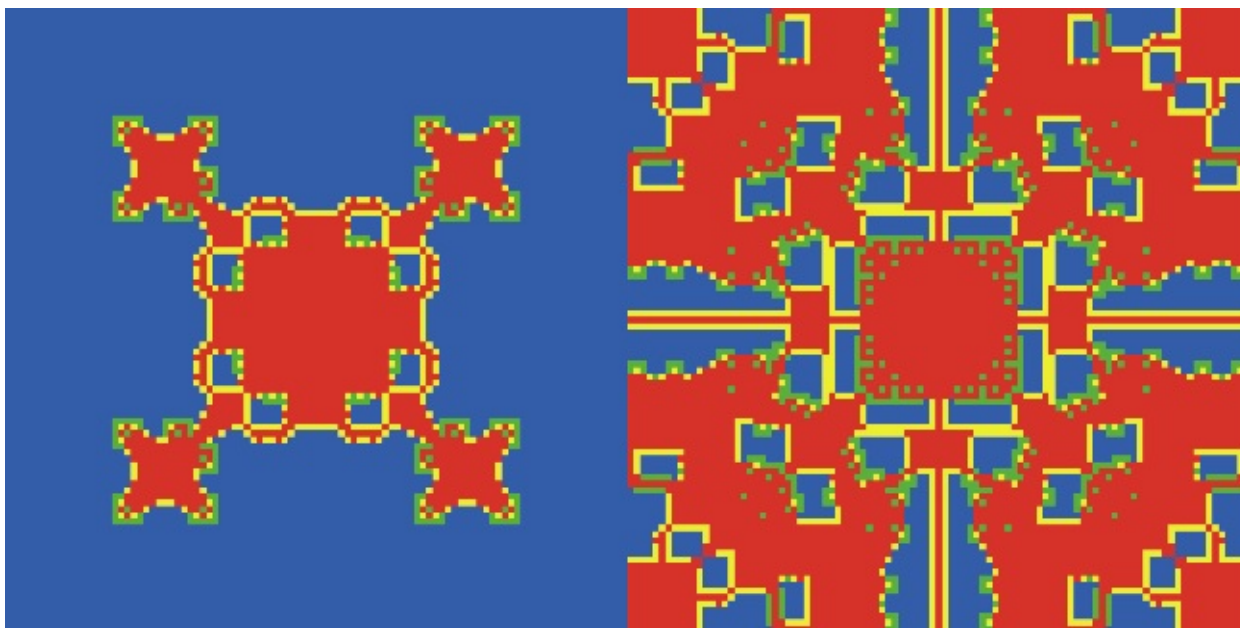


Figure 10: *Spatial patterns when updating is synchronous but agents imitate always the best neighbor, even when this neighbor has not a better payoff.*

NETLOGO EXAMPLE: SPATIAL VARIATIONS

Model: Right click on the Link and Save it. Open Netlogo and Run it with NetLogo.

Different Distributions

There are various types of distributions in NetLogo:

```
random-poisson mean
random-normal average stdev
random-exponential average
random-gamma alpha lambda
```

with alpha equal to $\text{average} * \text{average} / \text{variance}$ and lambda equal to $1 / (\text{variance} / \text{average})$

If we run the model for 10,000 ticks and draw a random number from each distribution we get the following distributions.

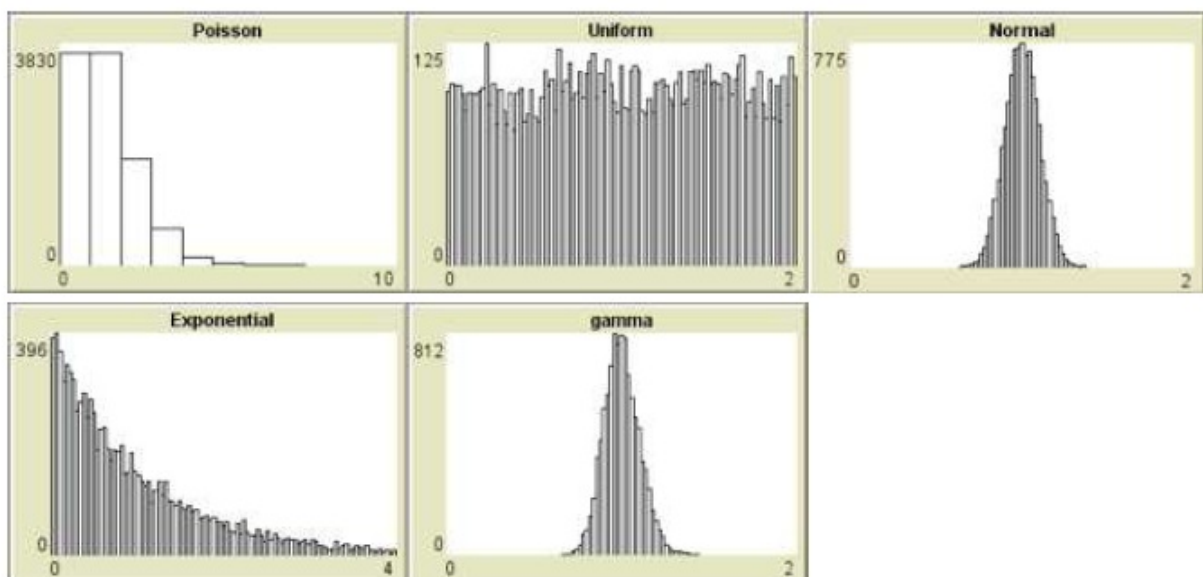


Figure 11

The average of the distributions converge to 1 as seen in the Figure 12 below. It takes a few hundred observations before the mean becomes close to one. This shows that it is important to have many observations to do models with random numbers. Each run is different and to have an informed decision on the results of the model, you have to run the model many times. Luckily NetLogo has a tool that makes it convenient to run a model many times and store the results in text files for further analysis. We discuss this in the next section.

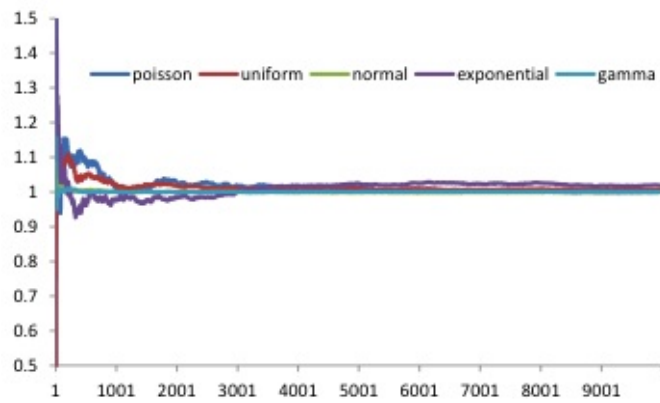


Figure 12. The mean of observations of 5 distributions. On the y axis is the number of observations generated.

NETLOGO EXAMPLE: RANDOM DISTRIBUTIONS VARIATIONS

Model: *Right click on the Link and Save it. Open Netlogo and Run it with NetLogo.*

How to use BehaviorSpace

BehaviorSpace is a tool of NetLogo to automatically run a large number of simulations and collect the data that you are interested in in a text file and/or Excel file.

BehaviorSpace can be found in NetLogo under Tools. If you click on BehaviorSpace you will see the following:



Figure 13

Before we go further, it would be useful to get a model from the library so that we can run some experiments for an actual model. We will select PD N-Person Iterated from the NetLogo library. There you will find:

Social science -> unverified -> Prisoner's Dilemma-> PD N-Person Iterated.

This model simulates a population of agents who, every timestep, are randomly partnered up with another agent to play a [prisoner's dilemma](#) game. Each agent has a strategy. In the interface you can determine the amount for each of the 6 strategies. The strategy unknown is meant for you to try out new strategies. As a default it is a [Tit-for-Tat](#) strategy.



Figure 14.

If you run the model, you will see that the strategies have different average payoffs in the longer term. Actually the defect strategy is found to give the highest amount of payoff.

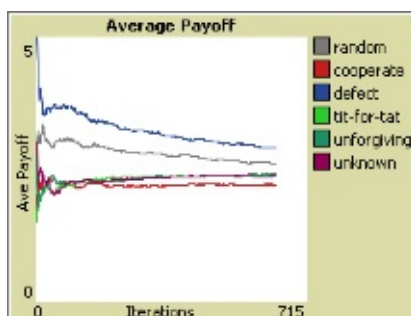


Figure 15.

Suppose we want to explore the results for a number of different distributions of strategies. Will defect always get the highest payoffs?

Instead of doing this manually, we will use BehaviorSpace. Go again to BehaviorSpace in “tools.” When the following screen pops up:

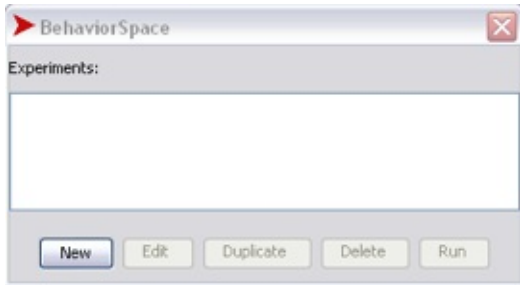


Figure 16.

click on “new.” We will now generate the settings for running an experiment. In the following figure we see that the global variables (including the sliders on the screen) are listed in the list of variables you can vary. The values given are the default values. We also see the number of *repetitions* we want to run the model for each parameter setting.

Then we see the information we will save in a text file or Excel file. As a default this is “count turtles.”

We can store information for each timestep or only the results at the end of the simulation (*Measure runs at every timestep*).

The setup commands define that the command *setup* will setup the model.

The go commands define that the command *go* will let the model run forever.

The time limit is the maximum number of ticks that we will allow the model to run.

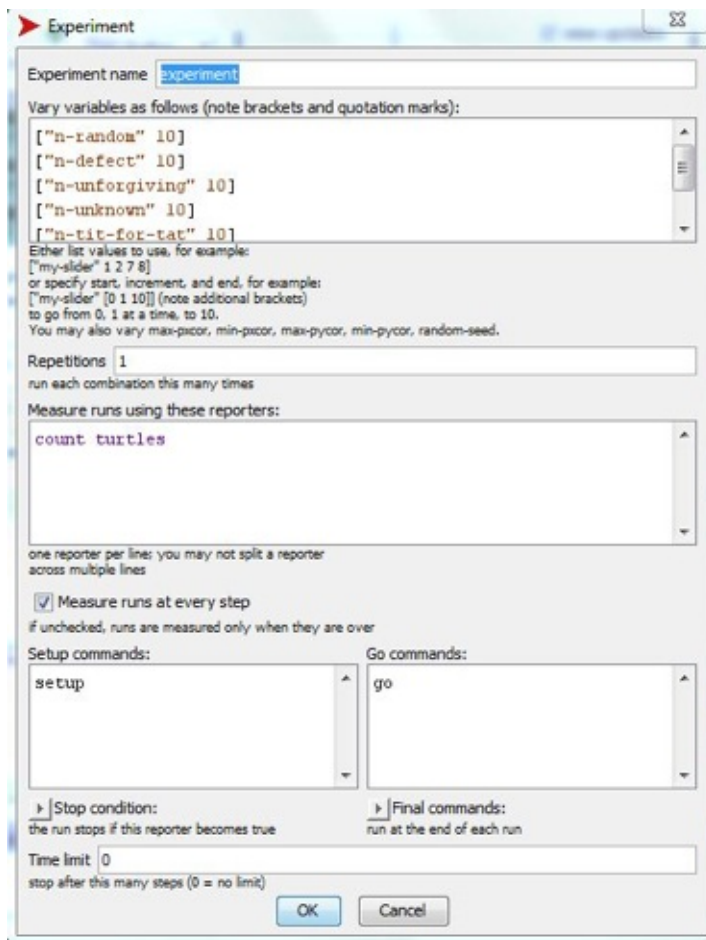


Figure 17.

Now we want to run a series of experiments by varying the distribution of agents. We will vary the variables in the following ways:

```
["n-defect" 5 10]
["n-cooperate" 5 10]
["n-unforgiving" 5 10]
["n-random" 5 10]
["n-unknown" 0]
["n-tit-for-tat" 5 10]
```

This means that we run $2^5 = 22222 = 32$ combinations of the model with different variations to determine how many agents have the strategy defect, cooperate, unforgiving, random, and tit-for-tat. These strategies have 5 or 10 agents. We do not include the unknown strategy.

Now we have to define what information to save. The default case is the amount of turtles, which is not very useful in our case since it remains constant. More interesting is the information in the plot of the interface.

If we go to the code and look what information is used to plot these graphs, we see that we can use the following information to save for our analysis

```

random-score / (num-random-games)
defect-score / (num-defect-games)
cooperate-score / (num-cooperate-games)
tit-for-tat-score / (num-tit-for-tat-games)
unforgiving-score / (num-unforgiving-games)

```

We don't want to save the information for every timestep since the information that is saved is the average over all of the timesteps so far. Therefore, we should uncheck Measure runs at every timestep.

We want the model to run 1,000 ticks, so we select a time limit of 1000.

Now we get something like

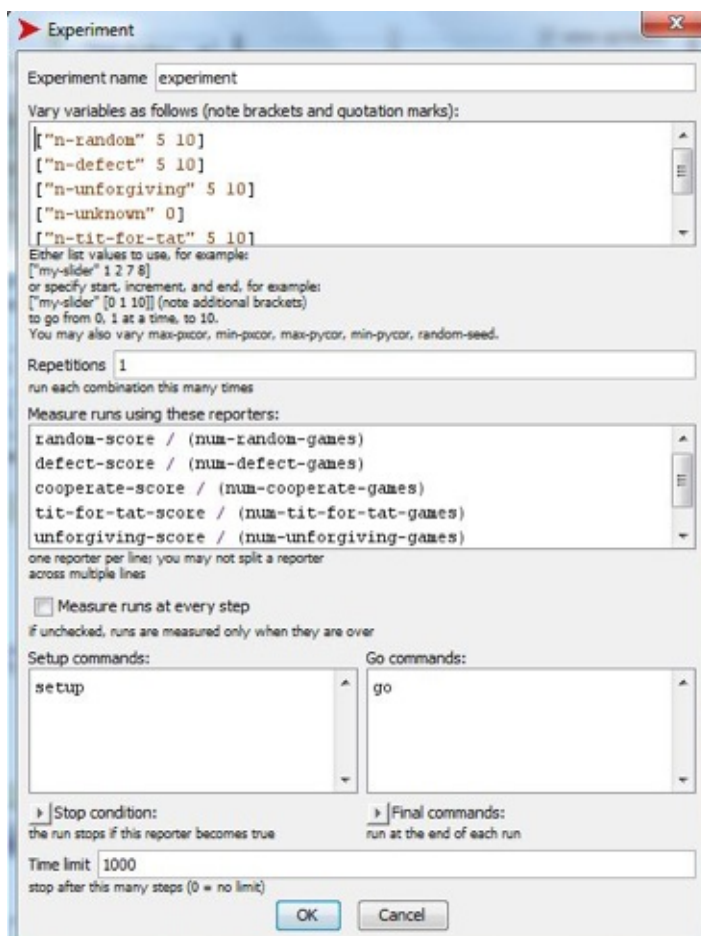


Figure 18.

Now we click OK and get

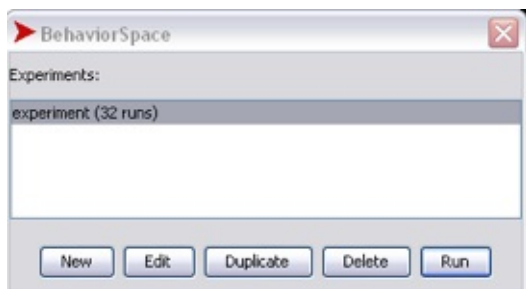


Figure 19.

Which means that we have defined 32 runs. Note that we run each parameter settings only once. If we would have defined 10 repetitions, we would get 320 runs. For this exercise we want to keep the runtime short.

Now we have to click on “Run” and get the following:



Figure 20.

We have to decide to save it as a spreadsheet (Excel), a table (ascii text file), both a table and spreadsheet, or whether you don't want to save the data at all. When we click on one of the options we are asked where to save the data on the computer. When we have done this, the computer runs the simulations and stores the data. We will then get the following window:

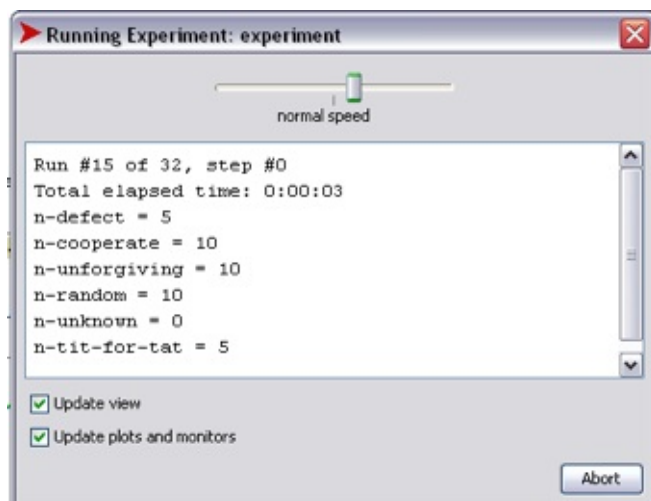


Figure 21.

We see here how many runs it has done so far. We can speed up the runs by unchecking “Update view” and “Update plots and monitors” so that the computer does not spend time to update the screen for each calculation. And we can put the slider to high speed instead of

normal speed. For this exercise it is not very important, but for more complex models we may run BehaviorSpace for many hours, and then it is useful to reduce the time for the calculations by 90%.

Now we can look at the results. By importing the data from the table file into Excel, we get the following picture. We may check the differences in how the information is stored between spreadsheet and table. I prefer to use table for my own analysis.

BehaviorSpace Table data (NetLogo 4.0.4)														
experiment														
03/06/2009 17:22:52.734 -0700														
min-poor	min-poor	min-poor	min-poor	min-poor	min-poor	min-poor	min-poor	min-poor	min-poor	min-poor	min-poor	min-poor	min-poor	min-poor
-10	10	-10	10	-10	10	-10	10	-10	10	-10	10	-10	10	-10
[run number]	n-defect	n-cooperate	n-unforgive	n-random	n-unknown	n-for-tat	[tag]	random-co	defect-co	cooperate	for-tat-co	unforgive	score f	[sum-unforgive-gather]
1	5	5	5	5	0	5	1000	2.240209	2.784492	1.954541	2.333333	2.390705		
2	5	5	5	5	0	10	1000	2.309567	2.642278	2.327485	2.426279	2.517176		
3	5	5	5	5	10	0	5	1000	2.353096	2.614346	1.994322	2.331081	2.41841	
4	5	5	5	5	10	0	10	1000	2.443063	2.558179	2.082074	2.354568	2.429632	
5	5	5	5	10	5	0	5	1000	2.384946	2.6	2.295769	2.383838	2.449595	
6	5	5	5	10	5	0	10	1000	2.542169	2.430007	2.289399	2.548868	2.549869	
7	5	5	5	10	10	0	5	1000	2.288106	2.675294	2.195325	2.499659	2.438993	
8	5	5	5	10	10	0	10	1000	2.323681	2.588671	2.244604	2.464796	2.562084	
9	5	10	5	5	5	0	5	1000	2.436085	2.937808	2.222085	2.408247	2.429684	
10	5	10	5	5	5	0	10	1000	2.595266	2.808662	2.330244	2.484482	2.568722	
11	5	10	5	10	10	0	5	1000	2.603463	3.295385	2.035623	2.396021	2.446889	
12	5	10	5	10	10	0	10	1000	2.61486	2.970494	2.244707	2.436587	2.517829	
13	5	10	10	5	5	0	5	1000	2.525876	3.199236	2.206034	2.546075	2.590388	
14	5	10	10	5	5	0	10	1000	2.596323	2.797095	2.401388	2.572782	2.608888	
15	5	10	10	10	10	0	5	1000	2.513734	3.051771	2.244888	2.493971	2.546585	
16	5	10	10	10	10	0	10	1000	2.632905	3.009511	2.275325	2.544955	2.599037	
17	10	5	5	5	5	0	5	1000	2.673766	2.929965	1.646321	1.994686	2.048896	
18	10	5	5	5	5	0	10	1000	2.957956	2.481084	1.822382	2.196522	2.160622	
19	10	5	5	10	0	5	1000	2.069951	2.36725	1.78044	1.925374	2.092074		
20	10	5	5	10	0	10	1000	2.680423	2.452002	1.84791	2.176823	2.235821		
21	10	5	10	5	0	5	1000	1.977636	2.382796	1.857895	2.286783	2.197917		
22	10	5	10	5	0	10	1000	2.108434	2.359756	1.979332	2.244283	2.295252		
23	10	5	10	10	0	5	1000	2.629506	2.438029	1.929302	2.236446	2.27071		
24	10	5	10	10	0	10	1000	2.968364	2.43969	1.994885	2.243821	2.304349		
25	10	10	5	5	0	5	1000	2.301754	2.70503	1.92	2.197082	2.243282		
26	10	10	5	5	0	10	1000	2.393701	2.738575	2.068578	2.280759	2.370343		
27	10	10	5	10	0	5	1000	2.290762	2.860331	1.790396	2.157287	2.229588		
28	10	10	5	10	0	10	1000	2.382271	2.886289	1.953991	2.280537	2.364796		
29	10	10	10	5	0	5	1000	2.37418	2.645802	2.036613	2.304024	2.374589		
30	10	10	10	5	0	10	1000	2.293578	2.644475	2.240558	2.370369	2.38296		
31	10	10	10	10	0	5	1000	2.293033	2.72118	1.998	2.244813	2.288297		
32	10	10	10	10	0	10	1000	2.253201	2.680683	2.194879	2.327381	2.403957		

Figure 22.

We found now for a few cases that the defect always strategy is not the best. Since the model is stochastic, you need to run each parameter setting many times to be statistically sure about this.

More information can be found at

<http://ccl.northwestern.edu/netlogo/docs/behaviorspace.html>.

Summary, References and NetLogo Primitives

Summary

This chapter provided information on the consequences of random numbers on different applications in NetLogo. A model with random numbers does have different results each simulation. Therefore we need to do many simulations. Behaviorspace is a helpful tool to run a model many times.

References

Huberman B.A., and Glance N.S. (1993) Evolutionary games and computer- simulations. *Proceedings of the National Academy of Sciences of the United States of America* 90(16): 7716-7718.

Nowak, M.A., R.M. May (1992) Evolutionary Games and Spatial Chaos, *Nature* 359 (6398): 826-829.

NetLogo Primitives

[random](#)

[random-poisson](#)

[mean](#)

[random-normal](#)

[random-exponential](#)

[random-gamma](#)

Agents and Resources

Population Dynamics

Key concepts

In this chapter we will:

- Introduce torus landscapes and carrying capacity.
- Present a model where agents are born and die.
- Demonstrate factors that restrict the size of the population in a landscape.
- Show a phase diagram to present model results.
- Introduce primitives `die` and `hatch`.

Introduction



Figure 1.

How many people can our planet sustain? The answer depends on the amount of food produced, energy available, the rate at which people consume resources, and how resources are shared. In this chapter we will discuss the use of models to address questions on the carrying capacity of the environment. We present a model where agents interact with their environment. The agents need to have resources to survive. They will search the environment to collect those resources. The amount of resources available is limited and can be regenerated at a modest pace. If the agents consume more resources than the environment can produce, there might be a problem for the population in the long term.

Although the model presented in this chapter is rather abstract, it addresses a number of broader questions on sustainability.

Basic Model

Let's start with the basic model we will use in this chapter. The model consists of an environment and an agent who lives in this environment.

Environment

Given below is a landscape of $N \times M$ cells. The edges are connected to derive a torus – donut shaped – environment (Figure 2).

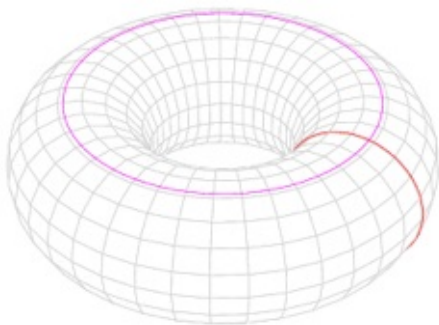


Figure 2. Torus Representation

When we put agents on the landscape they will not experience any edges. Each cell can contain a resource unit. This resource unit provides an amount of energy R to an agent, if the agent collects it. If an agent has collected the resource unit of a cell, a new resource unit can appear on the cell with a probability pr .

To implement this in NetLogo we will look at each patch to determine whether there are resources on the cell. If there are no resources on the cell then there is a chance `prob` that new resources will be added to the cell. In a simple model the only options available are resources on the cell or no resources on the cell. This can be implemented as below. The figure to the right shows a screen shot of such a landscape.

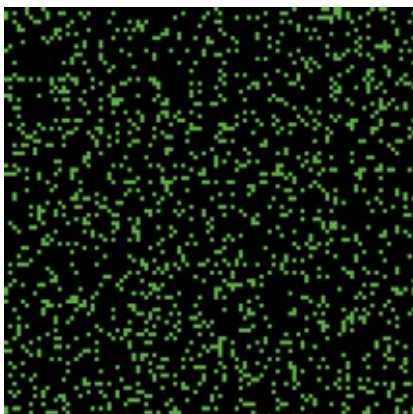


Figure 3: Screenshot of the landscape.

```
if pcolor = black [  
  if random-float 1 < probab  
  [  
    set pcolor green  
  ]  
]
```

Agents

Agents need energy to survive. Each time step they use m units of energy. This is their metabolism. They can derive energy from collecting resource units from the landscape. Agents move randomly around and if they are on a cell with a resource unit, they collect this resource unit. If agents have run out of energy they will die. On the other hand, if their energy level reaches more than bt , the birth threshold, the agent will split into two agents. This cloning is a simplistic way of simulating reproduction.

We will now discuss a NetLogo program that simulates the model described above.

First, we have to define that agents will have the attribute energy. This is the only turtle attribute that we need to include. Of course the turtles already contain a number of default attributes such as location, color and heading.

```
turtles-own [energy]
```

To initialize the model we have to define the initial level of energy the agents have. In this example, we assume that at the start all agents have 10 units of energy. We put the agents randomly on the landscape and represent them as white dots.

We also need to initialize the environment. For each patch we flip a coin to define whether the patch has a resource unit or not. If the patch has a resource unit we define the color of the patch green, otherwise it remains black. Note that we used a standard attribute of the patch to define whether a resource unit is available, and so we will not have to define a new attribute for patches. The initial density of resource is assumed to be equal to `initial-density`, which is a value defined by a slider in the interface.

```

to setup
  ca
  set-default-shape turtles "dot"
  create-turtles number [
    set energy 10
    set color white
    setxy random-xcor random-ycor
  ]
  ask patches [
    if random-float 1 < initial-density [set pcolor green]]
end

```

Once we click on go the agents will make a number of decisions within each tick. The first problem we face is which order decisions need to be made. Will an agent eat first and then move? Does each agent perform the same actions in the same order? In which order do we update the agents? Will some agents get priority? These decisions on the order in which decisions are made can have important consequences on the outcomes of the model. In our simple model it will not have a huge impact. We assume that the order of the actions is movement, eating, metabolism, reproduction, and dying.

First all agents will move before we move to the next action. You see that the agents change their direction randomly before taking one step forward. To model the random type of movement we change the direction of the agents each time step. The `heading` of the agent is changed by moving it to the right. The number of degrees it moves to the right is a random number between 0 and 45. Then the agent moves to the right again, also at a randomly drawn number between 0 and 45. On average the heading of the agent is directed straight forward, but most agents will change its heading a small amount to the right or left. With a small change the agent will change its heading by several degrees in one step. After the direction is defined, the agent will make one step forward using `fd 1`.

```

ask turtles [
  rt random 45
  lt random 45
  fd 1]

```

If they are on a patch that is colored green (thus has a resource) the agent will collect the resource unit, which will generate `penergy` (slider value) units of energy. It is possible that more than one agent is on a cell. In that case the agent who has drawn first to eat will collect the resource unit and will put the color of the cell equal to black.

```
ask turtles [
  if (pcolor = green) [
    set pcolor black
    set energy energy + penergy]]
```

Then the energy level of all agents is reduced with the amount `metabolism` (slider value).

```
ask turtles [set energy energy - metabolism]
```

If the energy level is more than `birth-threshold` (slider value), we first half the energy level of the agent before we generate a copy. `hatch` means that the agent is cloned. If we don't half the energy first, we will get two agents with more energy than `birth-threshold`. In such a situation, the system would generate energy by producing offspring, which is not possible.

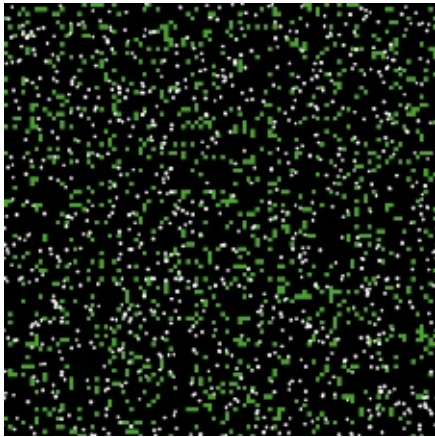


Figure 4: Screenshot of a typical run of the model.8.

```
ask turtles [
  if energy > birth-threshold [
    set energy (energy / 2)
    hatch 1]]
```

Finally, if an agent has a negative unit of energy, the agent will die and is removed from the system.

```
ask turtles [if energy < 0 [die]]
```

We also need to update the resource units on the patches every time step. If a patch has no resource unit, we assume that with a probability `replenishment-rate` (slider-value) a new resource unit will be added to the patch. The NetLogo model for this was discussed above when we discussed the environment.

Now we can simulate the model in NetLogo. In Figures 4 and 5 you see some typical results of the model. You will see that the population level depends critically on the level of the parameters, which can be changed by the sliders. If metabolism is high compared to energy per resource unit, the population is more quick to die out. In other situations we see that the population reaches a somewhat stable level of resources, or has some periodic fluctuations. Why do we get fluctuations of the population size? The agents eat up the resources quickly and produce a lot of offspring. The increasing population causes a scarcity of resource units. Due to the scarcity, agents cannot get enough energy to maintain their metabolism and more agents will die than are born. When the population size drops, more resources become available for those who survived, etc.

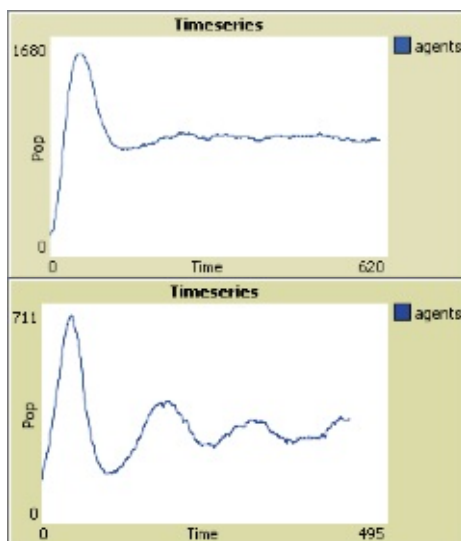


Figure 5: Two possible outcomes of the population level over time.

How many agents can survive in this landscape? The default parameters of the agents are

```
metabolism = 1
birthrate = 20
foodenergy = 3
```

The environment is 100 x 100 cells, thus 10,000 cells. Suppose the environment produces `feedenergy * replenishment-rate * 10,000 cells`, then the environment produces 300 units of energy. This could feed 300 agents. However, agents will have to move to another cell to find food. Agents move randomly so it will not be the most efficient exploration. At least we can expect that the total number of agents that survive over the longer term will be lower than 300 agents. When we run the model we see that the population stabilizes around 200 agents. For each increase in the replenishment rate we can expect a linear increase of the population.

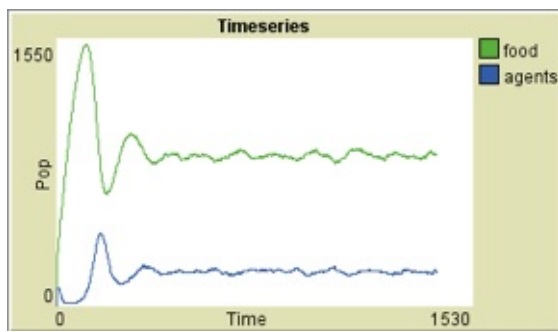


Figure 6: Resource size and population size using a replenishment rate equal to 0.01.

NETLOGO EXAMPLE: POPULATION DYNAMICS

Model: Right click on the Link and Save it. Open Netlogo and Run it with NetLogo.

Density Dependent Resource Growth

We assumed in the previous section that the resource units will always recover from an empty cell using a fixed replenishment rate. This is very unlikely with many resources. The more trees in an empty spot in a neighborhood, the more likely that seedlings will fall on this spot and a new tree will be generated. If there are no trees around, no new trees will regenerate. This is called density-dependent growth and it is a common mechanism in many ecosystems.

For the whole environment we can now expect a logistic growth. In fact, the model above is a spatially explicit version of the famous [logistic growth function](#). Suppose the resource size is defined by $X[t]$ for time step t . Then the resource size at time step $t + 1$ is defined as

$X[t+1] = X[t] + r * X[t] * (1 - X[t] / K)$ Where r is the regrowth rate and K the [carrying capacity](#). In our environment above, K is 10,000, namely 10,000 cells. The highest regrowth rate will happen when half of the resource is available, meaning $X[t] = 5,000$ for $K = 10,000$. If r is equal to 0.1 the resource grows over time, as in Figure 7. This is the classic S figure of the logistic growth function.

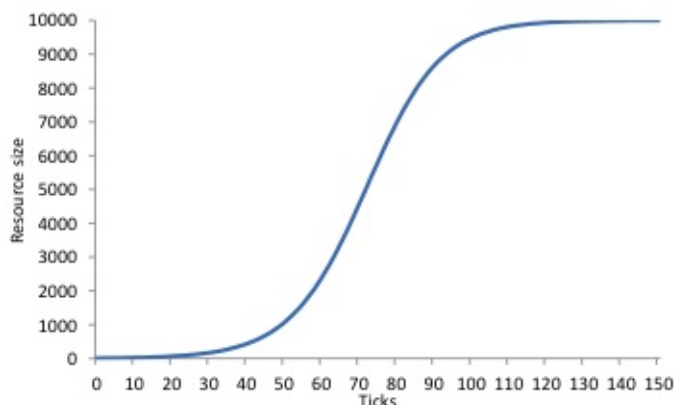


Figure 7. Resource level of logistic growth model with $r = 0.1$ and $K = 10,000$.

In a spatial model, the regrowth rate not only depends on the value of r , but also on how the resource is distributed spatially. To model this in NetLogo we may initially think about the following implementation:

```
ask patches [
  if pcolor = black [
    if random-float 1 < replenishment-rate *
      (count neighbors with [pcolor = green])
      [set pcolor green]
  ]
]
```

This means that for every patch that has no resource, there is a probability that it will regenerate. This probability will be higher in relation to the number of the eight neighboring cells that have resource units.

However, this implementation is often affected by a typical error people make in agent-based models. The problem is the following: All cells are updated randomly, one by one. If you have an empty cell and this cell becomes a cell with a resource unit, this cell is no longer seen as empty by the next cell to be updated. This can lead to strange results since the order in which cells are updated affects the results. It is theoretically possible that an empty landscape with just one cell with a resource unit could lead to a complete recovery in one time step. This is an unrealistic representation.

So how to implement the regeneration of the cells where we only take into account the state of the cells at the start of the time step? We need to add an additional attribute to the patch that remembers the old state of the cell. Then we can correct the model in the following way.

```
ask patches [set oldstate pcolor]

ask patches [
  if oldstate = black [
    if random-float 1 < replenishment-rate *
      (count neighbors with [oldstate = green])]
    [set pcolor green]
  ]
]
```

With a replenishment rate equal to 0.01 we get the dynamics we observe in Figure 8. The resource and population numbers do not stabilize and keep going up and down. When there is a lot of food, the agents start to consume a lot and produce many offspring. This leads the resource levels to go down, making it harder for the agents to find food. Since their metabolism needs energy every time step, some agents will die. As a result, the population size falls, and fewer resources are eaten. This, in turn, leads to the rise of the resource.

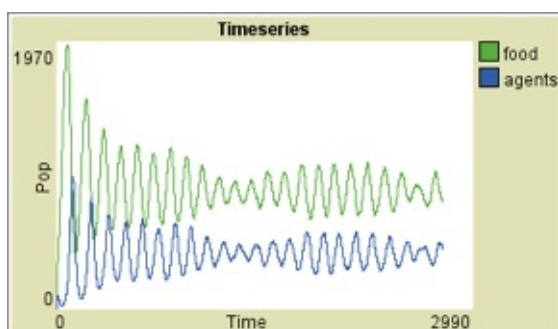


Figure 8: Population and resource size for density-dependent regrowth of resource with regrowth rate equal to 0.01. (Note that food is resource size divided by 4 to have the numbers at a similar scale.)

We can portray the dynamics of the population and resource in a so-called [phase diagram](#) (Figure 9). This shows the cyclical dynamics of the two variables leading the system to circle around in a confined area of the phase diagram, but never settle into an equilibrium.

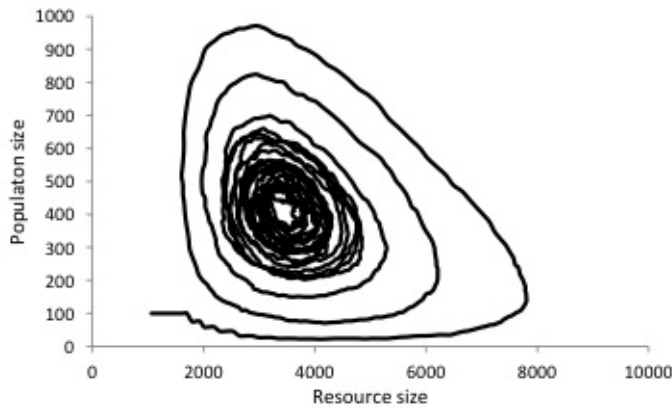


Figure 9: Phase diagram of population and resource with regrowth rate 0.01.

When we increase the regrowth rate to 0.1 the population level circles between 2500 and 5500, but never stabilizes (Figure 10). The population does not die out. When the resource is low, agents die out and the resource recovers. However, most species do not reproduce by cloning, and we will explore the consequences of having two parents in the next section.

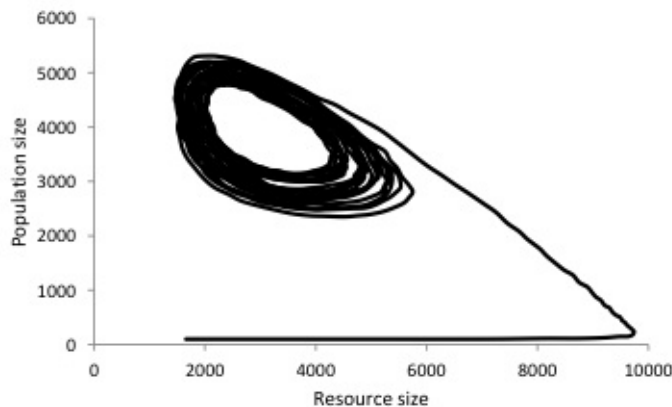


Figure 10: Phase diagram of population and resource with regrowth rate 0.1.

The Effects of Mating

Suppose agents can only reproduce when two agents meet who have energy beyond the minimum level of energy. Such a rule will imply that not all agents with more energy than the `birth-threshold` will get offspring. They have to find the right mate.

To implement this, an agent who has the minimum level of energy required for producing offspring needs to look around and check whether there is an agent nearby with sufficient energy. The primitive `in-radius` gives the set of agents that are found within a radius of a defined size. Thus `turtles in-radius search-radius with [energy > birth-threshold]` is the set of agents in radius of size `search-radius` who have more energy than the minimum amount of energy needed to produce offspring.

We only want to move on if there are any agents that meet this condition. Thus before moving on we use primitive `any?`, which checks whether there are agents within the agent set, and thus

```
if any? turtles in-radius search-radius with [energy > birth-threshold]
```

Now there is another problem. The agent who is looking for a mate is included in the agent set since it is an agent within the search radius. We have therefore to find a way to exclude the agent who is searching for a mate. Therefore we check what the identity is for the agent looking for a mate and exclude this from the list of agents that become a mate

```
let agentwho who  
if any? turtles in-radius search-radius with [energy > birth-threshold and who != agentwho]
```

So, at this point we have a set of candidate mates. The agents are not very picky. We assume they skip speed dating and just accept any mate that meets the minimum conditions. Hence to select the mate we take one agent of the set of candidates. The number of candidates might be one, and then that one is chosen. But if the set is larger than one, one agent is randomly selected.

```
let mate one-of turtles in-radius search-radius with [energy > birth-threshold and who != agentwho]
```

We assume that initial energy of the child is 10% of the combined energy of the two parents. Each parent contributes an equal share to the energy of the offspring. This is implemented as:

```
let childenergy (energy + [energy] of mate ) / 10
set energy energy - 0.5 * childenergy
ask mate [set energy energy - 0.5 * childenergy]
[ hatch 1 [set energy childenergy] ]
```

When we run the model with regrowth rate equal to 0.01 and a search-radius of 1 we get the phase diagram of Figure 11. The amplitude of the cycle is a bit larger than Figure 9, without mating. What is the reason for this?

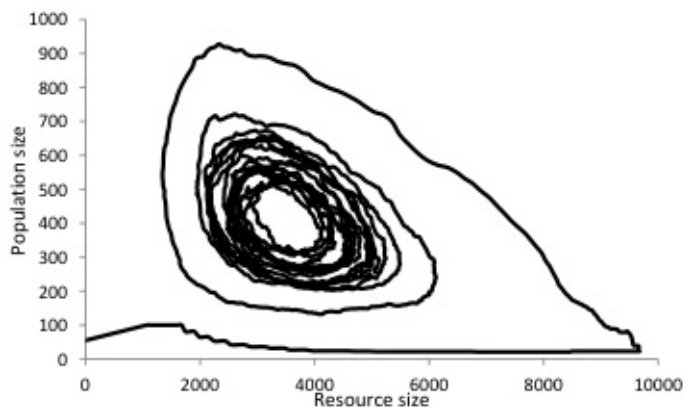


Figure 11: Phase diagram for population and resource size with mating.

Summary and NetLogo Primitives

Summary

In this chapter we introduced models of agents who eat, reproduce and die on a landscape of a renewable resource. We then explored the consequences of different assumptions on the population and resource levels.

NetLogo Primitives

set-default-shape

hatch

die

who

Foraging

Key concepts

In this chapter we will:

- Introduce search behavior of agents.
- See the consequences of pooling and sharing resources.
- Introduce primitives `list`, `breed`, `face`.

Introduction

Introduction

Foraging refers to searching for food resources and exploiting these resources. Basically, a forager needs to search for food and there are different strategies on how to do this. What is the best strategy? We will see that the best strategy depends on the forager's environment.



Foraging of White-nosed Coati at Rara Avis near Las Horquetas, Costa Rica.

Foraging is not restricted to animals or ancient human societies. We use the term foraging nowadays to refer to shopping in modern urban environments, such as in a supermarket or by surfing the web. In each of these cases we are looking for resources and we can follow different strategies.

One foraging strategy would be to more or less walk around randomly and hope to get lucky. If you don't know the environment and nobody is around to advise you, this random search is the best you may do. But as you get to know the environment, you may develop a map of the landscape. This will help you to be more efficient in looking for resources. If other agents are around, you may exchange information on good locations. But what if the environment is changing frequently? Then remembering good locations does not help.

In this chapter we describe a simple model of agents foraging on a landscape. We will see how different foraging strategies can lead to different population levels in the same resource environment.

Creating a Landscape

In this section we create a landscape where different types of food resources are located. Unlike the model in the previous chapter where one resource was spread throughout the whole landscape, now we assume that there are multiple types of resources spread over the landscape.

Before we discuss how we implement the landscape we need to introduce a new primitive, namely `list`. This primitive allows us to store a list of values, turtles, strings. This is helpful to store information in an efficient way. To define a list you can simply put the values in a list between brackets separated by spaces like `set listname [1 2 3 4]`.

If you want to use a value of the list you call it by `item i` where `i` is the location of the value in the list. Note that Netlogo starts counting with 0 at the first location. Thus

```
item 1 listname
```

is equal to

```
2
```

To change the values of a list you need to use `replace-item i`. For example, if you want to replace the value 3 to 5 then you do

```
set listname replace-item 2 listname 5
```

```
listname is now [1 2 5 4].
```

To add an item at the end of the list you use `lput`. For example, you want to add the value 6 to `listname` then you will do the following

```
set listname lput 6 listname
```

This leads to `[1 2 5 4 6]`.

However, if you want to add a value at the beginning of the list you use `fput`. For example, you add 7 in front of `listname` then you do

```
set listname fput 7 listname
```

which leads to `[7 1 2 5 4 6]`.

We now go back to the foraging model landscape. We will distinguish 3 food sources. Each food resource has a certain density, a level of energy and color that displays in the interface. We now define the lists that we will use in creating the maps:

```
set density [0.01 0.005 0.001]
set foodenergy [10 20 100]
set foodcolor [green red blue]
```

To create the environment we will randomly place patches on the landscape that contain one of the three types of food. Only one of the food types can be on a patch. We also want to have the number of patches for each food type equal to the number needed to generate the density that is defined in the list above. If the number of cells is `world-width * world-height` then the number of cells that will be needed to create the density for food type *i* is equal to `item i density * (world-width * world-height)`.

Now we can create the algorithm that will create the initial map of food resources. For each food type we will turn patches into food sources until the desired density is reached.

```
let iter 0
let counter 0
while [iter < 3]
[
  set counter 0
  while [counter < (item iter density * (world-width * world-height))] [
    ask one-of patches
    [
      if pcolor = black [
        set pcolor item iter foodcolor
        set counter counter + 1
        set whichfood iter + 1]
    ]
  ]
  set iter iter + 1
]
```

This leads to a landscape like in Figure 2.

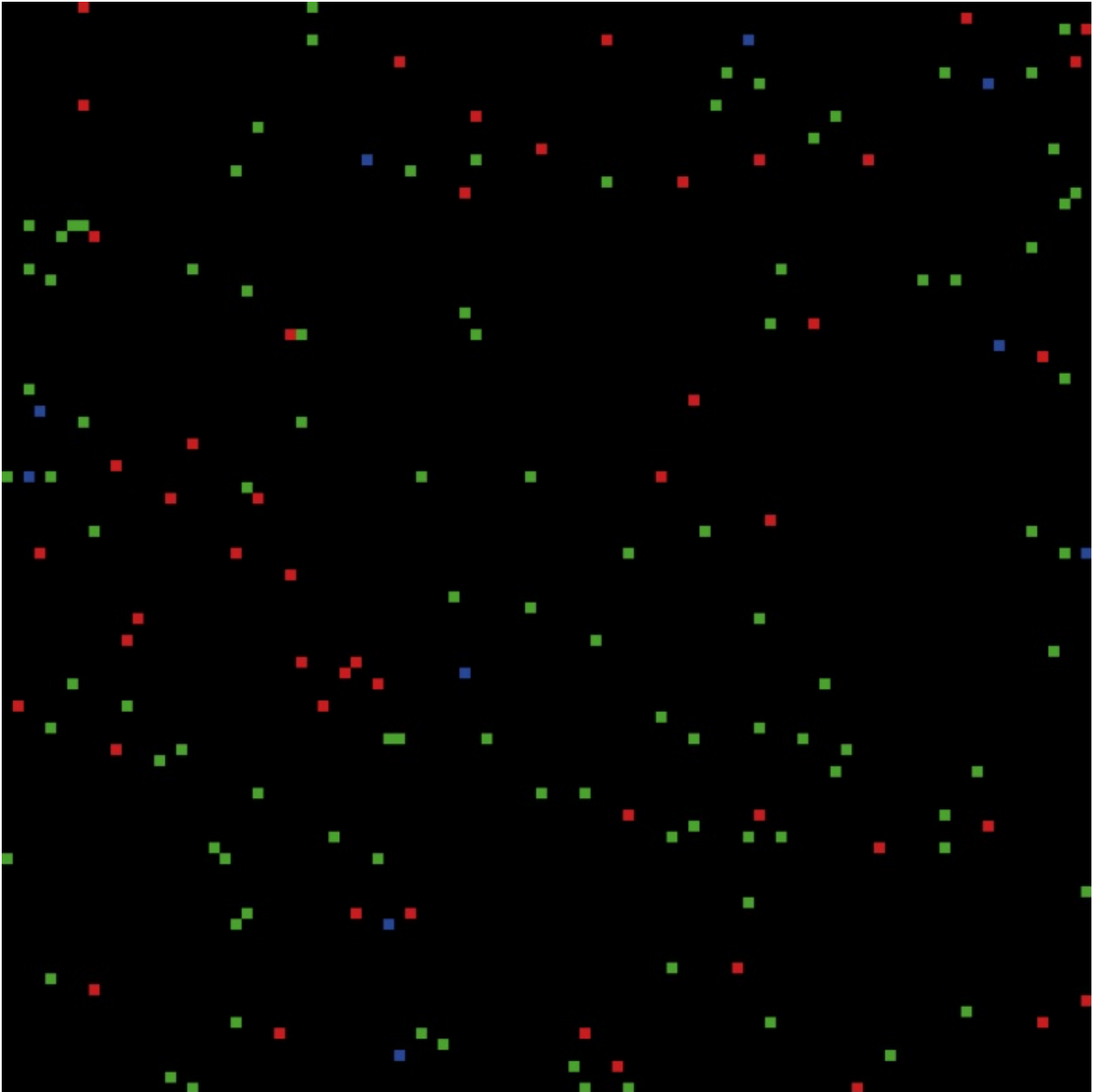


Figure 2: Map generated with 3 food types.

What happens if a food item is eaten? Do we replace the food item immediately at the same place or will it appear later at another place? With our model we will explore the consequences of uncertainty, so we will make a model where we can define how frequently a food item is replenished at the same location or at another random location at the map. We also will include the possibility of a delay of replenishment.

If a patch is not colored black, there is a food item. The agent determines which food item it is, collects the food and determines the replenishment.

```
if pcolor != black [
  let iter 0
  while [iter < 3]
  [
    if pcolor = item iter foodcolor
    [
      set pcolor black
      set whichfood 0
      set energy energy + item iter foodenergy
    ]
  ]
]
```

With a probability `pmove` the food will be replenished at another location. If the food location is replaced, a countdown process will start to time when the food will become available. This also happens when the food location does not change.

```
ifelse random-float 1 < pmove [
  ask one-of patches with [whichfood = 0]
  [
    set countdown countdownmax
    set whichfood (iter + 1)
  ]
][
  set countdown countdownmax set whichfood (iter + 1)
]
set iter iter + 1
]
```

We created a countdown for food items when they are eaten, so each tick we will need to update the countdown. To do this we added the procedure `regrowth` that is triggered each tick. If the `countdown` equals 0 the patch will be colored according to the food type on the patch.

```
to regrowth
  ask patches with [countdown > 0]
  [
    set countdown countdown - 1
    if countdown = 0 [set pcolor item (whichfood - 1) foodcolor]
  ]
end
```

Foraging

The basic foraging model that is introduced in this section assumes that the agents move around in the landscape without a memory and exchange information with other agents. Agents stumble on resources and consume them.

The movement is modeled as follows. Agents move on in the direction they were moving with a probability `probcont` and otherwise change their direction. Every step agents also consume energy through their metabolism. This is modeled in the following way.

```
if random-float 1 > probcont [rt random 45 lt random 45]  
  fd 1  
  set energy energy - metabolism
```

Like the model in the previous chapter, agents die when no energy is left, and reproduce when their energy level surpasses a threshold.

Figure 3 shows a typical simulation run. When we start with 100 agents, the population grows to about 400 agents. When we vary the level of uncertainty (`pmove`) we don't see a change in the results. This is not surprising. Agents move around without specific directions and come across resources by accident. Including a more variable landscape of resources will not affect the results.

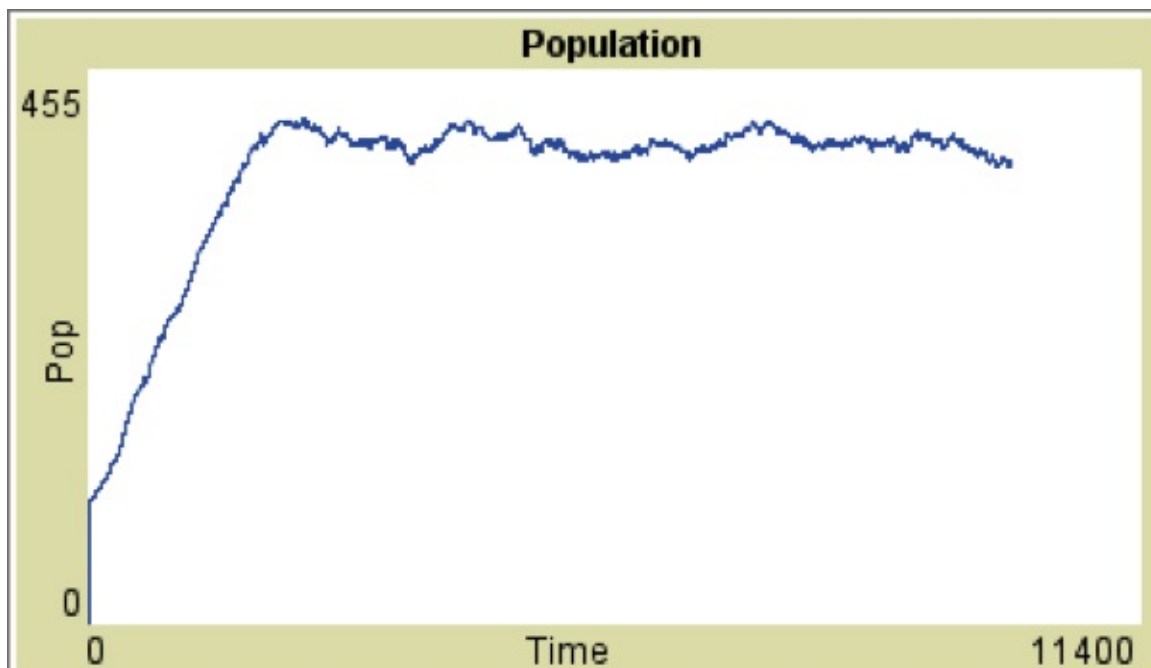


Figure 3. Population level over time for a typical run of the model.

Memory

What will happen if agents can remember locations where they found food before? If their energy level starts getting low, they may check the locations where they have been successful in the past. We can model memory in different ways, and we adopted a very simple way to model memory. Every time an agent eats a food item that holds more energy than a certain minimum level, the agent will mark that location and keeps it in memory. When the energy level of the agent is below a minimum level, the agent starts to look into its memory and goes to the location of opportunity that is closest to the current location of the agent. If there is no food at that location, the location will leave the memory of the agent.

To implement this in Netlogo we will use turtles to mark the locations. This will help us later to direct the agents to the locations. However, this causes a problem. Turtles can now be foragers and markers. In order to distinguish the different uses of turtles we will use the primitive `breed`. By defining different breeds of turtles you can create different types of agents with different attributes. To define a breed you provide the name of the breed when they are multiple or single like

```
breed [foragers forager]
breed [markers marker]
```

You can now define attributes of the foragers by using `foragers-own` and use ask for forager agents directly.

When we model that agents update their memory, they are creating a marker turtle and put the marker agent into the memory of the agent. This is implemented in the following way:

```
if item iter foodenergy > thresholdmemory [
  let marked self
  hatch 1 [
    set breed markers
    set marked self
  ]
  set memory lput marked memory
]
```

Note that we use a temporary variable `marked` to create a variable that captures which turtle is the agent. The location of the marker agent is the location where the agent is at the moment of the creation of the marker agent. Memory is a list and we put the marker in the memory.

When agents are moving around and their energy level goes below a threshold

`THusememory` we first check if there are any markers in the memory of the agent. By `length memory` we get the number of objects in the list `memory`. If there is no memory there is no need to use it. If there is a memory, we will go through the list of markers and check the distance between the agent and the location of the marker. `distance x` gives the distance between the current position of the agent and turtle `x`. The marker with the shortest distance will be chosen as the target to move to.

The primitive `face` is used to redirect the `heading` of the turtle towards the marker that is targeted. Thus if the agent is in need it turns its direction to the closest location in its memory of good opportunities.

```
if energy < THusememory [  
  let sizememory length memory  
  if sizememory > 0 [  
    let i 0  
    let mindistance 100  
    while [i < sizememory]  
    [  
      let dist distance item i memory  
      if dist < mindistance [  
        set mindistance dist  
        set target? true  
        set target item i memory  
      ]  
      set i i + 1  
    ]  
    face target  
  ]  
]
```

When agents arrive at the location of the target we will remove the target from the memory whether there is food on the patch or not. How do we know whether the agent arrived on the patch? The distance between the agent and the target marker should be small. We could say that the distance is zero, but since agents are not always located in the center of the patch, we may end up with agents getting stuck since they are on the right patch but not exactly at the location of the marker.

To avoid agents getting stuck on a patch trying to find the right location we allow some marker of error. This is the reason we check whether the distance between the agent and the target marker is smaller than half a patch.

If the agent is near the marker we set `target?` on `false` since we arrived at the target. We also remove the target from the memory. To avoid getting a model with many ghost markers, which can make the model run very slow, we remove the marker turtle from the model by letting it `die`.

If there is no food on the patch, the agent will check its memory for another target. If the memory is empty the agent will move around in a kind of random walk as defined above. If the agent finds a patch that contains food a new marker turtle is generated and added to the memory.

```
if distance target < 0.5 [  
  set target? false  
  set memory remove target memory  
  ask target [die]  
]
```

If an agent gets offspring we need to take special care about the memory. The memory is assumed to be based on the experience of the agent. We assume that the parent does not pass its memory to its offspring. This could be a possible assumption, and we suggest exploring what the consequences would be. But suppose the newborn agents will be born with a blank slate. A clone of an agent will have its parent information on `memory`, `target?` and `target`. To erase the memory and target information from the clone of the parent agent we will define a hatched agent as below.

```
hatch 1 [  
  set memory []  
  set target? false  
  set target nobody  
]
```

Now we will run the model for different combinations of parameter values. We want to know what the effect is of having a memory. Furthermore, we would like to see whether the benefits of having a memory change according to the rule when agents use their memory and the kind of environment they live in.

In Figure 4 we see the results that occur when agents remember every location of successful food collection (`thresholdmemory` equal to 19). We see that the population size is higher than the baseline, which is the population level without memory. But the effect of memory varies on the context. Population size is highest when the environment is stable (`pmove` equal to 0) and when agents use the memory only when they are really hungry. A stable environment makes it more likely that memory actually makes sense. One should also not wait to the last moment (a low value of `THusememory`) to use the memory since it might be too late to extract find the resources on time.

We see that when agents use their memory very quickly, the benefit of the memory declines. The reason for this is that agents return to a location before the food item has been replenished. Since it takes 50 ticks for a food item to replenish, it makes sense to use the

memory only if the energy level becomes really low. Otherwise agents walk around without being successful and forget about possible good locations.

Figure 5 shows the results that occur when the agents are really picky about which types of locations to remember. For instance, when they only remember those food items with the highest energy levels, the benefit of memory is very modest. In fact, most agents will not have anything in their memory since they have not experienced these high level resources.

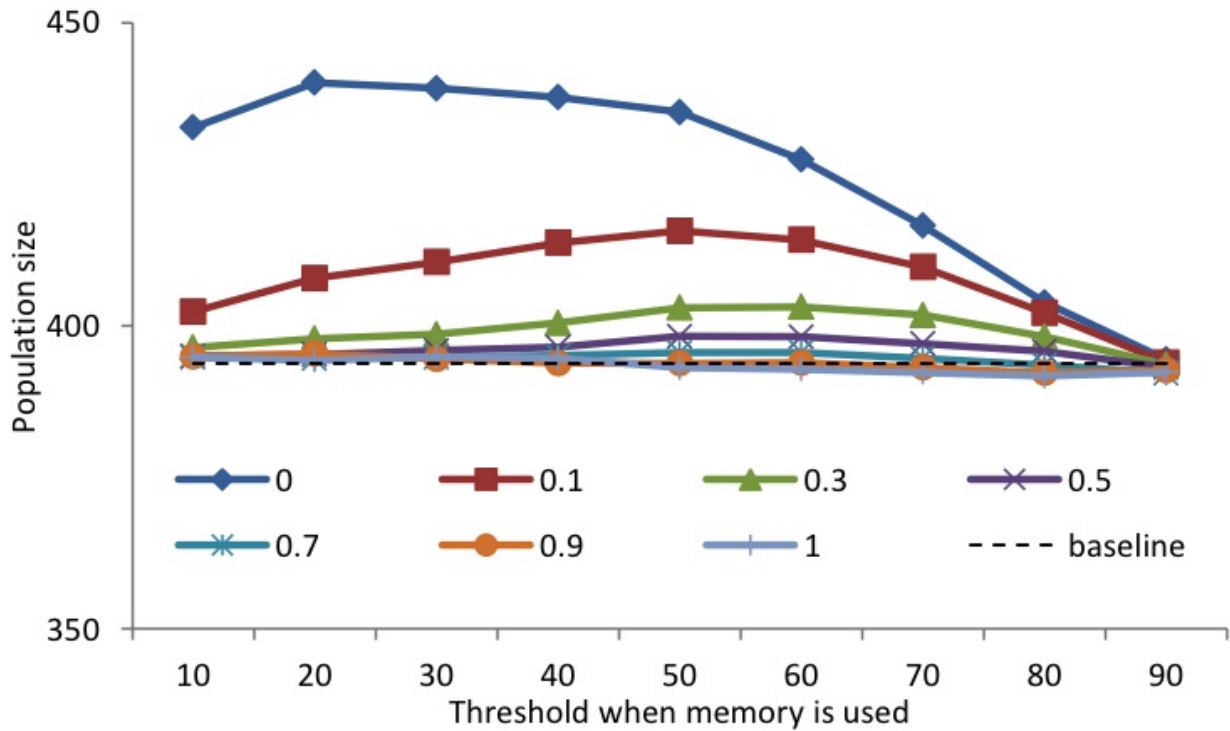


Figure 4: Population size average of the last 5000 ticks for different parameter combinations and a `thresholdmemory` equal to 19. Different lines represent different values of `pmove`.

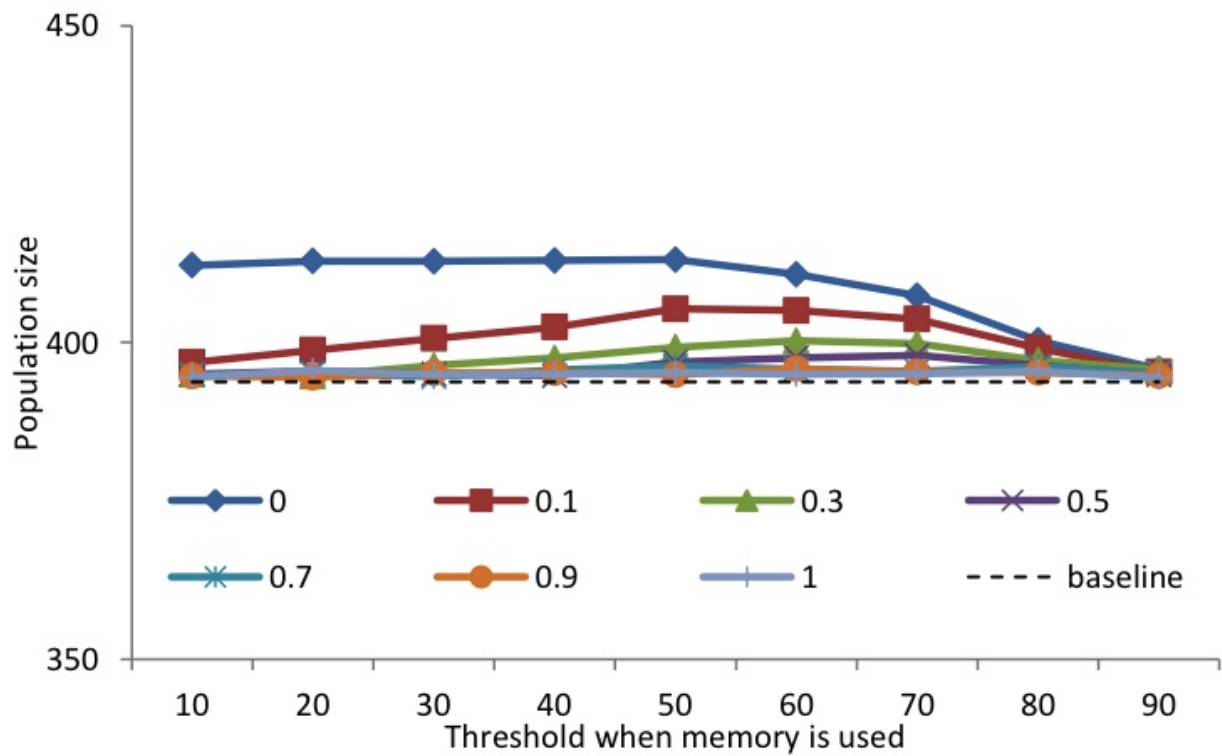


Figure 5: Population size average of the last 5000 ticks for different parameter combinations and a `thresholdmemory` equal to 99. Different lines represent different values of `pmove`.

NETLOGO EXAMPLE: FORAGING

Model: Right click on the Link and Save it. Open Netlogo and Run it with NetLogo.

Summary and NetLogo Primitives

Summary

In this chapter we looked at foraging behavior of agents in looking for food resources. We introduced new primitives like `list`, `face` and `breed`. We see that it benefits agents to use memory, but only when food items do not change locations every time they are consumed.

NetLogo Primitives

`list`

`item`

`replace-item`

`lput`

`fput`

`breed`

`self`

`face`

`heading`

Evolution

Key concepts

In this chapter we will:

- Discuss the principles of natural selection and evolution and how to model this.
- Show the difference between natural selection and genetic drift.
- Introduce primitives in-cone, mouse-down?, mouse-inside?

Introduction

Evolution refers to any change across generations in the inherited attributions of the population. The principle of evolution is largely studied in biological systems, but similar processes are observed in social systems. It was Charles Darwin who formulated the basic scientific theory of evolution by means of natural selection. This theory is still the basis for much of biological research. In the more than 150 years since the publication of his famous book *On the Origins of Species* we have derived many more insights in the biological mechanisms of evolution. For example, DNA was discovered to encoded the important information of life forms that is transmitted from parents to children.



Figure 1. Charles Darwin as a young man in the 1830s

It is beyond the focus of this chapter to have a detailed discussion on evolution. Instead we will consider the basic principles that Darwin proposed and show how these can be used to model evolution in agent-based models. Darwin observed that (1) populations produce more offspring than will survive, (2) individuals differ in their traits which allow some individuals to have more offspring than others, and (3), individual characteristics are inheritable. This means that the characteristics of agents who have more offspring will get a larger share in the population than the characteristics of agents who have less offspring. This leads to the

saying that those who fit the environment better will produce more offspring. It does not mean that these characteristics are better in all circumstances, but at least in the circumstances that led them to get more offspring. In order to model the evolution of a population we need to have variation, mutation, and selection.

We will illustrate the principle of evolution with an interactive and fun model in NetLogo. The model Bug Hunt Speeds can be found in the model library in the folder Curriculum Models and the subfolder BEAGLE Evolution. The model has bugs walking around on the landscape. There are six types of bugs who differ in their speed. Hence the variation in the population is speed. You can catch bugs with your mouse. Click with the arrow on a bug and it will disappear. One of the existing bugs will be cloned to keep the population at a stable level. You will see that it is easier to catch slow bugs compared to fast bugs. You act as the selection mechanism and it is likely that the average speed of the remaining bugs will start to increase.

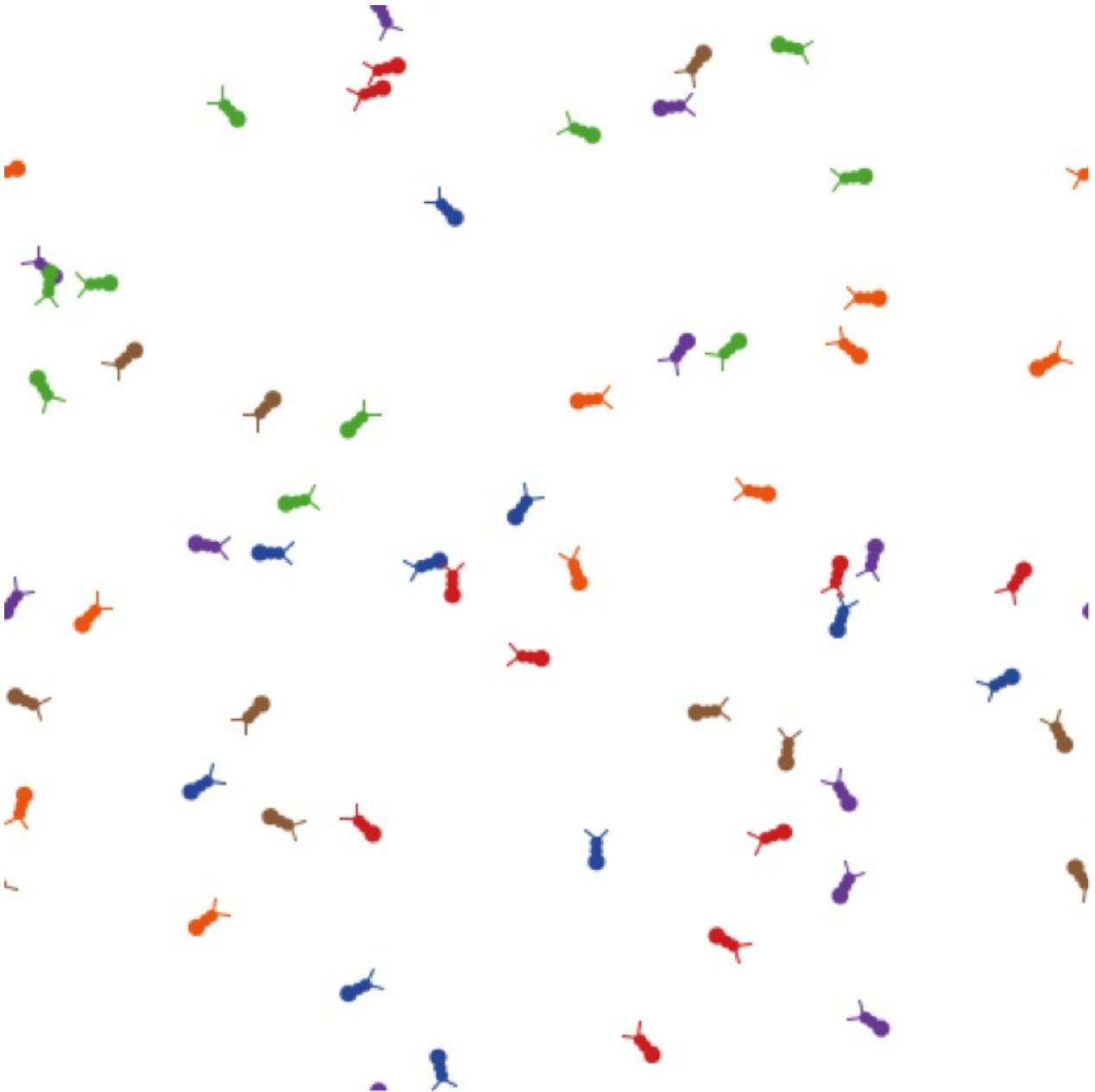


Figure 2. Screenshot of the Bug Hunt Speed model.

Figure 3 shows the distribution of the bug variation at the start of the simulation and after a few minutes. Over time there will be more fast bugs and the slow bugs will have become extinct. Figure 4 shows that the average speed is increasing steadily over time.

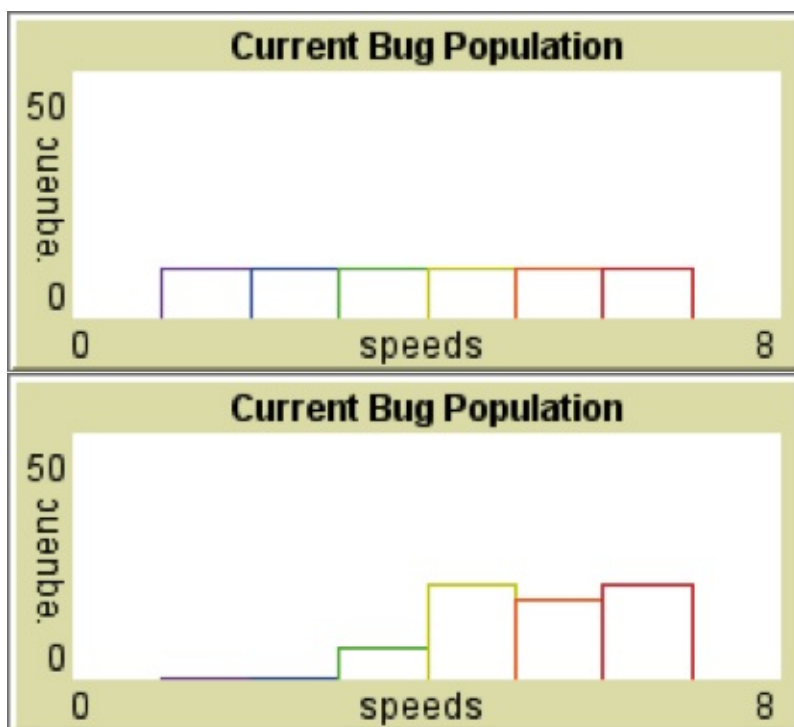


Figure 3. The top graph shows the initial distribution of the six types of bugs; the bottom graph shows the distribution after a few minutes of bug hunting.

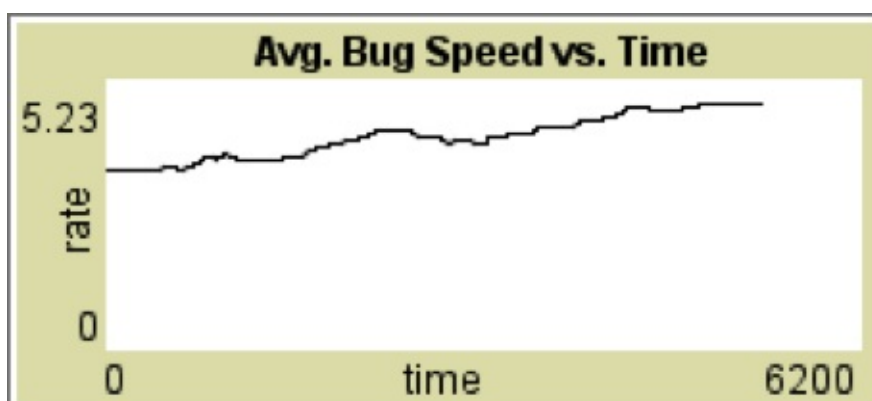


Figure 4. The average speed of the bugs over time.

The results in Figures 3 and 4 reflect a setting [flee] where bugs move when they see a predator nearby as opposed to staying stationary. You may try to catch the bugs with the flee slider turned on and off. What are the differences in the results? Can you explain why?

In the Bug Hunt Speed model some primitives are introduced that we had not seen before. The first one is `in-cone` and is used to model the flee behavior. The primitive `in-cone x y` is called by a turtle and simulates a cone of vision for the turtle. The turtle can see up to distance x and the angle of the cone is y using the current heading of the turtle. If y is equal to 360 the cone is a radius with size x .

In the code of the Bug Hunt Speed model the bugs check whether there are any predators in the cone with a distance of 2 patches and an angle of 120 degrees. Hence the bug cannot look back. If the bug sees a predator it will change its heading 180 degrees away from the predator so that it can flee.

```
ifelse flee? [  
  ifelse any? predators in-cone 2 120 [  
    set candidate-predator one-of predators in-cone 2 120  
    set target-heading 180 + towards candidate-predator  
    set heading target-heading  
  ]  
]
```

An interesting aspect of this model is that you are the predator instead of a computer agent. So how would you model the interaction between the mouse and the model? First it uses `mouse-xcor` and `mouse-ycor` which defines the coordinates of the mouse in the 2D view of the model. Second the model checks whether the mouse button is down by `mouse-down?`, and inside of the view by `mouse-inside?`. If the mouse button is not down or the mouse is not inside of the 2D view, there is no reason to check whether we have caught a bug. Otherwise, we will check whether one of the bugs is inside the radius of the predator.

Even though we use only one predator we still use `ask predators` and `one-of predators`. Since there is only one predator, it will always be our mouse.

```
if not mouse-down? or not mouse-inside? [ stop ]  
let prey [bugs in-radius (size / 2)] of one-of predators]
```

Bug-bird Co-evolution

Instead of playing the game and simply catching bugs we can model the hunting behavior of the predator—the bird. Once we do this we can even model an entire population of birds.

The model distinguishes two types of hunting behavior that we discuss below. Each bird first determines whether there are any bugs within the vision of the bird.

```
set candidate-bugs bugs in-cone initial-bird-vision 120
```

If there are multiple bugs within the bird's vision, which one will the bird follow? The first strategy is for the bird to lock onto a target bug. If the bird has not defined a target it will choose the nearest bug.

The following code defines the heading of the bird before it is moved. The first line is to define the closest bug, which is the bug with the shortest distance to the bird out of the list of candidate bugs.

```
set closest-bug min-one-of candidate-bugs  
[distance myself]
```

The following statement defines that if there was no pursuit and the agent wants to pursue a target bug, the closest bug is the target bug.

```
if (target = nobody and bug-pursuit-strategy = "lock on one") [  
  set prey-agent closest-bug  
  set target prey-agent  
  set heading towards prey-agent  
]
```

If the bug goes after the closest bug it will have the heading towards the closest bug.

```
if (bug-pursuit-strategy = "closest" and target != closest-bug) [  
  set prey-agent closest-bug  
  set target prey-agent  
  set heading towards prey-agent  
]
```

We notice that nothing is said about what would happen if the agent already has a target. This target may change its position forcing the bird to adjust its heading to remain successful. Therefore we add the following, which will be added directly after the closest bug

is determined.

```
if (target != nobody and bug-pursuit-strategy = "lock on one") [  
  set heading towards prey-agent  
]
```

When birds and bugs reproduce there is a mutation of the speed and vision levels of the parent. Reproduction is asexual, but the child is not an exact clone of the parent. To include mutation the hatched new bug or bird will be adjusted for speed and vision. Below is the code for the speed of the bug. In 50% of a parent's offspring, the speed will be higher than the parent and in the other 50% it will be lower. The amount it will be higher or lower is a random value between 0 and the mutation rate. Note that the model includes upper and lower values for speed. Speed cannot be negative, and because of physical constraints speed is also assumed to be constrained to a maximum value.

```
ifelse random 2 = 0  
  [set speed (speed + random-float bug-speed-mutation )]  
  [set speed (speed - random-float bug-speed-mutation )]  
  
if speed > max-speed [set speed max-speed]  
if speed < 0 [set speed 0]
```

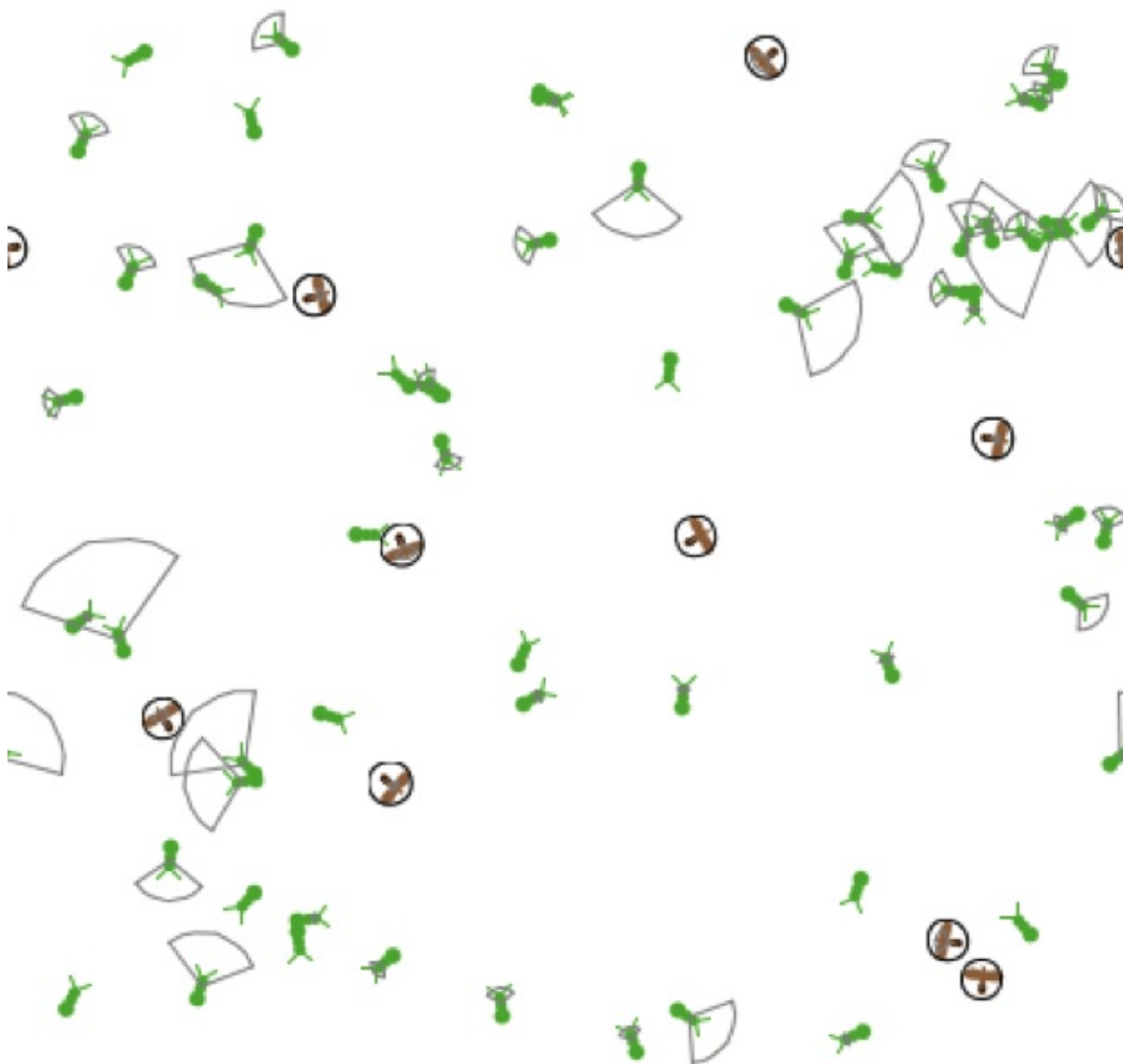


Figure 5. Screenshot of Bug Hunt Coevolution.

Figure 6 show some typical results of this model. Bugs benefit by going faster to survive. The same holds for birds, especially if the birds start to speed up. When a bug's vision increases it starts to flee earlier, so a bird benefits when it can spot a bug first (and move faster than the bug).

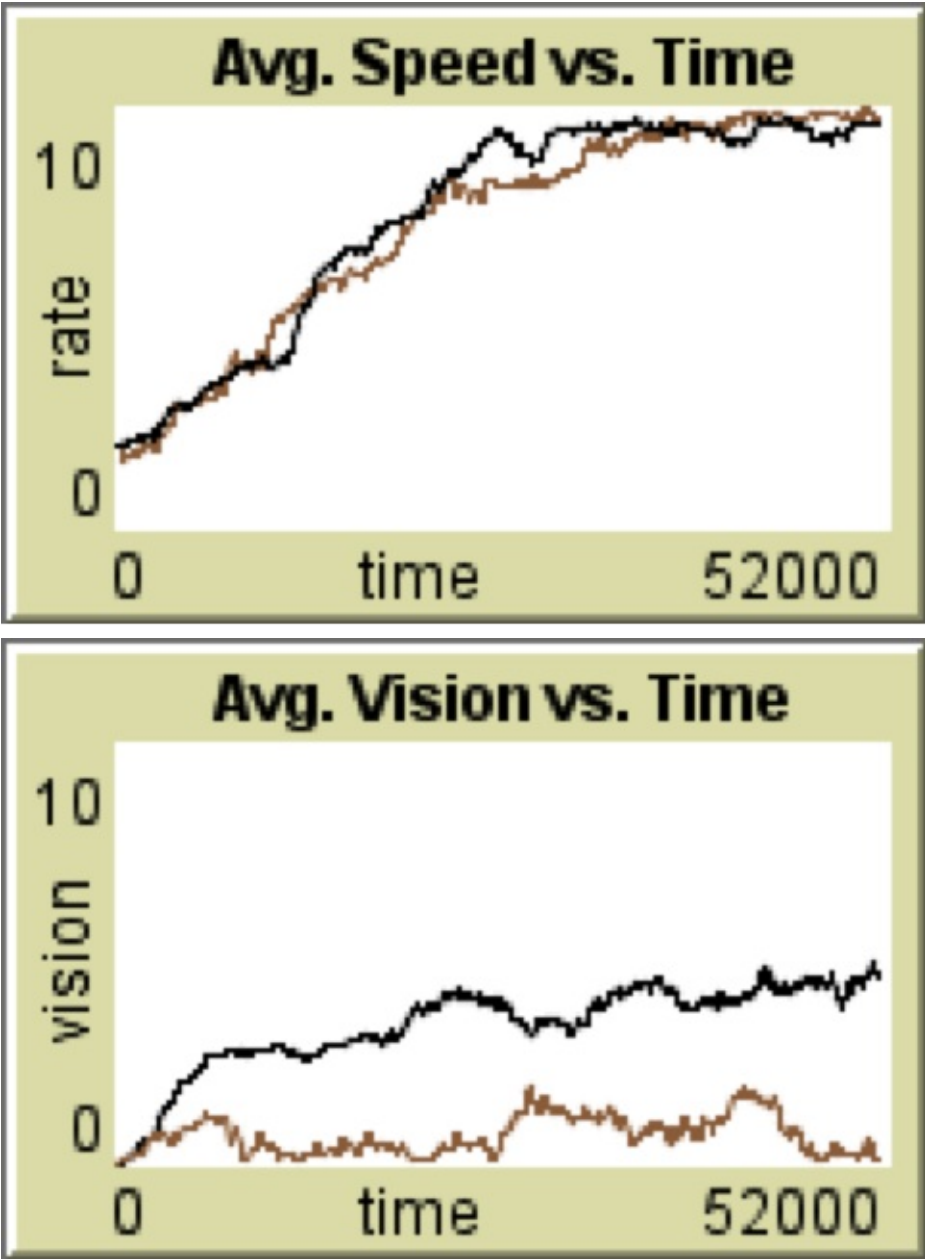


Figure 6: Evolution of speed and vision of birds and bugs.

Genetic

Predators are not the only cause of death of a prey. Random events can cause prey to die. Just being at the wrong spot at the wrong moment. Random events such as fire, illness or just random mutations can cause a change in the variation with long-term consequences. In large populations random events are likely to have no impact on the direction of the evolution of a species, especially when random events affect only a few individuals at a time. Exceptions are big events, such as a meteor hitting the Earth and causing the extinction of the dinosaurs. But for small populations random events can cause a change in the direction of evolution. We call this [genetic drift](#) and it is a different mechanism than natural selection. Such a genetic drift can be caused when a sub population gets isolated on an island, lake or small terrestrial habitat.

The model Bug Hunt drift allows the study of genetic drift. Instead of picking slow bugs because they are easier to catch, you can click on the button to pick one of the bugs at random. We see that over time, after clicking many random bugs, one of the bug types start to dominate (Figure 7). This is not the slowest or fastest bug, but bug type with speed 4. Genetic drift can lead to new variations of a species that do not have a fitness benefit.

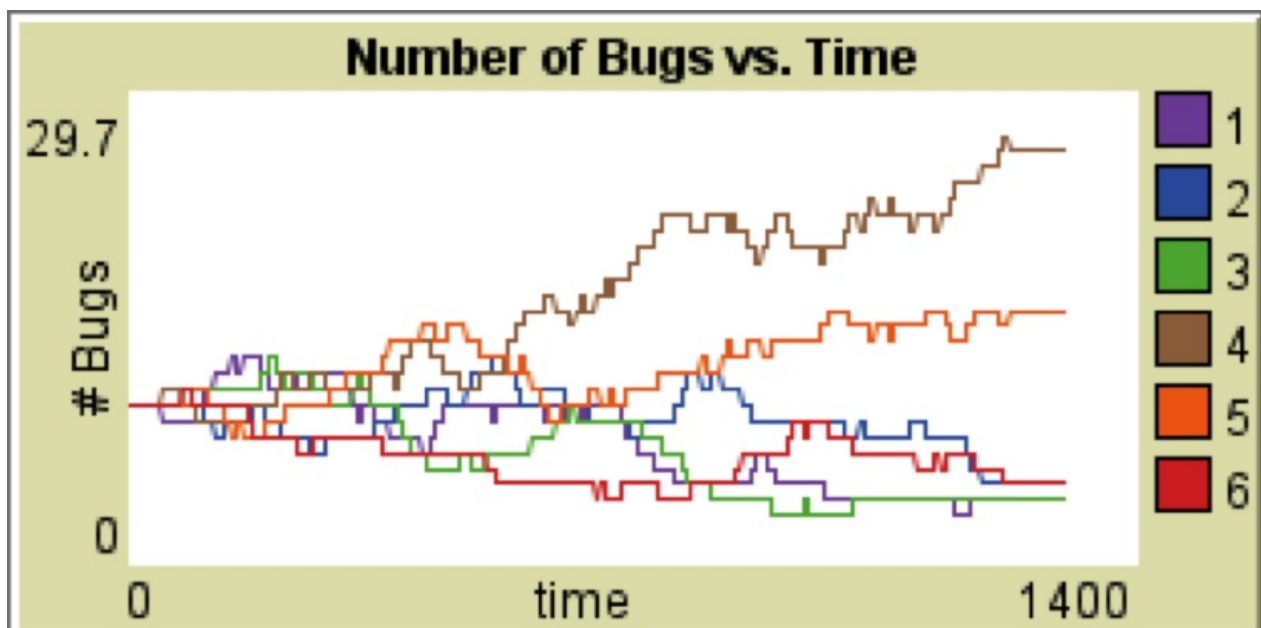


Figure 7: Distribution of types of bugs over time for a typical simulation with random removal.

Summary and NetLogo Primitives

Summary

In this chapter we showed how to model the basic principles of evolution through the creation of variety in the population, selection of agents and inheritable traits. We also introduced a number of primitives such as agents having a cone vision and interacting with the model via a mouse.

NetLogo Primitives

heading

mouse-xcor

mouse-ycor

nobody

towards

in-cone

mouse-down?

mouse-inside?

Tragedy of the Commons

Key concepts

In this chapter we will:

- Introduce the tragedy of the commons.
- Show that agents can self-govern their commons.
- Introduce primitives `diffuse,` `repeat.`

Introduction

Many different types of resources are shared by groups of people and it can be a challenge to avoid overharvesting of the commons. For example, consider fishers fishing the oceans, farmers pumping up groundwater, oil companies drilling for oil, or movie watchers using the limited bandwidth of the community internet connection. In 1968 biologist [Garrett Hardin](#) wrote a famous essay in the journal *Science* titled “[The Tragedy of the Commons](#).” He used the metaphor of sheep herders sharing a meadow. While each herder will benefit as an individual from adding an extra sheep to the meadow, as a group they will bear the costs of the additional grazing and especially when it creates a situation in which grazing occurs faster than the resource can be regenerated. The effect of overgrazing is shared by all herders, but the benefit of the extra sheep goes to the sole owner of the sheep.

To avoid overharvesting of the resource Hardin argued that there is a need for strict governmental regulation or privatization of the resources. Without these interventions the herders would inevitably overharvest. Hardin’s essay was highly influential among emerging environmentalists who were becoming increasingly concerned with the many human-caused, environmental problems. They called government to action and new regulations were written to reduce pollution and avoid overharvesting.



Figure 1: Garrett Hardin.



Figure 2: Elinor Ostrom.

Since Hardin's essay, an increasing awareness also emerged that a tragedy is not the only possible outcome when people share a common resource. There are many examples of long-lasting communities who have maintained their common resources effectively. Since the 1980s there has been an increasing interdisciplinary effort to debunk the simplistic view of the tragedy of the commons. [Elinor Ostrom](#) and others showed through comparative analysis of many case studies that humans can self-govern their common resources. In her classic study "Governing the Commons" Elinor Ostrom defines eight principles that enhance the likelihood of self-governance including effective monitoring and strict boundaries of the resource.

The study of the commons is a broad interdisciplinary research field. Although the main focus has been on environmental resources, applications to health care, digital commons and traffic have been explored.

In this chapter we show through some simple models what happens when agents are greedy, what happens when they imitate other successful agents and what happens when monitoring is included.

Modeling the Tragedy (1)

We present a simple model where agents harvest from a common resource without restrictions. We will see that this leads to an overharvesting of the common resource. There are number of tweaks to the model, which introduces some new primitives again.

First we model the resource. Not every cell has the same carrying capacity. First we set a percentage percent-best-land of the cells at the maximum carrying capacity. This is the best land in the landscape.

```
ask patches
[
  set max-resource-here 0
  if (random-float 100.0) <= percent-best-land
  [
    set max-resource-here max-resource
    set resource-here max-resource-here
  ]
]
```

The 10 iterations are used to smooth the resource values some more, leading to the spatial configuration as in Figure 3.

```
repeat 10
[
  diffuse resource-here 0.25
]
```

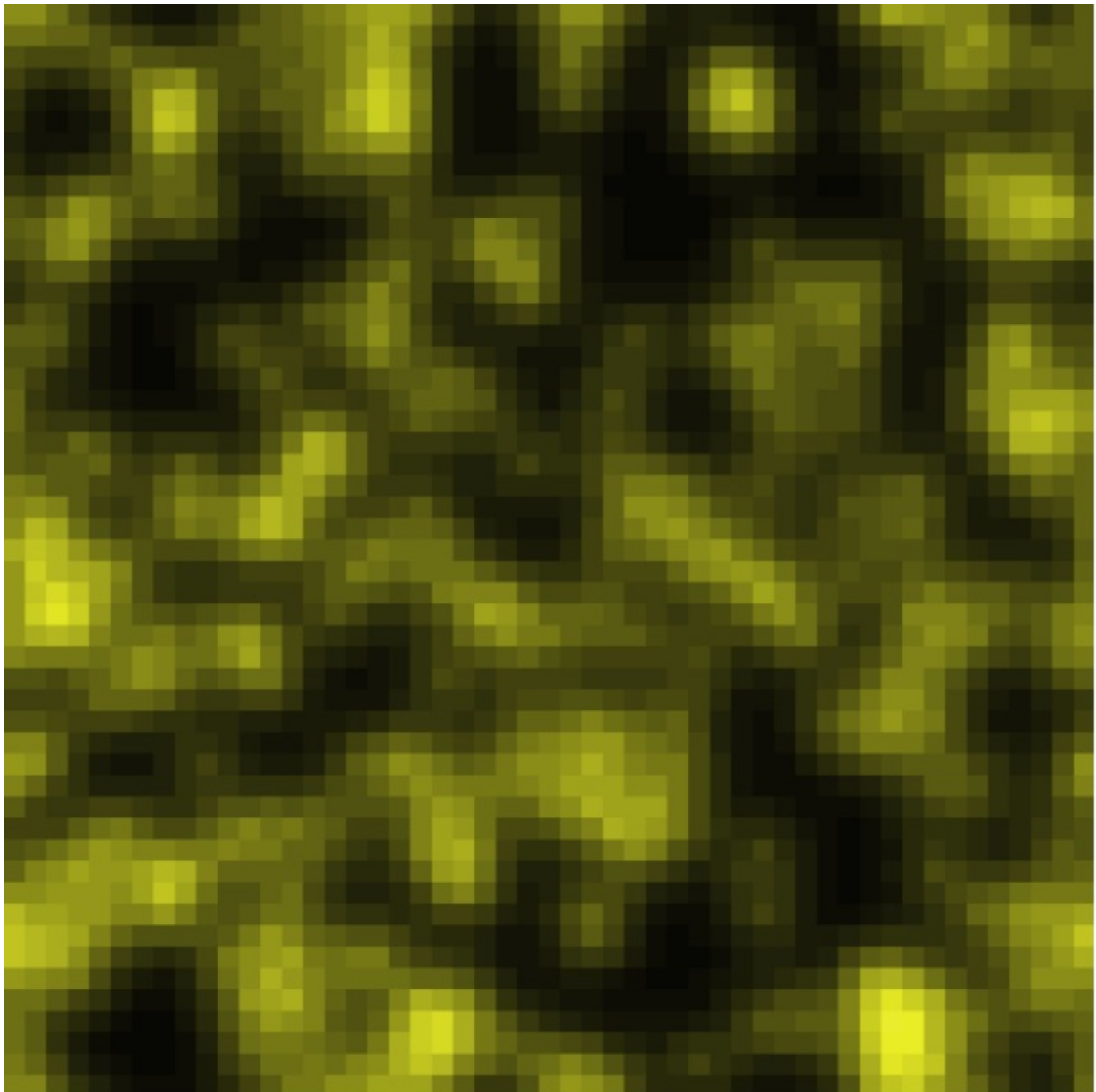


Figure 3: Spatial configuration of the resource on the landscape. Lighter colors refer to higher levels of the resource.

The agent looks to the north, east, west and south patch. For each direction it will look a number of patches ahead according to its vision and gives, as an answer, the resource level of the patch with the highest resource level. The agent then determines which of the neighboring directions has the highest level of the resource. If there is more than one neighbor with the highest resource level one of them is drawn at random. The agent then turns toward the best neighboring patch and makes a step forward.

Each tick the agent will harvest as long as the patch has more than one unit of the resource available. The resource will regrow using a logistic equation using a regrowth rate of 1% per tick. Figure 4 shows the results when agents take resources whenever they are available. These greedy agents lead to a collapse of the resource.

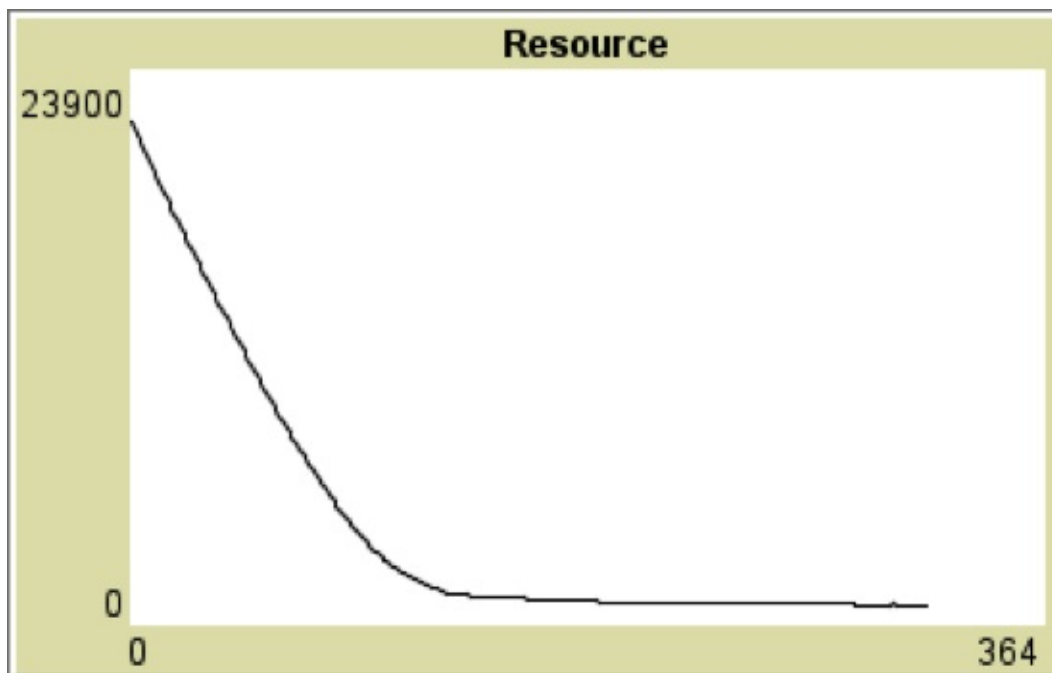


Figure 4. Resource size over time for the greedy agents.

If we include a norm that agents will not harvest if the resource is below a certain level we may derive a more sustainable outcome. We assume that agents will not harvest a resource when it drops below 1 unit, or less than 50% of the carrying capacity of that cell. The maximum growth rate happens with a resource level of 50% of the carrying capacity. Hence if agents follow the norm they will receive a maximum return in the longer term. This outcome is often called the [maximum sustainable yield](#). Suppose we have the logistic growth equation for resource X:

$$X[t+1] = X[t] + r * X[t] * (1 - X[t]/K) - H[t]$$

Where $H[t]$ is the harvest at time step t , r is the regrowth rate and K is the carrying capacity. In the long term, the sustainable solution is to harvest each time step the same amount as the resource generates. This means that

$$H = r * X * (1 - X / K)$$

If X is equal to 0, meaning a collapse of the resource the harvest level is 0. The other equilibrium is when X is equal to $0.5 * K$. This is derived by taking the derivative of $rX(1 - X/K)$ and set it equal to 0.

Figure 5 shows the result is the norm of 50% of the carrying capacity. We see that the resource is not collapsing but stays at a steady level.

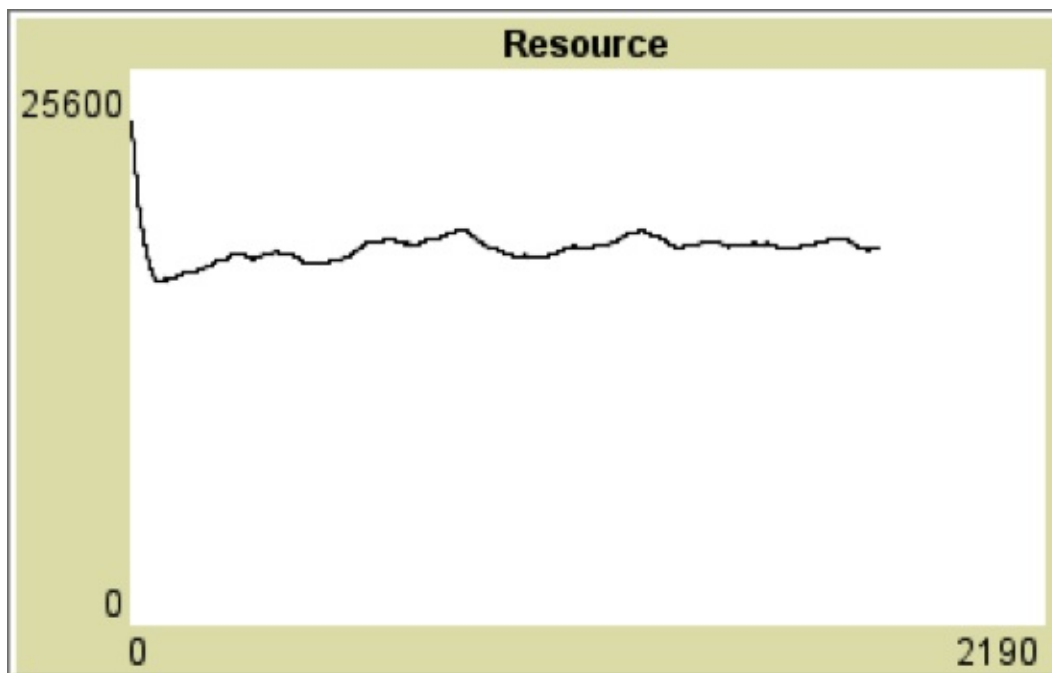


Figure 5: Resource size over time for the norm-obeying agents.

What if agents can imitate the behavior of others? We assume that agents imitate norms used by other agents who have greater wealth than themselves. The bigger the difference the more likely that an agent will copy the behavior of the other agent. Wealth is defined as the discounted sum of earnings of the past. Each tick past earnings are multiplied by 0.95 meaning a discount rate of 5%. This means that recent earnings have a higher weight than earnings far in the past.

If agents imitate the norms of others perfectly, and all agents start with the same norm of 50% of the carrying capacity, we will not see any differences compared to Figure 3. Although some agents may earn more than others, there is no diversity of traits.

Modeling the Tragedy (2)

We now introduce that agents may make errors in reading the norm that others use. If they imitate, they imitate the norm used by the other agent, but we will add a noise term to it. The errors in imitation may lead some agents to start using a norm above 50% and others below 50%. Over time we see that the norm will drive towards 0% and the resource will be depleted as shown in Figure 6.

How do we interpret this result? Agents are willing to obey norms of sustainable use of the resource but will adjust their strategy if they see that others do better by not obeying the norm. In the model all agents had good intentions, yet just by imperfect copying we get a crowding out of the moral norms of sustainable use of the resource. Hence good intentions are not sufficient for a sustainable use of the resource.

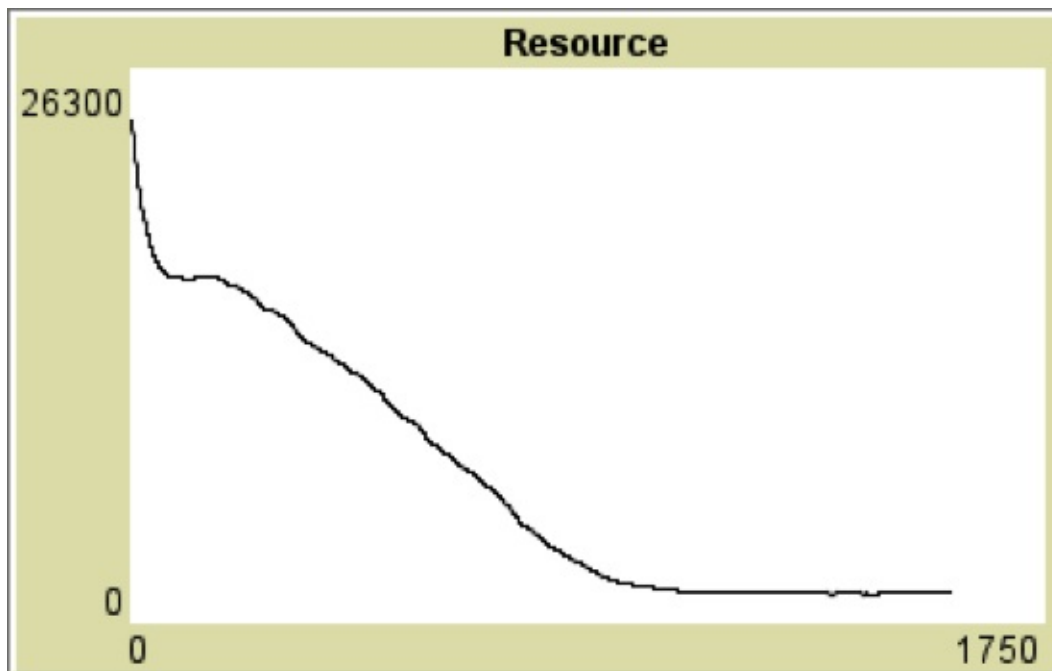


Figure 6. Resource size over time for norm-obeying agents who imperfectly imitate better performing agents.

We now include the option that agents can give each other a warning. For a small cost one agent can reduce the earnings of another agent who is caught breaking the norm. In the least, an agent can sanction those who harvest when they would not harvest. A third agent can also then sanction the first agent, if that agent breaks the norm that the third agent was assuming.

Each turtle will check which agent in its vision has the lowest `harvestedlevel` used. The variable `harvestedlevel` is calculated in `harvest` by dividing `resource-here` by `max-resource-here`. The primitive `in-radius` defines a radius of size `radius` around the agent.

```
let otheragent min-one-of turtles in-radius radius [harvestedlevel]
```

When there is another agent in the radius, the agent checks whether this agent uses a lower norm than itself. If so it will punish the agent and reduces its wealth with `costpunished`, at a cost of `costpunish` to itself. Agents are not allowed to punish if their wealth is not more than `costpunish`

```
if otheragent != nobody [
  if [harvestedlevel] of otheragent < norm-min-resource [
    set wealth wealth - costpunish
    ask otheragent [set wealth wealth - costpunished]
  ]
]
```

When an agent gets punished so frequently that its wealth drops below 0, we replace it with the norm of a random other agent and put its wealth equal to 0. Figure 7 shows that the resource diminishes slowly if agents use peer punishment. The `radius` used is 10, the `costpunish` 0.01 and `costpunished` 0.1. The level of punishment is relatively high. It is more than a verbal warning or a slap on the hand. We will explain below why this was needed.

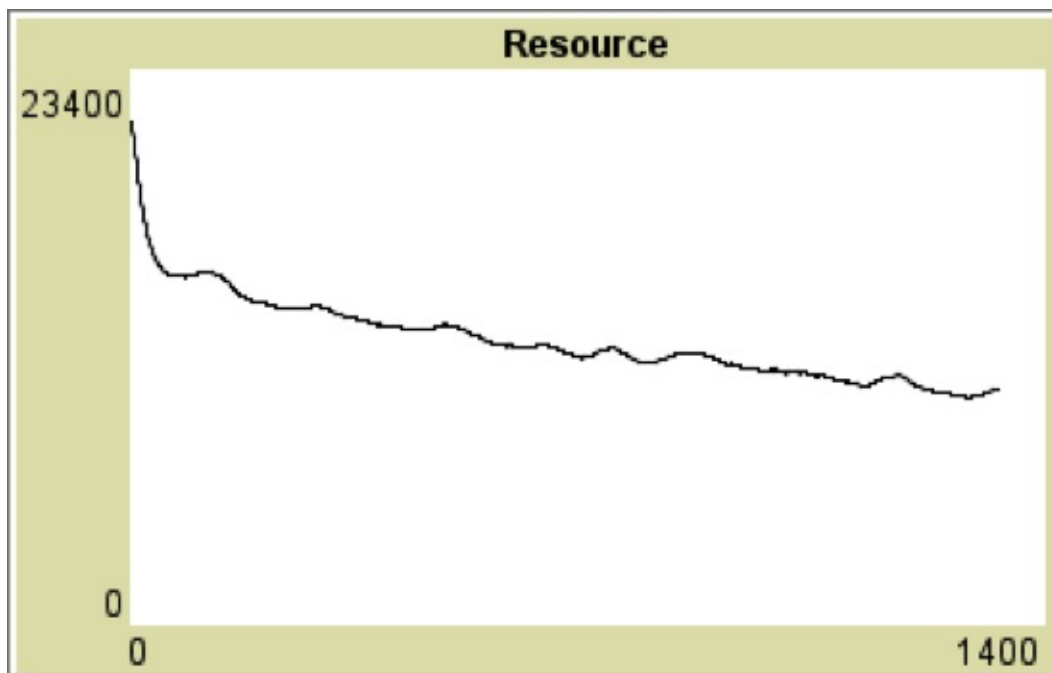


Figure 7: Resource size over time for simulation with punishment.

Figure 8 shows the average level of the norm. On average the norm slowly declines below 0.5. However, if we look at Figure 9 where the avatars are sized to the norms they use, we see a huge variability. Some agents are very greedy and collect whenever they want. The punishment they may receive from others is not sufficient to stop them.

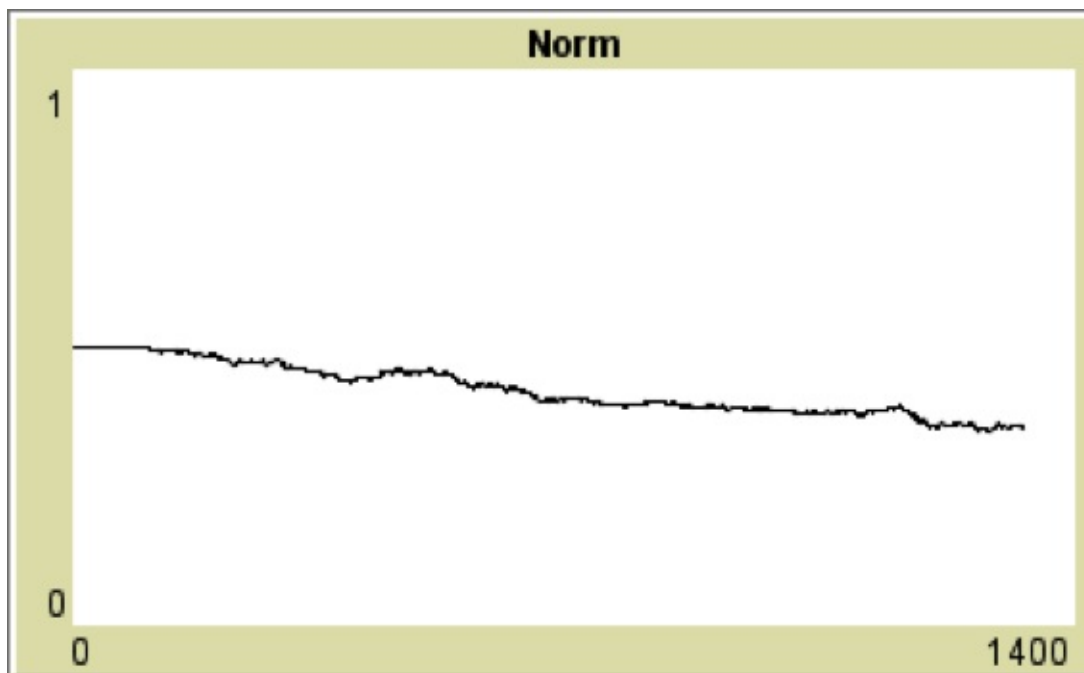


Figure 8: Norm-min-resource on average of the agents over time for punishment simulation.

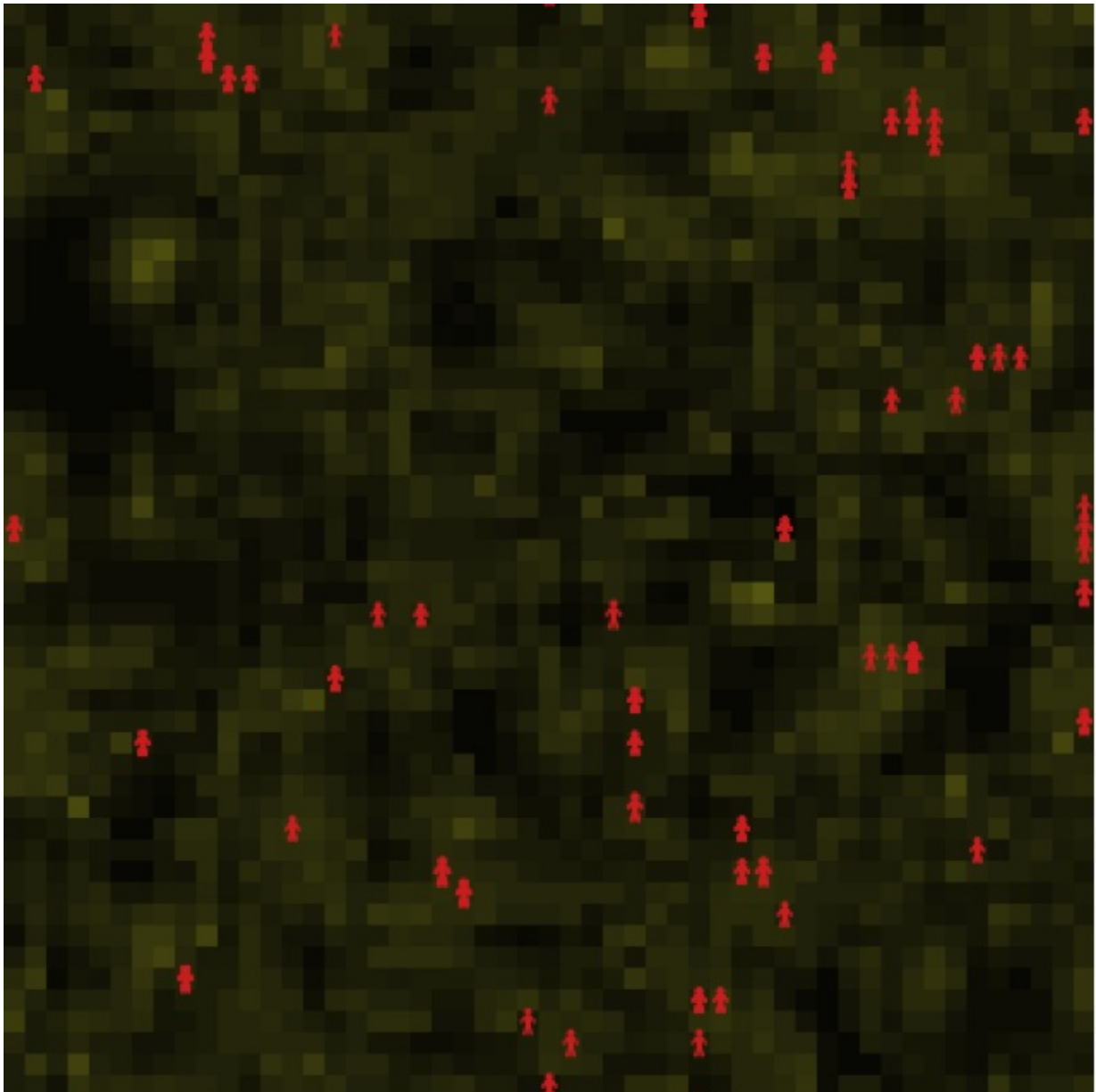


Figure 9: Avatars of agents sized to the level of the norms agents use.

One of the challenges is to anchor the norm to the sustainable level. The agents use informal norms and they may forget the sustainable norm they originally started with. How do we maintain the memory of the agents? One option is to start having formal rules that state clearly what the official allowable levels of resources may be. Furthermore, the sanctioning level might not be the same for everybody. Agents who persist in breaking the rule may get higher penalties. This is what we call graduated sanctioning. The first time agents break a rule, they get a warning, and over time they get higher and higher sanctions. You may adjust the model to include those aspects and see whether you get a sustainable outcome.

NETLOGO EXAMPLE: HARVESTING RESOURCES

Model: Right click on the Link and Save it. Open Netlogo and Run it with NetLogo.

Summary, References and NetLogo Primitives

Summary

We show that a resource will not be well maintained if agents are opportunistic and harvest without restrictions. The resource does much better if agents obey a norm of sustainability, but this is sensitive to how accurately agents adjust their norms when imitating others. To reduce the likelihood of crowding out of norms of sustainable use agents can sanction each other.

References

Hardin, Garrett (1968). Tragedy of the Commons. *Science* 162: 1243-1248.

Ostrom, Elinor (1990). *Governing the Commons*. Cambridge University Press.

NetLogo Primitives

repeat

diffuse

Financial Resources

Key concepts

In this chapter we will:

- Explore the consequences of banking rules on inequality
- See the effect of herding behavior in stock markets
- Introduce the Gini coefficient
- Introduce the primitive `sort`

Introduction

Financial resources have different properties than physical resources. We can borrow money to buy something that we want to have now and pay it back —with interest—in the future. If everybody expects a company will be successful the stock price will go up. Some people will even benefit from a dramatic downward trend in stock prices if they had bought options to sell stocks at high prices, which then enables them to buy those depressed stocks cheaply and sell them once they regain their value.

The economic and financial crises in recent years showed unexpected behavior in our economy's performance that was not predicted by traditional economic models. In this chapter we will discuss two examples of using agent-based models to understand economic systems. The first example is about a bank that gives loans to people. How much money does a bank need to have in available cash to do their business and how do rules on how much reserves are required affect income inequality? In this example we also discuss the Gini coefficient, a measure of inequality.

The second example focuses on herding behavior in financial markets. Newspapers report on bubbles in financial markets. What are these bubbles and where do they come from? We show that herding behavior, people buying when other do, leads to bubbles.

The two examples we discuss in this chapter illustrate how models can help us to provide some understanding on phenomena in economic systems by exploring the consequences of some rules on interactions in markets.

Banking Rules

There is more money in the economy than people have earned. This is due to the fact that banks give people loans. In fact they give more loans than they have money in the bank. Since people who have money in savings deposits do not come the next day to ask about their money, banks use savings dollars for loans multiple times. The proportion of the total savings that banks keep on-hand is called the bank's reserves.

The government regulates the reserve ratio level, mandating how much the banks need to keep in reserve. The bank in the model is a super bank that represents all banks in an economy.

The model represents an economy in a very simple way. When two agents are on the same patch they will give a few dollars to another agent. After this exchange agents make up their balance and then sort out how much money they have in their wallet. If the agent has money in their wallet they will put this in savings or pay off their debt if they have savings. If they don't have savings and they have a negative balance from the exchange during the last tick, they will have to take a loan if the bank has funds available. If this is not the case the agent will start with a negative balance in the next period. In all, agents pay off their debts as soon as they have money available, and take loans when they have spent more than they have and loans are available.

The basic algorithm in the go procedure is as follows.

```
ask turtles [  
  ifelse ticks mod 3 = 0  
    [do-business] ;;first cycle, "do business"  
    [ifelse ticks mod 3 = 1 ;;second cycle, "balance books"  
      [balance-books]  
      [bank-balance-sheet] ;;third cycle, "bank balance sheet"  
    ]  
]
```

For the detailed procedures we refer to the model's code. Nevertheless, first we will discuss one variable that we calculate in the model, namely the [Gini coefficient](#). The Gini coefficient is a measure of inequality and named after Italian scholar [Corrado Gini](#) who proposed this measure in 1912. If the Gini coefficient is 0 there is perfect equality where all values of a frequency distribution, such as income, are the same. A Gini coefficient of 1 means that there is a maximal inequality, for example if one person has all the income.

To calculate the Gini coefficient we first order the income from lowest to highest. Then we create a curve of the cumulative earnings by the bottom x% of the population and relate this to the x% of the population (See Figure 1). We also can create a 45 degree line which represents that x% of the population get x% of the cumulative income, which means a perfect equality of income. The Gini coefficient is now the ration of the area that lies between the line of equality and the curve over the total area under the line of equality as shown in Figure 1.

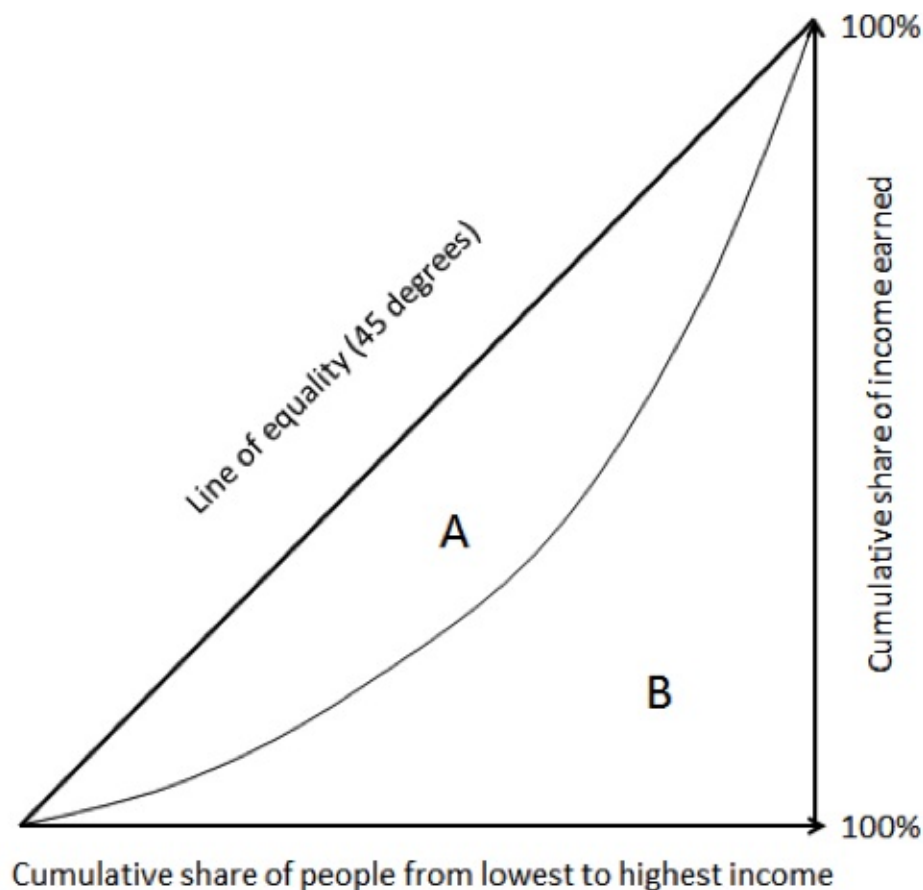


Figure 1: The Gini coefficient is calculated as $A / (A + B)$.

To implement this in Netlogo we make use of the primitive sort by which we sort the agents in the order of their wealth. Then we add the difference of the equality line and the curve for each agent, and scale it to get a value between 0 and 1.

```

let sorted-wealths sort [wealth] of turtles
let total-wealth sum sorted-wealths
let wealth-sum-so-far 0
let index 0
set gini 0

repeat people [
  set wealth-sum-so-far (wealth-sum-so-far + item index sorted-wealths)
  set index (index + 1)
  set gini gini + (index / people) - (wealth-sum-so-far / total-wealth)
]

set gini (gini / people) / 0.5

```

We will now show some results of the model. The first simulation (Figure 2) shows the outcome if banks only need to keep 10% of their savings in the bank. We see that the amount of money in the economy increases substantially. Furthermore, there are many rich and poor people, and almost no middle class. People have a lot of debt or a lot of money on the bank.

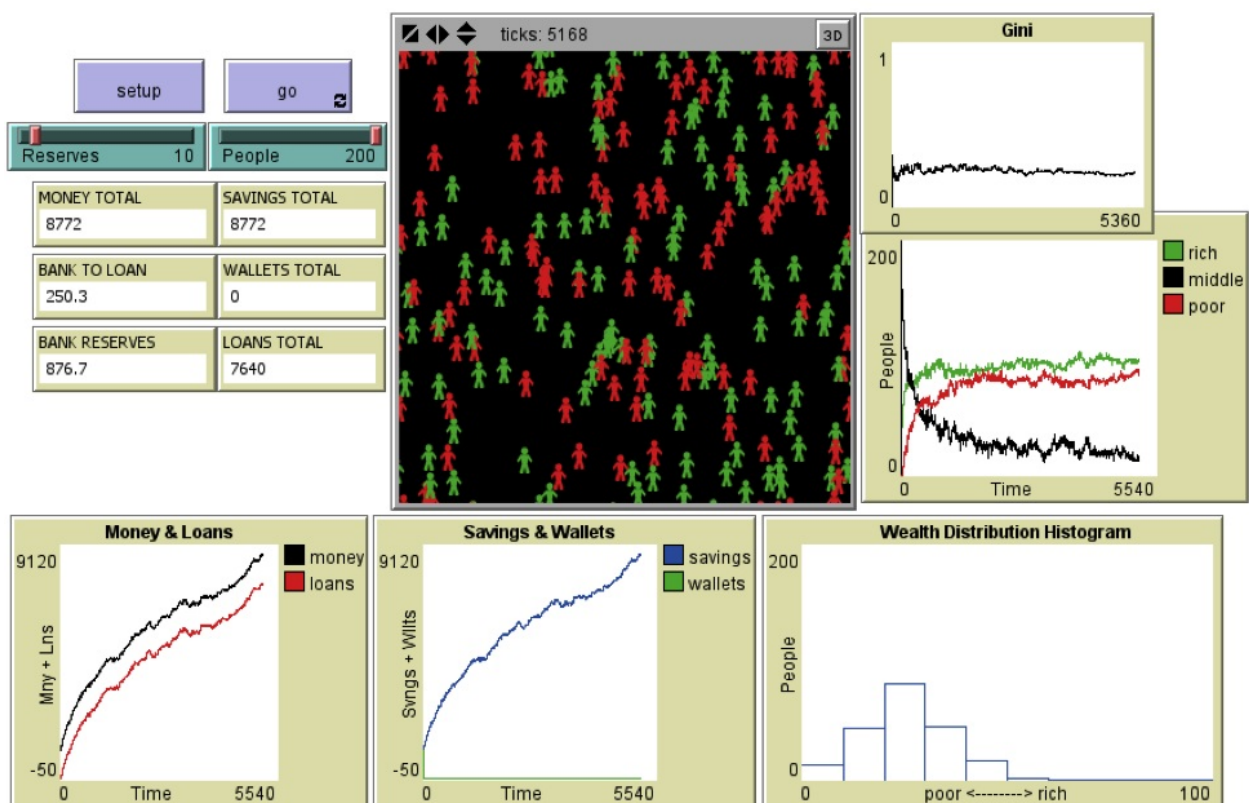


Figure 2: Screenshot of results if Reserve slider is set at 10.

When we put the Reserve slider at 100 there is no generation of money (Figure 3). Now there are many middle class agents and a few very rich agents.

One limitation of the model is that there are no interest rates. Banks make money by charging a higher interest rate to those who borrow money than they give to those who have savings in the bank. Try to implement interest into the model and see what the bank earns and what happens to the money in the economy.

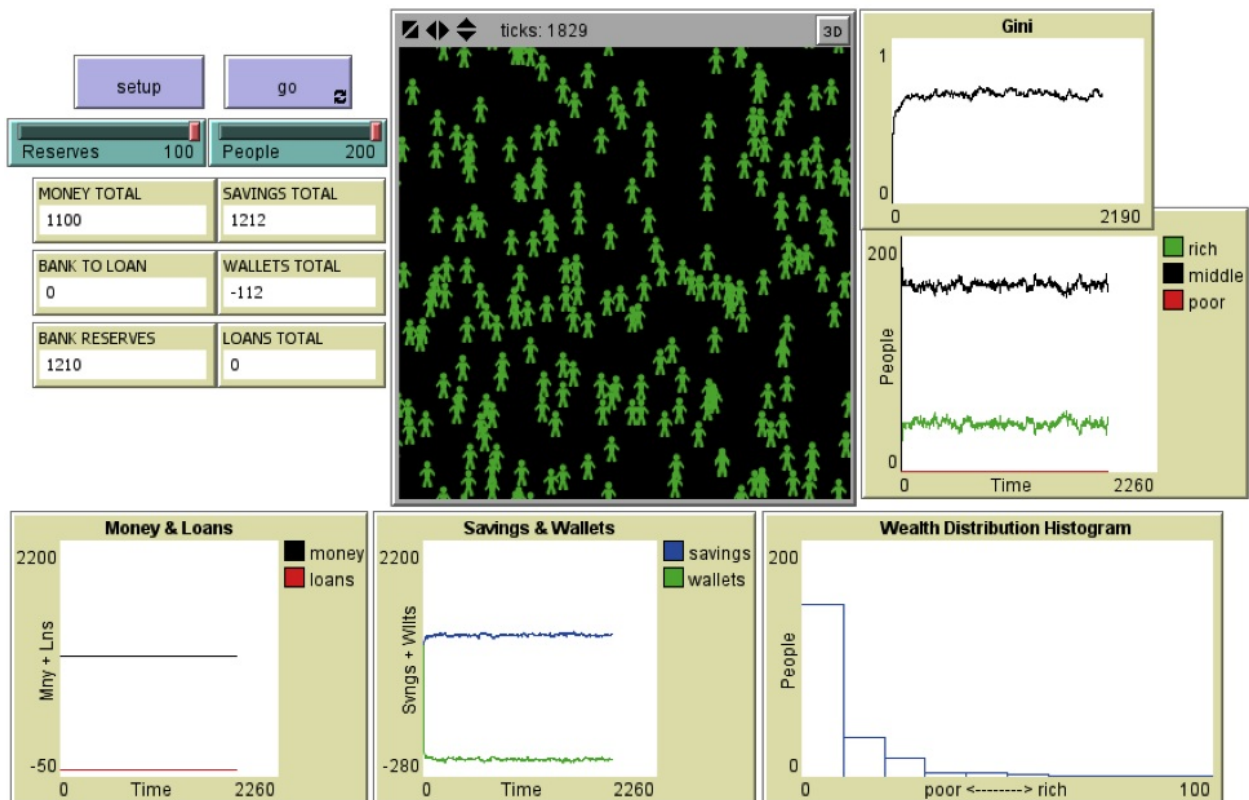


Figure 3. Screenshot when Reserve slider is set at 100.

NETLOGO EXAMPLE: BANK RESERVES

Model: Right click on the Link and Save it. Open Netlogo and Run it with NetLogo.

Financial Bubbles

The price of a stock on the financial market represents the expected value of the company, which depends on the expected returns in the coming years. If everybody has the same information and all relevant information is public you would expect that the price would reflect the true value of the firm. Some people might be a bit optimistic and others a bit pessimistic, which may lead to trade of the stock. But as long as individuals don't have foreknowledge there is no reason to expect that the price does not represent something other than the true value.

However, this is not what we see with actual financial markets, which can experience dramatic “mood” swings leading to major fluctuations in price. The reason that this happens is due to what is called herding behavior ([link is external](#)). People start to imitate the actions of others and sell when others sell, not because of individual price expectations, but because opinion on the quality of a financial asset changes due to interactions with others. When we get clusters of similar opinions this may lead to bubbles.

Each patch represents a trader who has a position on the market who may want to buy or sell a stock. If the position is -1 the agent wants to sell, and if the position is 1 the agent wants to buy. In the traditional model the position is influenced by white noise `random-normal mean-pnoise stdev-pnoise`, which leads an average of 50% of the agents to be positive and the others to be negative. This will lead to a kind of random walk. We implement this in the following way.

```
ask patches [  
  ifelse ((random-normal mean-pnoise stdev-pnoise) > 0)  
  [  
    set trader-position 1  
    set number-of-shares number-of-shares + 1  
  ][  
    set trader-position -1  
    set number-of-shares number-of-shares - 1  
  ]  
]
```

The price goes up if there is a net sum of buyers and goes down if there is a net sum of sellers. Since the market is assumed not to be perfect, some noise is added to the price. In the results below we assume that `mean-noise` is equal to 0 and `stdev-noise` is equal to 0.25.

```
set price-converter 0.001
set returns sum [trader-position] of patches * price-converter
set noise random-normal mean-noise stdev-noise
set log-price (log-price + returns + noise)
```

Figure 4 shows how the positions of buying and selling are distributed in the landscape. The pattern is very much like a random distribution.

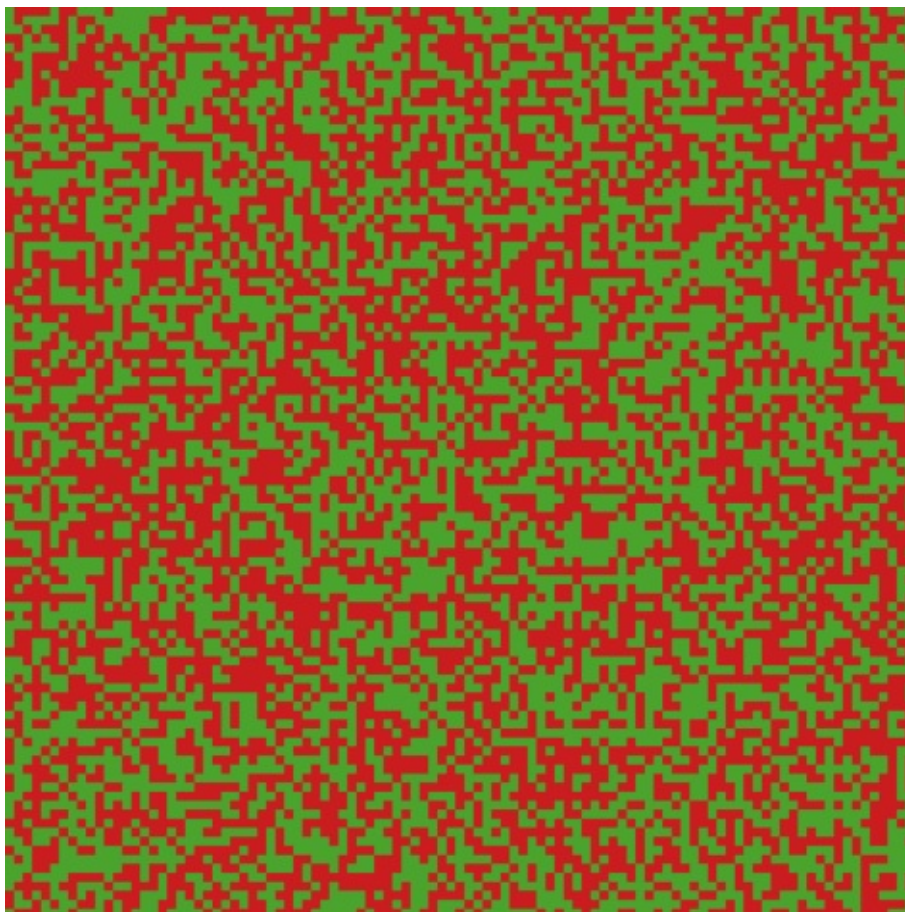


Figure 4. Map of the positions of traders. Red is selling and green is buying.



Figure 5. Price development over time.

When people are influenced by their neighbors we will get herding behavior. We model this herding behavior by having agents take into account the positions of their four neighboring traders, north, east, west and south. The sum of the positions is weighted by A . Thus if the neighboring agents have a selling position, with a high value of A the agent will also have a selling position. A is assumed to be agent-dependent and drawn from a normal distribution with mean 0.5 and standard deviation 0.25 in the results below.

```
ifelse ((A * sum [old-position] of neighbors4 + random-normal mean-pnoise stdev-pnoise) > 0)
```

Note that we have to check the `old-position` of the neighboring agents if we assume sequential updating of the information.

Figure 6 shows the spatial distribution of herding agents. We see now pockets of red and green agents. This leads to mood swings in the market. Figure 7 shows the price fluctuations on the market, which are much more extreme compared to the modest price fluctuations in Figure 5. Prices go up for a while leading to a bubble and can then experience a dramatic collapse. Note that the equilibrium price is 4.

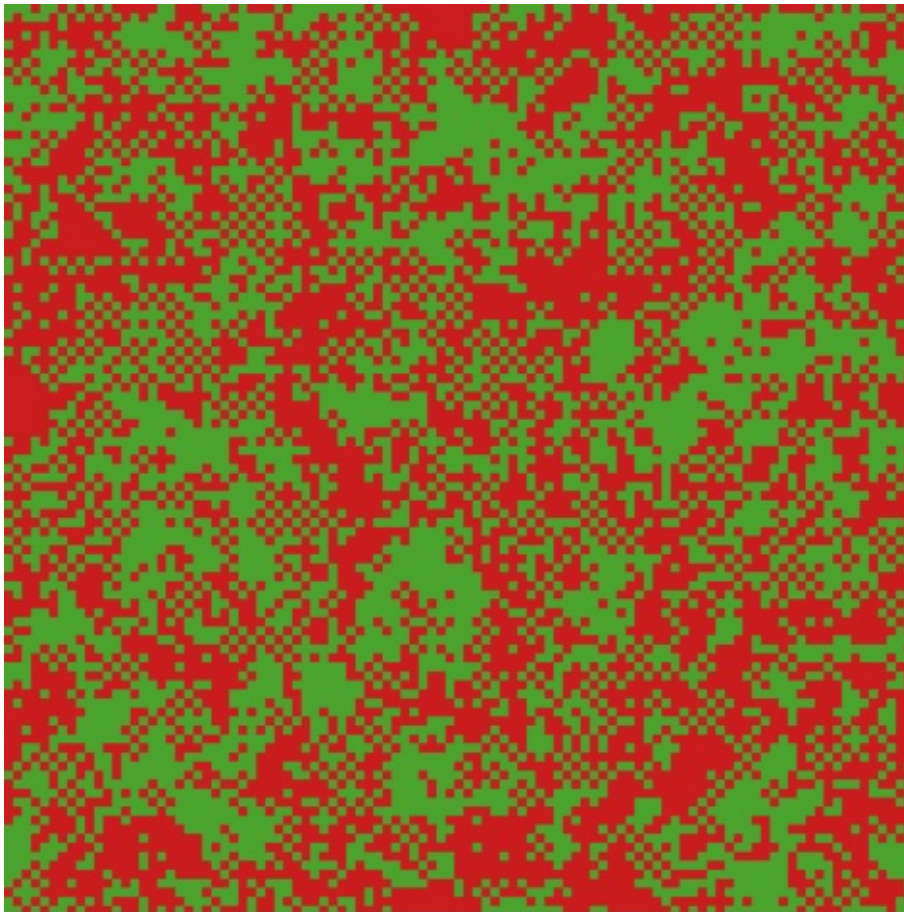


Figure 6: Map of sellers and traders with herding effects.

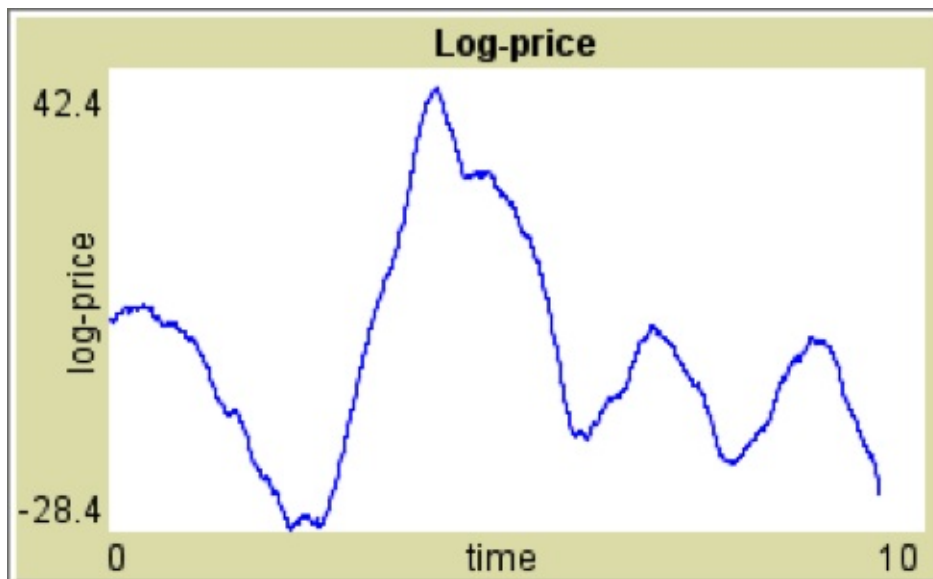


Figure 7. Price development over time.

NETLOGO EXAMPLE: FINANCIAL BUBBLES

Model: Right click on the Link and Save it. Open Netlogo and Run it with NetLogo.

Summary and NetLogo Primitives

Summary

In this chapter we discussed two examples of financial resources. We show how rules for banks affect the amount of money in the economy and how herding behavior leads to bubbles in financial markets. We also introduced the Gini coefficient, which is a metric to measure inequality.

NetLogo Primitives

`mod`

`sort`

Agents & Their Friends

Networks

Key concepts

In this chapter we will:

- Introduce the concept of networks.
- Present different types of networks such as random network, small world network and scale-free network.
- Discuss metrics to compare networks such as degree, path length and clustering coefficient.
- Introduce the primitives `link`, `link-neighbors`, `end1`.

Introduction

In previous chapters the interactions were defined by the environment, agents running around looking for resources, or by random interactions. These models did not include the observation that in real social systems interactions are not random.

Nowadays, many people are part of social network websites like [Facebook](#). Members are not connected by a random sample of other members, but they are connected with their “friends.” If we look at the networks in social network websites we will observe a structure. The structure of a network can affect a number of social phenomena, for example, the spread of a rumor. If a well-connected Facebook member puts a rumor on her page, this information will spread much faster among the Facebook community compared to the case in which the same rumor is put on the page of a person who has only one friend.

When we include network structures into our agent-based models we will be able to study the effect of network structure on the diffusion of information, norms, diseases, behavior, etc. We will discuss various applications in the coming chapters. This chapter will discuss the basics of networks.

Basics of Networks

A network is the collection of nodes that are connected by links. A node can represent a person, a family, a nation, a computer, a species, etc. Links defines a relationship between nodes such as friendship, kinship, a highway, who-is-eating-whom, etc. In the Netlogo model we will have turtles defined as the nodes. The links are defined by a special category `link` next to `patch` and `turtle`.

Links can be directed. A link that represents that agent A is the parent of agent B is a directed link (Figure 1). But a link that represents a friendship between agents A and B can be an undirected link. That is, if both agents see each other as friends.

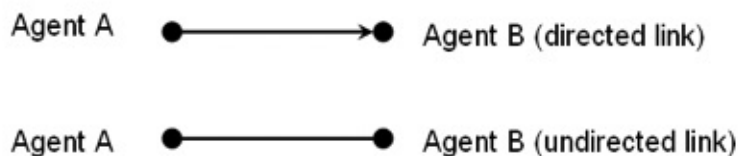


Figure 1: Directed and undirected links between agents A and B.

Each agent can have a number of links. The number of links an agent has is called the degree. If an agent has five undirected connections, then we would say that the degree of the agent is five (Figure 2). In some networks the degree is the same for each agent, while in others the degree is different for each agent. The distribution of degrees in a population of agents is an important characteristic of a network.

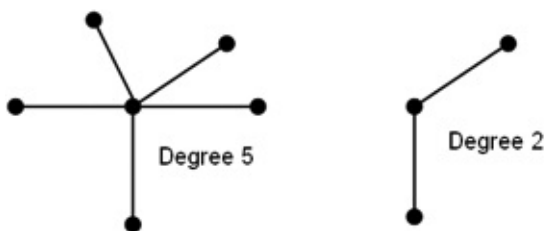


Figure 2. Central node has degree 5 (left) and 2 (right).

Another characteristic of networks is defined by the paths between nodes. In the figure below you see a path between the two red nodes. There are various paths possible. There are even different shortest paths of four links between the red nodes. We can calculate the average shortest path between each two nodes of a network. This leads to the [average shortest path length](#) of a network. Network structure affects the length of the shortest path and therefore it becomes an important indicator to compare networks. The shorter the average path length, the faster we will expect information will diffuse within a network.

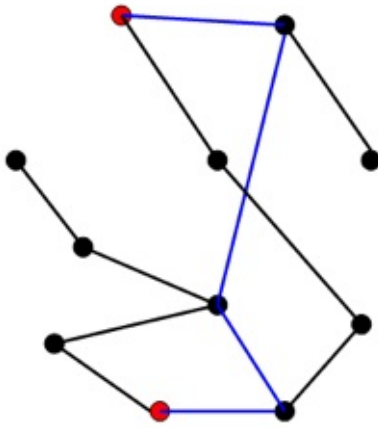


Figure 3. Shortest path – 4 links – between two red nodes.

Finally we look at **clustering coefficient** of a network. This coefficient indicates how frequently links of a node are linked with each other. In social networks we can think of friends of person A also being friends of each other. A higher amount of friends of friends leads to more clustering. In the figure below you see that the red node has three (yellow) friends, but none of these friends are connected. Thus the clustering coefficient of the red node is zero. To calculate the clustering coefficient for a network, we take the average of the clustering coefficients for all nodes of a network.

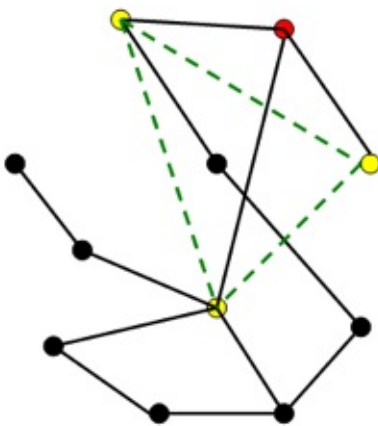


Figure 4. Dotted green lines are missing connections between 3 yellow friends of red node.

A network is visualized as a graph. Often these graphs have no spatial representation. It has when it represents highways or other physical networks. Otherwise, these graphs are just a graphical representation of the relationships. We will now discuss a number of different types of networks and see how they differ in the network metrics degree, path length and clustering.

Random Networks

Let's start with looking at a simple network, the random network. In this network there is a population of n nodes. One by one we add links between nodes. We do this by randomly picking one of the nodes, and linking them randomly with another node.

In the Netlogo code, you see that adding a random link is done as follows. Pick two turtles and check whether there is already a link between them. To do this we make use of `link-neighbor? x`, which is a primitive that is true if there is a link between the turtle and X . If there is no link already we can create a new link between the two turtles using `create-link-with x`.

```
to add-edge
  let node1 one-of turtles
  let node2 one-of turtles
  ask node1 [
    ifelse link-neighbor? node2 or node1 = node2 [add-edge]
    [ create-link-with node2 ]
  ]
end
```

In Figure 5, you see how the network is visualized during the construction of links. The red links and turtles are the largest connected component of the network.

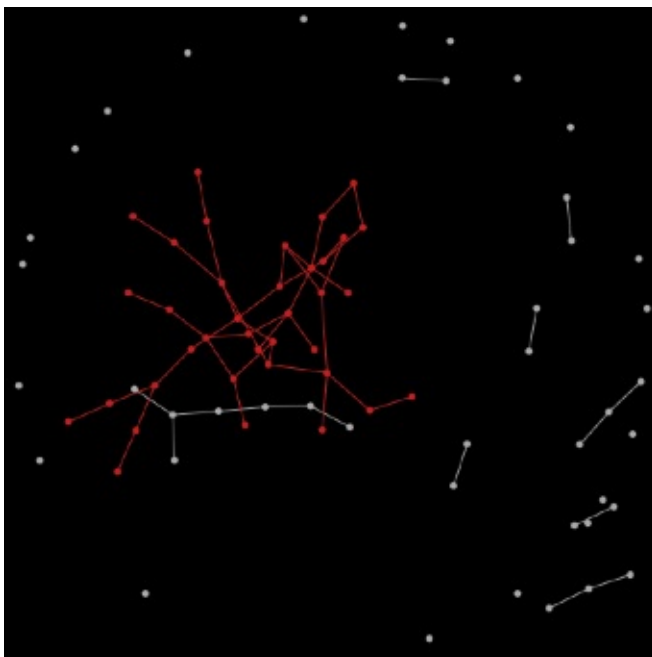


Figure 5 Creating a random network.

We can visualize the relative size of the biggest component over time (Figure 6). Initially the size is very small, just a few turtles. Then there are jumps, as small components get connected. It takes a while until all turtles are connected.

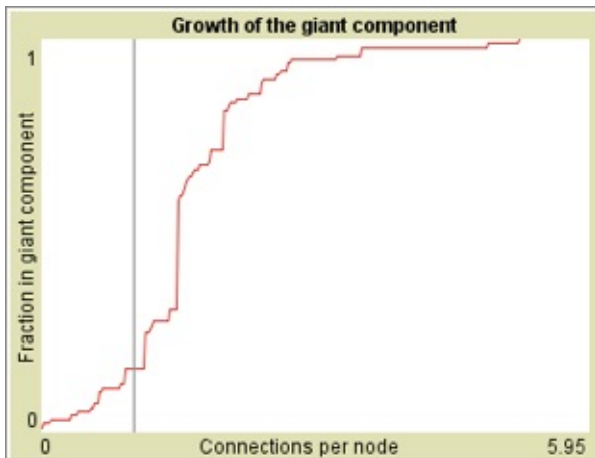


Figure 6. The fraction of the biggest component of connected turtles as part of the whole network. The vertical line is the moment in time where there are as many links as turtles.

Not all turtles have the same number of links (Figure 7), but they will each have a minimum of 1, since the model ran until all turtles were connected. In the example simulation we see that the number of links varies from 1 to 12. The median degree is 4. The clustering coefficient is 0.054 and the average path length is 2.96 links.

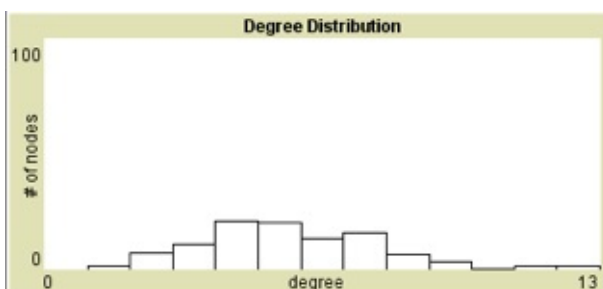


Figure 7. The distribution of degrees in the example run.

NETLOGO EXAMPLE: NETWORKS

Model: Right click on the Link and Save it. Open Netlogo and Run it with NetLogo.

Small World Networks

One may have experienced an encounter with a stranger who is linked within just a few connections with you. In this way, one might be just a few handshakes away from the president of the United States. Let's define "handshake" here as literally shaking someone's hand as you meet in person. Let's exclude the handshakes presidents give to the crowd when they have rallies. So, how many handshakes are you away from the president? You may be surprised at how small this number is for most people. Let's do this with a person picked at random from the billions of adults living on this planet and try to define how many handshakes they are away. Any guess? Here we would expect that this number is not much more than a handful of handshakes.

This suggests that the average path length of social networks around the world is small. Psychologist [Stanley Milgram](#) did an experiment in the 1960s where he asked a number of people in the midwestern U.S. to send a letter to somebody in Boston. However, they did not receive the address of the person, but just a number of hints, like occupation. To reach their target, people had to send the letter to somebody they thought would have more success in finding the target. Milgram was interested in whether the letters would arrive at their target address, and how many connections it would take. Not all letters reached their target, but of those that reached the target the average number of links was six. This led to the saying that all people are on average just six handshakes away. This experiment was replicated 35 years later with emails, and again the average number of links between the source and the target was found to be six.

Is the random network a good model for social networks? No, since social networks have another distinctive attribute. Many friends of a person are also friends of each other. This is called clustering within networks. To calculate the clustering coefficient of a node we will look at all the friends of this node, and count how many possible friendships exist between nodes. The total actual friendships among friends were divided by the total possible friendships. This is the clustering coefficient, which was 0.054 in the random network example. In random networks this coefficient is not very high since many connections are random, and friends are not friends of each other.

We will now look at a [small world network](#) that captures both the high level of clustering and a short average path length. The model was developed by [Duncan Watts](#) and [Steven Strogatz](#). They started with a regular network of nodes that are arranged in a circle. Each node is connected to two nodes to the left and two nodes to the right. This means that the

clustering coefficient is 0.50, since 50% of the links between the friends of a node are connected with each other. For a network of 100 nodes, the average path length is about 13 links.

When we relink a few nodes the average path length drops significantly. In the figure below, the relink of four nodes leads to a drop in the average path length from 13 to 9 links. To relink a node, we chose a random node A and then chose randomly from this node one of the links with another node B. The node B is then replaced with a randomly chosen node from the whole population of nodes, except node A and existing nodes of node A.

A new primitive we use is `end1`, which indicates the first turtle of a link. In a directed link this will be the source, and for undirected links it will be the turtle with the lower who number. We first check whether new links can still be added to the turtle of interest. If this is the case, a turtle is drawn who is not yet connected to the turtle of interest. We then create a link between both turtles. In Netlogo this looks like

```
let node1 end1
if [ count link-neighbors ] of end1 < (count turtles - 1)
[
  let node2 one-of turtles with [
    (self != node1) and (not link-neighbor? node1) ]
  ask node1 [create-link-with node2]
]
```

When we start relinking, we will make cross-cuts from one part of the network to another part (Figure 8). This will lead to shorter paths between ends of the network.

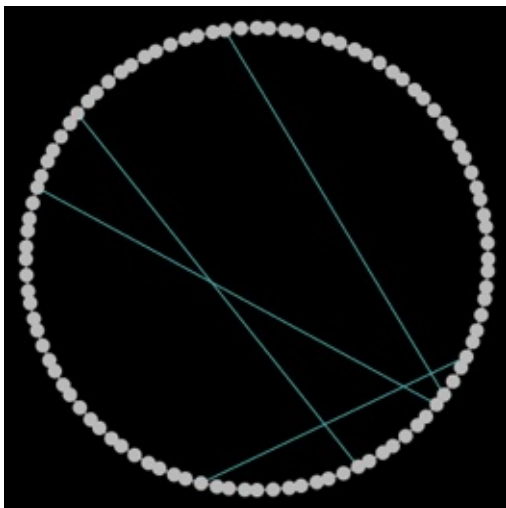


Figure 8. Relinking four nodes of a regular network.

If we relink many nodes, the average path length will initially drop quickly. The average cluster coefficient stays initially high. As we see in Figure 9, the shortcuts reduce the path length between nodes from both sides of the network, but do not affect the linking among friends. As we see in Figure 9, just a few relinkings lead to networks with high clustering and

a low average path length. This is the characteristic of small world networks. If we will relink all the nodes, we will end up with a random network with low average path length and low amount of clustering.

What we call a small world network, is a network with just a small fraction of short cuts. This leads to a high clustering coefficient and a short path-length. This is also the kind of network that we find in social networks. Most people are clustered and friends of friends are friends. But if just a small percentage of the friendships are with very different people, the average path length will be small.

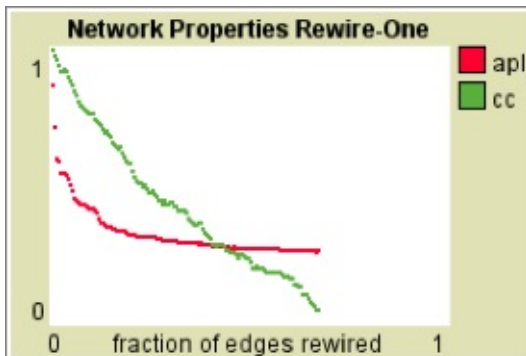


Figure 9: The average path length (apl) and clustering coefficient (cc) for increasing number of nodes relinked in a network of 100 nodes.

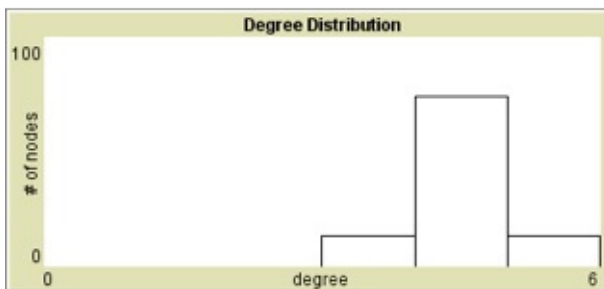


Figure 10. Degree distribution of small world network when 10% of the nodes are rewired.

Scale Free Network

The final cartoon network we will discuss is the scale free network. The term cartoon network is used since actual networks do not exactly follow the idealized networks we are modeling. More detailed simulation models are used to capture more detailed network characteristics of specific networks.

The scale free network captures the observation that the degree distribution is not normally distributed in many real world networks as we saw in Figures 7 and 10. For example, many websites do not link to others, but some have many links. This last category is considered to be a hub. If we plot the degree distribution on a [log-log scale](#) (Figure 11) we see a straight line. If it were a normal distribution, the frequency of observations with a high in-degree, say 100, would be close to zero. This means that in a scale-free distribution, the distribution has [fat tails](#). There are more outliers than would have been in a normal distribution.

These kinds of scale-free degree distributions have been found in many networks. Often they do not follow a straight line for the whole distribution, but for a large part, indicating fat tails of the distribution.

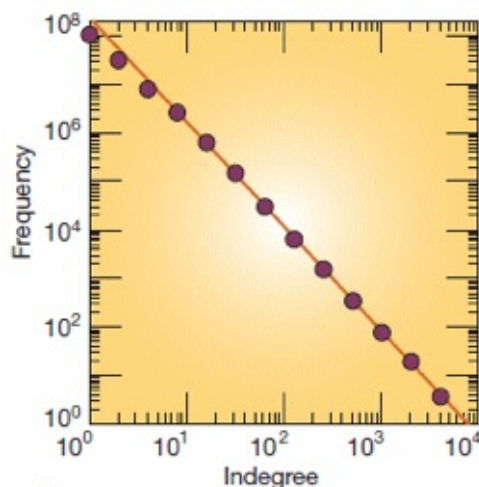


Figure 11: Degree distribution of world-wide web. Nodes are web pages; links are directed URL hyperlinks from one page to another. The log-log plot shows the number of web pages that have a given in-degree (number of incoming links). Source: Strogatz (2001).

Barabasi and Albert (1999) published a simple model that generates networks that have a scale-free degree distribution. The model is based on [preferential attachment](#). Starting with two nodes, every new node will be linked to the existing network, but with a higher preference to nodes with higher degree number. Thus nodes with high degree have a higher probability of getting more nodes connected. This is also called the “rich become richer” phenomenon.

To model the process to draw a connecting node based on the degree of the nodes already in the network, we implement the following in Netlogo. First we draw a random value between 0 and the total degree number in the network. Then in random order the degree of turtles are added until the algorithm surpasses the cumulative number of degrees the random number initially generated. This is a partnering turtle that is drawn to be connected to the new turtles. The higher the level of the degree the more likely that turtle will be drawn.

```
let total random-float sum [count link-neighbors] of turtles
let partner nobody
ask turtles
[
  let nc count link-neighbors
  if partner = nobody
  [
    ifelse nc > total
    [ set partner self ]
    [ set total total - nc ]
  ]
]
```

In Figure 12 you see a snapshot of the network generation in process. The size of the nodes is proportional to the degree. We see a few hubs and many nodes that are only connected to one other node.

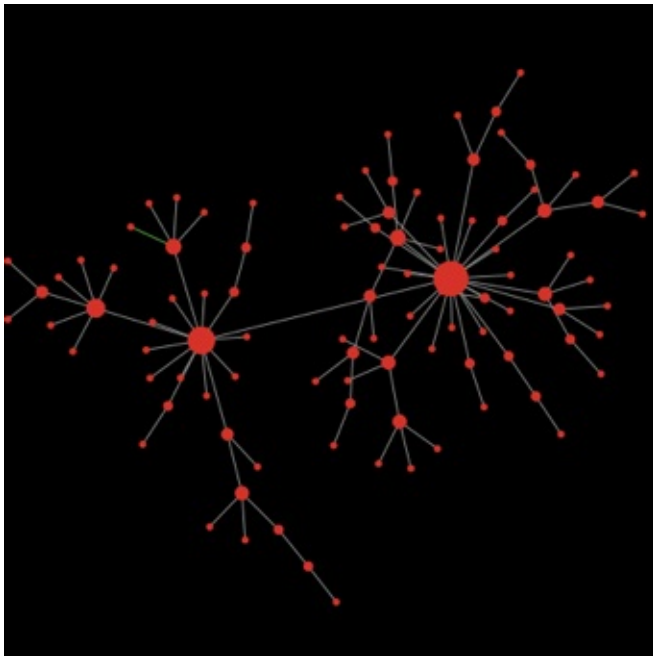


Figure 12. A typical scale free network.

Figure 13 shows the degree distribution of the network of 100 nodes generated in the example of Figure 12. The distribution approximates a scale-free distribution, meaning a straight line on a log-log scale of the number of nodes and the degree. The network has 66 nodes with degree one, and only two nodes with a degree higher than 8.

The clustering coefficient is 0, since friends of friends are not friends. The average path length is 4.14, which is higher than the average path length when the network was randomly generated (2.96).

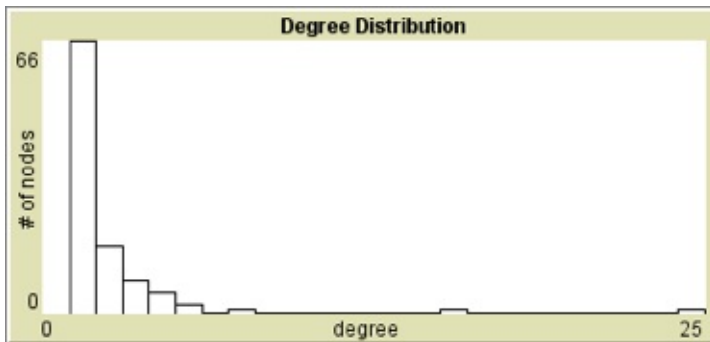


Figure 13. Degree distribution of the network in Figure 12 of 100 nodes.

Summary, Additional Reading, and NetLogo Primitives

Summary

In this chapter we introduced networks. We showed how to model networks in Netlogo and presented 3 stylized models of networks. In the next chapters we will see how network structure affects the results of various problems.

Additional Reading

Barabási, Albert-László (2004). *Linked: How Everything is Connected to Everything Else*. Plume, Cambridge, MA.

Barabási, Albert-László and Albert, Réka (1999). Emergence of scaling in random networks. *Science* 286: 509-512.

Newman, M., Barabási, A.-L., Watts, D.J. [eds.] (2006). *The Structure and Dynamics of Networks*. Princeton, N.J.: Princeton University Press.

Strogatz, Steven H. (2001). Exploring complex Networks. *Nature* 410: 268-276.

Watts, Duncan J. and Strogatz, Steven H. (1998). Collective dynamics of 'small-world' networks. *Nature* 393: 440–442.

NetLogo Primitives

[link](#)

[link-neighbors](#)

[link-neighbor?](#)

[end1](#)

[create-link-with](#)

Diffusion of Innovation

Key concepts

In this chapter we will:

- Study how innovation spreads in populations.
- Present models to study diffusion.
- Discuss the effect of network structure on the diffusion of innovations.
- Introduce primitives `other, turtles-here.`

Introduction

Facebook was introduced in February 2004 by [Mark Zuckerberg](#) from his dorm room at Harvard University. Eight years later more than 900 million people had a Facebook account. This is a dramatic success story of the diffusion of an innovation. Normally when somebody develops a new product it can take years to get it out on the market. For example, the [Sony Betamax](#) was considered to be a technologically superior product for recording TV programs, but it never became popular. Instead the [VHS](#) system from [JVC](#) became the dominant format. There are many flops and some successes in innovations. In this chapter we will use models to explore why this is the case.



Figure 1. Sony Betamax.

The study of diffusion focuses on the attributes of innovations that affect the speed and scope of the diffusion. The social network structure can affect the spread. When the average path length of agents within a network is small, the innovation may spread faster. However, the diffusion speed and scope also depends on the medium of diffusion. Is an innovation easy to acquire when one comes in contact with it? Can the idea be spread via mass media, or does one need to train a person to acquire the skill? In this chapter we discuss a number of examples of diffusion processes. We will look at the diffusion of innovations and the adoption of new products.

Diffusion of Innovations

The diffusion paradigm was developed by Ryan and Gross (1943), two rural sociologists who studied the diffusion of hybrid corn seed in two Iowa communities. The new hybrid corn, introduced in 1928, increased production by 20% and was more drought-resistant. However, farmers needed to purchase new seeds each year, instead of self-selecting seeds from their own plants. The new technology, therefore, led to important changes for the farmers. By surveying more than 300 farmers in 2 communities during 1941, Ryan and Gross showed that diffusion is a social process that spreads adoption in the community through subjective evaluation, rather than individual rational choice.



Figure 2. Hybrid corn.

Diffusion processes often replicate the observed stylized fact of a S-shaped curve of cumulated adopters of the innovation (Figure 3). In fact, the increasing number of adopters is, in essence, the diffusion process. The adoption of new products is a complex process that typically consists of a large body of agents interacting with each other over a long period of time. Traditional analytical models described diffusion processes on the market level, since available data is often aggregated without attention to spatial distribution or individual characteristics. When individual data is available, these are snap shots of adopters' surveys.

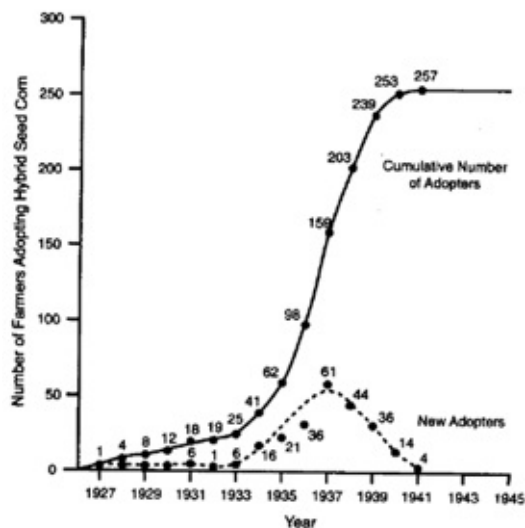


Figure 3. Diffusion of hybrid corn in two communities of Iowa (source: Ryan and Gross, 1943).

Diffusion processes have been studied in many disciplines such as anthropology, marketing, rural sociology and communications science. [Everett Rogers](#) (1962) synthesized the work of many studies and developed a framework of typical successful innovations. The initial adoption of an innovation is made by innovators, then early adopters, early majority, late majority and finally laggards may adopt the innovation too (Figure 4).

As we develop a model of the diffusion of innovation, we need to take into account that there is variability within the population when adopting something new (i.e., innovators versus laggards). Furthermore, there is likely to be interaction among agents that leads agents to imitate or copy the behavior of innovators. The position of the innovator within a social network may affect the diffusion pattern.

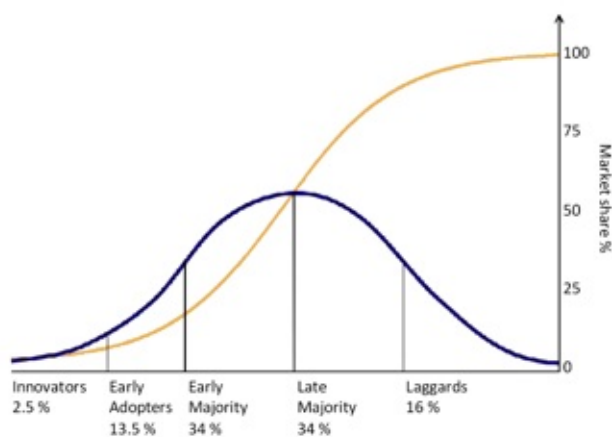


Figure 4. Diffusion of innovations. With successive groups of consumers adopting the new technology (shown in blue), its accumulative total (yellow).

Although empirical investigation of dynamic micro level processes is limited and difficult to accomplish, computational models have been developed which can be used to investigate the consequences of different assumptions of agents on the diffusion of an innovation.

The traditional model on the aggregate penetration of new products is the [Bass model](#) (1969) which follows Rogers' diffusion of innovation (Rogers, 1962). The Bass model emphasizes the role of communication, namely external influence via advertising and mass media, and internal influence via word of mouth. The decision of an individual is described as a probability of adopting a new product at time t , which is assumed to depend linearly on two forces. The first force is not related to previous adopters and represents the external influence, and the other force represents the internal influence and is related to the number of adopters.

$$dN(t)/dt = p[m - N(t)] + q/m N(t)[m - N(t)]$$

where $N(t)$ is the cumulated number of adopters at time t , m is the potential number of adopters, p reflects the influence that is independent of previous adoption, and q reflects the influence due to word of mouth. Although the Bass models fits very well with much of the data, its relevance to real consumer behavior is questioned. One of the problems is that the model only explains an observed successful diffusion process, and cannot predict whether a new product will be a success or a failure.

One way to simulate the diffusion process from an agent-based perspective, is to equip the agents with thresholds when to adopt an innovation. For example, [Mark Granovetter](#) (1978) presented a framework which assumed that thresholds to join a collective behavior differ among individuals, and that the type of distribution affects the macro-dynamics of the system. Granovetter (1978) focused on processes in which adoption could be monitored by the whole group. More relevant for many diffusion processes, is to base the threshold model on personal networks. Valente (1995) analyzes in detail threshold models for a number of data sets. Thresholds in his models are based on the individual personal network. An interesting finding is that less risky innovations spreads faster if the central connected agents have a low threshold for adoption. For more risky innovations, individuals who are less connected adopt the product first, which may reduce uncertainties and lead to diffusion to the whole group afterwards.

Simple Diffusion Model

We will now present a series of NetLogo models of diffusion processes. In the first model, we have agents walking randomly around. All agents are randomly placed in the environment. With a probability `probstep` an agent adjusts its direction and will move one step forward. All agents are white except for one who has the color red. Every time step an agent will check the frequency of agents with the red color in the patch the agent is staying. If the fraction of red agents is above the threshold `threshold` of the agent, a white agent will become a red agent. We will contrast the effect of each agent having a threshold equal to 0.5, and each agent having a threshold value randomly drawn between 0 and 1. To implement this in NetLogo we use the code below. `turtles-here` is the set of all turtles on the caller's patch. By adding `other` the set will not contain the caller. It checks first whether `sum0 + sum1` is larger than zero to avoid division by zero. If the fraction of red turtles is higher than the threshold, the color of the agent will be red and will stay the same, white or red, otherwise.

```
let sum0 0
let sum1 0
ask turtles
[
  set sum0 count other turtles-here with [ color = white ]
  set sum1 count other turtles-here with [ color = red ]
  if sum0 + sum1 > 0
  [
    if (sum1 / (sum0 + sum1) > threshold)
    [
      set color red
    ]
  ]
]
```

With an initial density of 1% of red agents we see a slowly growing number of red agents (Figure 5). When the threshold values vary among the agents the diffusion is faster compared to the same threshold value for all the agents. This is illustrated in Figure 6 where the heterogeneous case has a smaller number of ticks to come to a diffusion of red agents compared to a homogenous threshold of 0.5. When there is heterogeneity there are innovators who are more likely to adopt the innovation when just one red agent is on the patch.

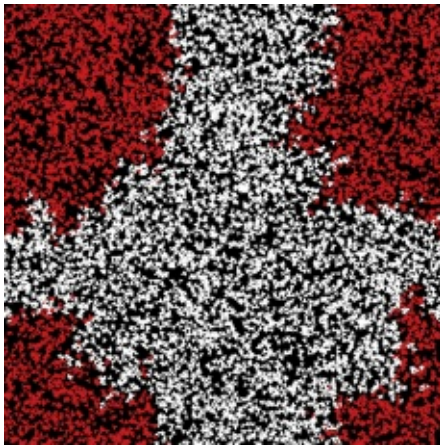


Figure 5. Spatial pattern of 10000 agents.

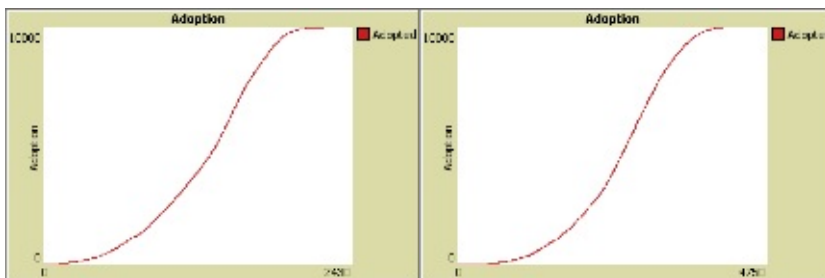


Figure 6. The top figure shows the number of adopted agents for heterogeneous threshold values. The bottom figure shows the number of adopted agents for a fixed threshold value of 0.5 for all the agents.

NETLOGO EXAMPLE: DIFFUSION

Model: Right click on the Link and Save it. Open Netlogo and Run it with NetLogo.

Diffusion in Social Network

Suppose agents are connected within a social network, like a small-world network. Starting with one red agent, white agents who have a red neighbor will adopt the innovation. This leads to the spread of the innovation through the network. As you can see in Figure 7, the diffusion takes short cuts of relinked nodes. The shorter the average path length in a network, the faster the innovation will spread.

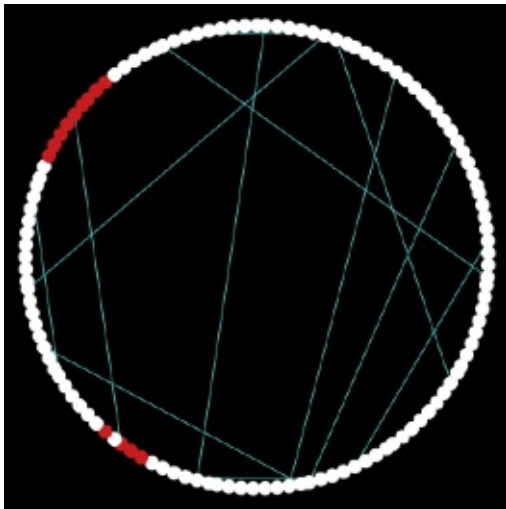


Figure 7. Snap shot of spread of innovation on a small world network.

To model the diffusion in networks we count for each node the number of friends that are red. All agents are updated synchronously. Therefore we first define the number of red friends for each node, and then check whether the threshold is passed. Check out what happens if the updating is not synchronous. The basic code of the model is now as follows.

```
ask turtles [  
  let node1 self  
  set nan count turtles with [link-neighbor? node1 and color = red]  
]  
ask turtles [  
  if nan >= threshold [ set color red]  
]
```

In Figure 8 some simulation runs are provided for a network of 500 nodes with different rewiring probabilities. There is one red agent at the start of the simulation. We see that when all agents are in a circle—no rewiring—the number of red agents spread linearly until all agents are adopted. When there is a small probability of a rewiring, the adoption is also linear until the short-cut is hit, which speeds up the adoption rate. We see that for larger rewiring rates the diffusion goes faster.

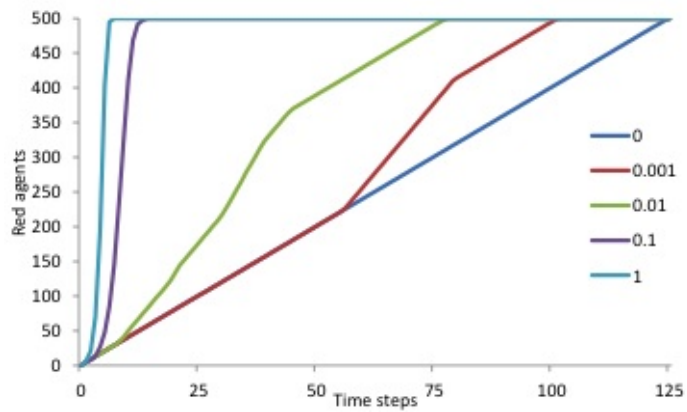


Figure 8. Diffusion in networks with different rewiring probabilities.

NETLOGO EXAMPLE: DIFFUSION ON NETWORKS

Model: Right click on the Link and Save it. Open Netlogo and Run it with NetLogo.

Lock-in of Products

Why do we sometimes have one product dominating a market, even if it is a product that some argue to be inferior? Examples are [Microsoft](#), [QWERTY keyboards](#), [Google](#) search engine, etc. This is the problem of [lock-in](#). Economist [Brian Arthur](#) (1994) studied the problem of lock-in of technologies and developed some models that simulates lock-in processes.

The basic idea is the following. Suppose there are two products to chose from, say Microsoft and Apple. The more people in your environment with whom you share files who use product A, the more valuable product A is. If a person has to make a choice between two products, he or she will be more likely to choose the product that has a majority of users, if the two products have similar performance.

We can implement this process in a simple model. Every time step agents made a choice. The choice for product A is proportional to the share of product A of both products as chosen in the previous time step. Starting the model with equal probabilities, one of the two products starts to dominate. Which product will dominate cannot be predicted from the start. Random events may give one of the products a larger market share and stimulating dominance in future time steps (Figure 9).

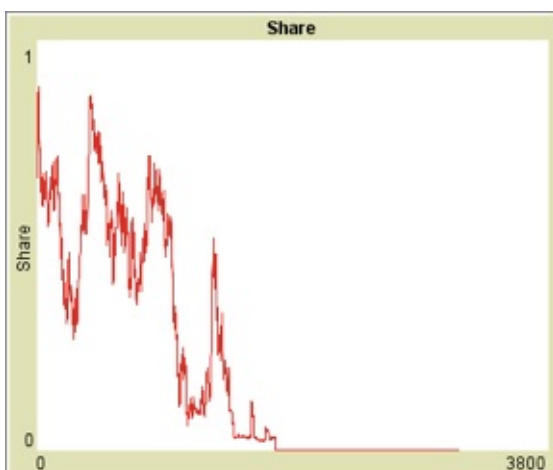


Figure 9. Share of red product among agents who bought a product.

In Figure 9 we show results of a model where every time step another agent will decide to enter to make a decision between product A and B. Every time step agents who have made a choice between A and B will reconsider it based on the current adoption shares. Suppose agents will not reconsider their decisions then we may get stable mixed shares of products A and B (Figure 10). Once a large number of agents have made their choice between A and B,

every new agent will choose relative to the frequency of products A and B. Hence, we need to include some reconsideration of agents in order to derive lock-in of one of the products with certainty.

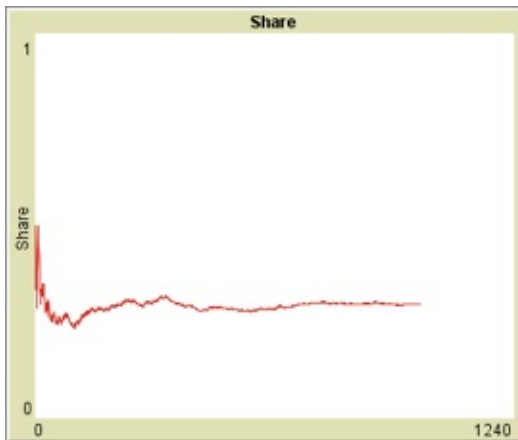


Figure 10. Share of red product among agents who bought a product when agents do not reconsider.

In Figure 11 you see the results of a large number of simulations. We run the model for 1000 time steps, and use different probabilities for agents to reconsider their choices. For each probability 1000 simulations were performed and the fraction of product A was determined at the end of the simulation. Then we ordered the simulations according to the share of product A. We see that for a reconsideration probability of 0, the share of product A follows a uniform distribution. However, for probability equal to 0.02, one third of the simulations have a fraction of zero, and one third has a fraction of 1. Hence the distribution is bipolar.

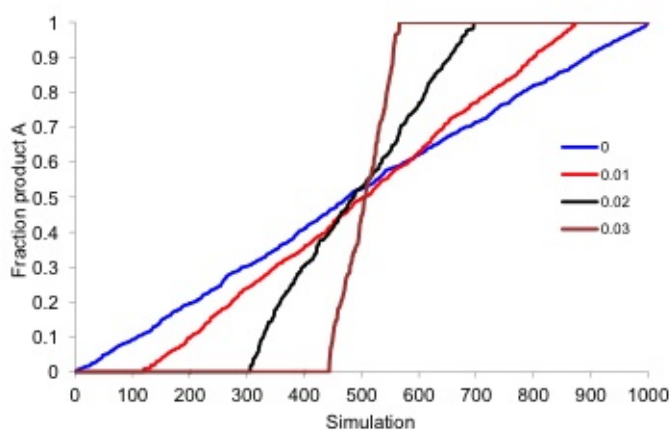


Figure 11. Fraction of shares of product A at the end of 1000 simulations for different probabilities of agents reconsidering their choice.

NETLOGO EXAMPLE: BE LOCKED IN

Model: Right click on the Link and Save it. Open Netlogo and Run it with NetLogo.

Summary, Literature, and NetLogo Primitives

Summary

In this chapter we discussed the diffusion of innovation and showed that different network structures lead to different diffusion speeds. We also saw that some innovations can be adopted by luck since more people with the innovation make it more attractive for others to adopt too.

Literature

Arthur, W. B. (1994). *Increasing returns and path dependence in the economy*. Ann Arbor: University of Michigan Press.

Axelrod, R. (1997). Dissemination of culture: A model of local convergence and global polarization. *Journal of Conflict Resolution* 41(2): 203-226.

Bass, F. M. (1969). A new product growth model for consumer durables. *Management Science* 15: 215-227.

Gladwell, M. (2000). *The tipping point*. New York: Little Brown.

Granovetter, M. (1978). Threshold models of collective behavior. *American Journal of Sociology* 83: 1420-1443.

Rogers, E.M. (1962). *The diffusion of innovations*. New York: Free Press.

Ryan, B. and N. C. Gross (1943). The diffusion of hybrid seed corn in two iowa communities. *Rural Sociology* 8: 15-24.

Valente, T. W. (1995). *Network models of the diffusion of innovations*. Cresskill, NJ: Hampton Press, Inc.

[Communications Theory/Diffusion of Innovations](#) Wikibooks (available under Creative Commons).

NetLogo Primitives

[turtles-here](#)

[other](#)

Epidemics

Key concepts

In this chapter we will:

- Introduce the SIR model to model epidemics.
- Define R_0 , which shows whether an epidemic will happen or not.
- Discuss the effect of network structure on the spread of infectious diseases.
- Introduce primitives `create-link-with`

Introduction

In the movie Contagion (Figure 1) a bird-flu–infected meal in Hong Kong causes a global pandemic killing millions of people. The lead actors try to trace down the source of the virus and calculate the variable R_0 , the basic reproduction number, to understand how fast the virus will spread.



Figure 1: Theatrical release poster for the movie Contagion.

Infectious diseases spread from human to human and can spread like wildfire. Depending on the disease, transmission can be caused by coughing, sexual intercourse, door knobs and toilet seats, and blood transfusion.

Infectious diseases are diffused through a population through the interactions of agents. In this chapter we will discuss agent-based models of infectious diseases. We will show how network structure affects the spread of the virus. Furthermore we can use the model to see the potential effects of interventions, like closing schools, to reduce the transmission of the virus.

The model we discuss in this chapter is based on a basic flu-type virus. Various other viruses contain more complex dynamics and stages of infectiousness.

SIR Model

A classic way to describe the diffusion of a disease in a population is to distinguish three components: susceptible, infected and recovered. The so-called SIR model ([link is external](#)) is typically studied using differential equations. There are three variables, S the number of susceptible, I the number of infected, and R the number of recovered. If we exclude population change and only look at the virus, we can model that the number of susceptible decrease over time. The rate of this decrease depends on the number of infected and susceptible people. The more susceptible people there are, the more people can become infected. The more infected people there are, the more they can infect susceptible people. β is the contact rate.

$$dS/dt = -\beta IS$$

The number of infected people is increased by the number that get infected minus the number of people who recover. People recover at a fixed rate and the recovery rate is given by ν .

$$dI/dt = \beta IS - \nu I$$

$$dR/dt = \nu I$$

Now we can define R_0 , which is called the [basic reproduction number](#). The ratio is derived by the expected number of new infections from a single infection in a population where all subjects are susceptible. If R_0 is above one, more people get infected with each infection and thus the virus spreads. To combat the epidemic R_0 needs to become lower than 1.

$$R_0 = \beta / \nu$$

SIR Model on a Network

The differential equation based model as described above assumes that there are many agents who are randomly interacting. We now discuss an agent-based model where we distinguish susceptible, infected and recovered agents.

Given are agents connected in a network. The network is generated as follows. First all the agents are randomly put on the landscape.

```
crt number-of-nodes
[
  setxy random-xcor random-ycor
]
```

Then we define the number of links that will be generated such that the network has a specific average degree `average-node-degree`.

```
let num-links (average-node-degree * number-of-nodes) / 2
```

Links are added one by one until the number of links is reached. Every time step one of the turtles is selected randomly and the nearest turtle to it is selected and a link is made between them.

```
while [ count links < num-links ]
[
  ask one-of turtles
  [
    let choice (min-one-of (other turtles with [not link-neighbor? myself])[distance myself])
    if choice != nobody [ create-link-with choice ]
  ]
]
```

Agents who are connected may affect each other's state. Infected agents may infect susceptible agents. Each susceptible neighbor of an infected agent has the probability `virus-spread-chance` to get infected. Each infected agent has a probability `recovery-chance` of recovering. If an agent recovers, it can become immune with probability `gain-resistance-chance` or become susceptible again. This is implemented in Netlogo in the following way.

```
ask turtles with [infected?]  
[  
  ask link-neighbors with [not resistant?]  
  [  
    if random-float 100 < virus-spread-chance  
    [  
      set infected? true  
      set resistant? false  
      set color red  
    ]  
  ]  
]  
  
ask turtles with [infected?]  
[  
  if random 100 < recovery-chance  
  [  
    ifelse random 100 < gain-resistance-chance  
    [  
      set infected? false  
      set resistant? true  
      set color gray  
    ]  
    [  
      set infected? false  
      set resistant? false  
      set color green  
    ]  
  ]  
]
```

We see that the virus spreads through the network (Figure 2). Initially three agents are infected and within a few time steps we see the red color start spreading.

If the probability of becoming resistant is high, the disease stops spreading early compared to lower level of `recovery-chance`. Figure 3 shows that with a low value of `recovery-chance` the virus spreads to about 40% of the population since agents get sick again after recovery (becoming susceptible), while it is contained to below 10% with `recovery-chance` equal to 50%.

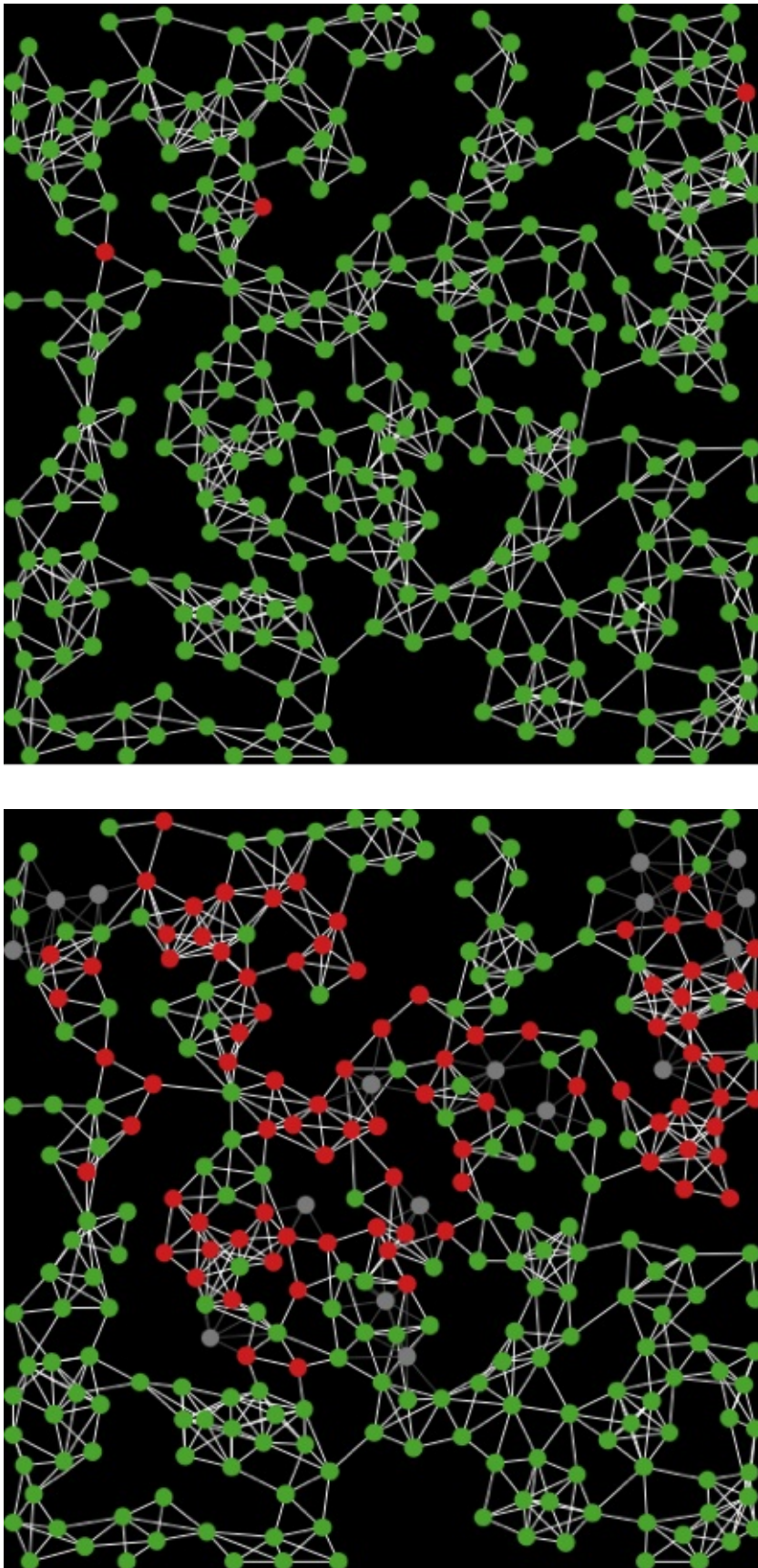


Figure 2: A network of agents where green agents are susceptible, red agents are infected and grey agents are recovered agents. Left is the initial state of the network. Right is the network after 214 time steps. The probabilities used are `virus-spread-chance` = 0.025, `recovery-chance` = 0.05 and `gain-resistance-chance` = 0.05.

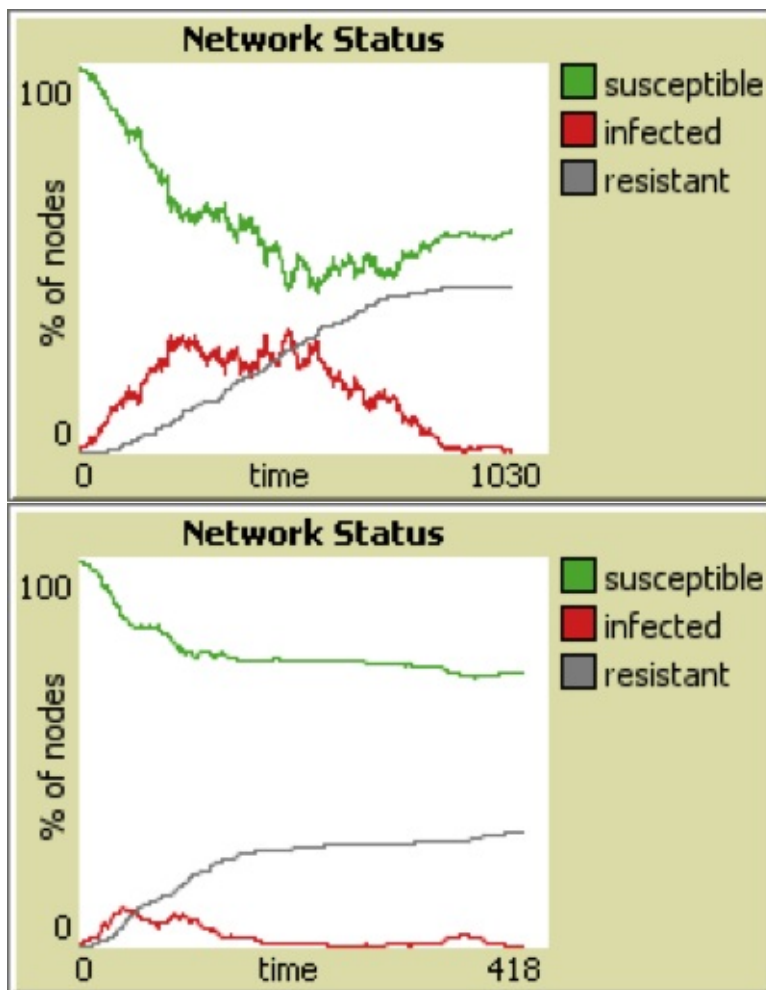


Figure 3: Spread of the virus in the population with `recovery-chance` = 0.05 (top) and `recovery-chance` = 0.5 (bottom).

We will now explore the consequences of the parameters `virus-spread-chance` and `recovery-chance` and assume that agents who recover do not become susceptible again (`gain-resistance-chance` = 1). We will see the consequences for average degrees 4, 6 and 9 for a population of 500 agents. Each parameter combination is run 100 times. Figure 4 shows that a higher degree leads to more agents getting the virus. Furthermore, we see that a higher probability of spreading the virus leads to more agents getting sick, and a lower recovery rate also leads to more agents getting sick.

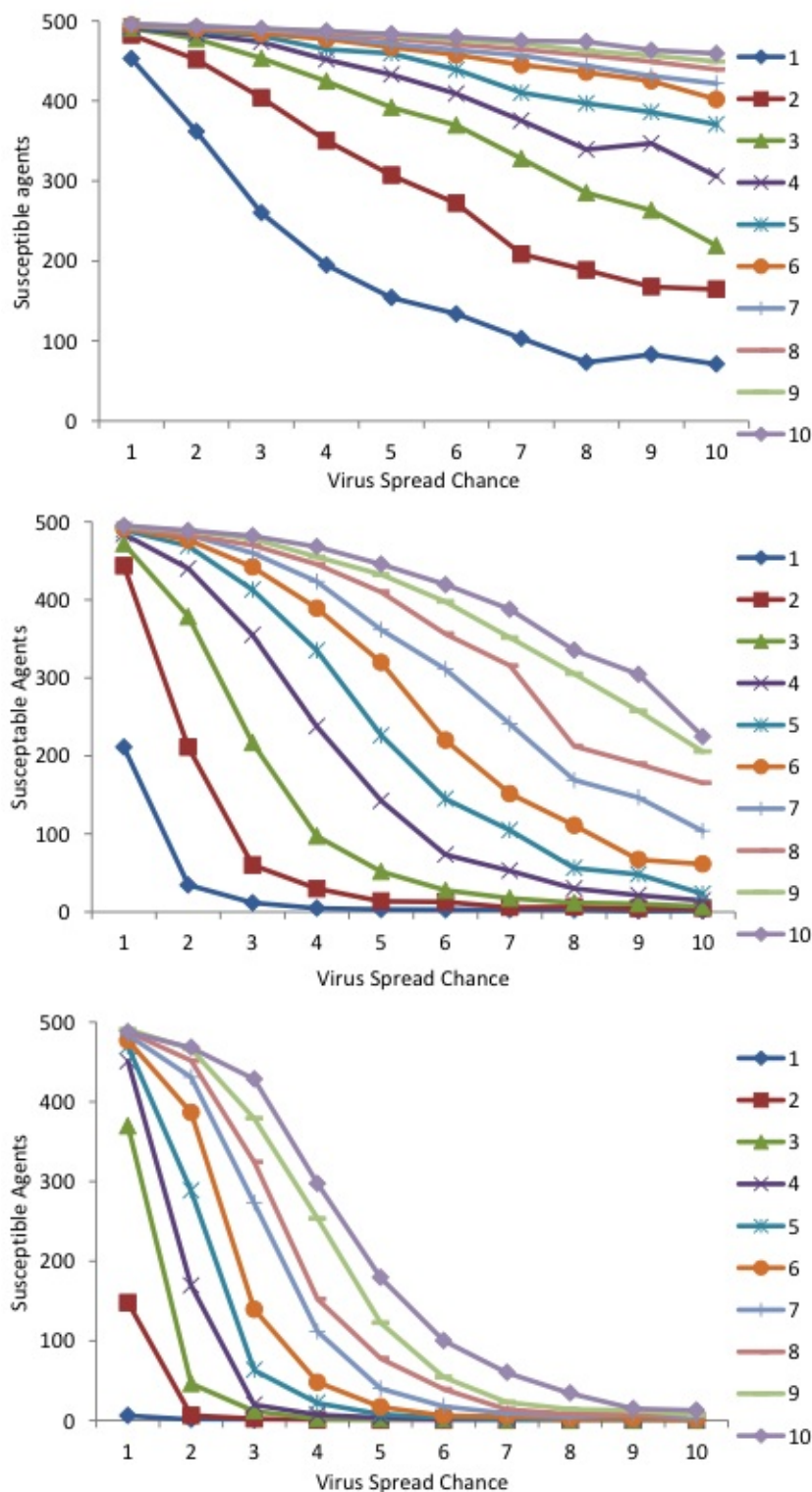


Figure 4: The average number of susceptible agents out of a population of 500 after 100 runs of 1000 time steps for different parameter settings. The upper figure is for an average degree of 4, the middle for an average degree of 6 and the lower figure for an average degree of 9. The horizontal axis shows the probability the virus is spread (from 1% to 10%) and the individual lines are the recovery chance (from 1% to 10%).

NETLOGO EXAMPLE: CONTAGION ON A NETWORK IN

Model: Right click on the Link and Save it. Open Netlogo and Run it with NetLogo.

Policy Intervention

In order to stop the spread of infectious diseases we have a number of options. We can vaccinate people if a vaccine is available. Another option is to reduce the interactions between agents. In practice, schools are often closed during an outbreak of a virus since children are an effective host in the spread of diseases like the flu. With sexually transmitted diseases like AIDS, targeting prostitutes is a common approach since they are the hubs in interactions between agents. As an exercise we implement the option of deactivating an agent by clicking them with your mouse. Run the model and see whether you can isolate the virus (Figure 5). As you can see this is not an easy approach.

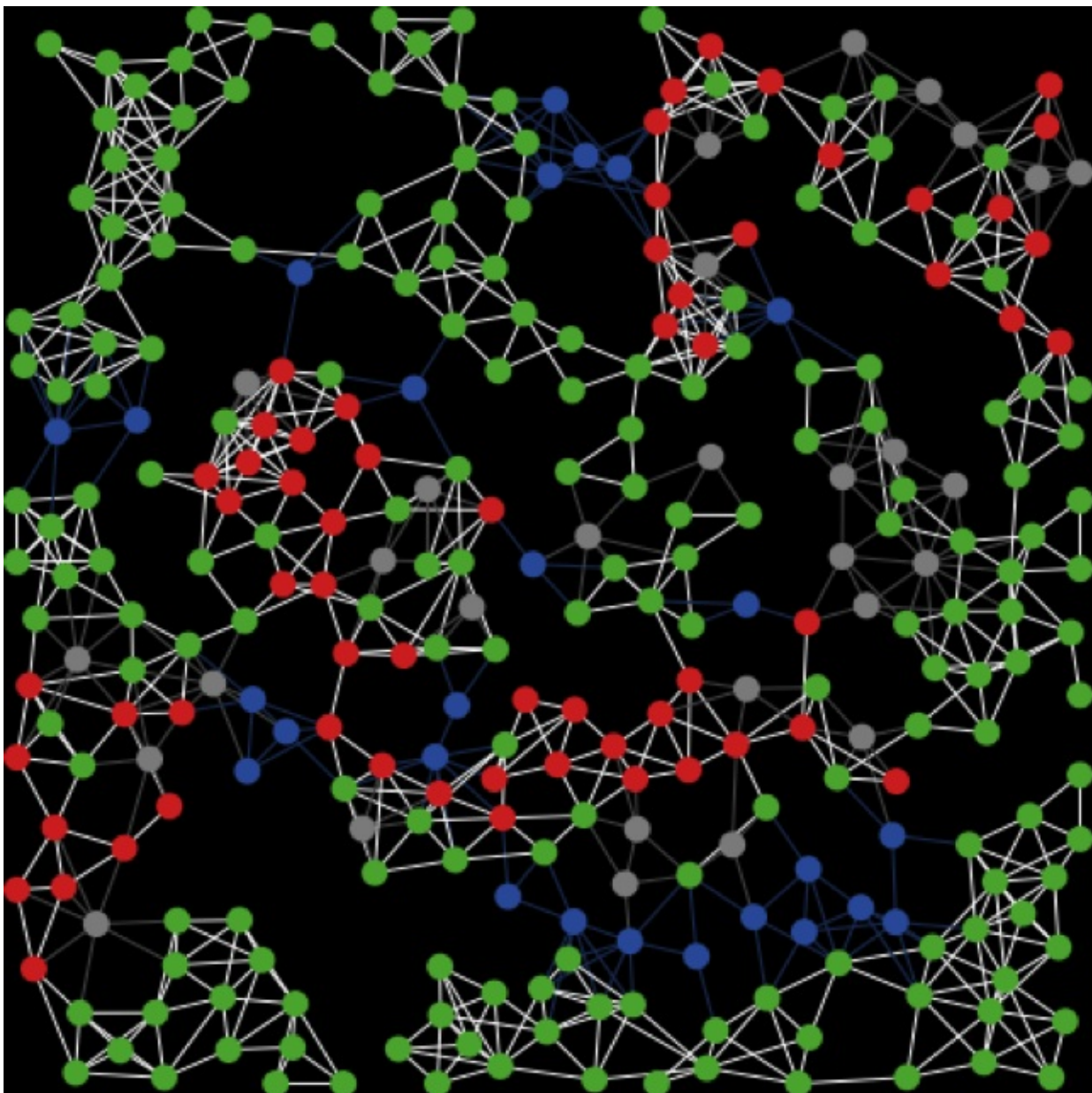


Figure 5: Screenshot of network where user tried to isolate virus (red nodes) by deactivating nodes (blue).

Another approach is to deactivate the links with the highest degrees. To model this in NetLogo we create a while loop and every iteration we find one of the turtles that has the most neighbors and has not been isolated yet. We then isolate that agent. When an agent is isolated we give them the color blue.

```
let i 0
while [i < numberofisolatedagents]
[
  let target max-one-of turtles with [color != blue] [count link-neighbors]
  ask target [
    set infected? false
    set resistant? true
    set color blue
  ]
  set i i + 1
]
```

We simulate the model 1000 times for different numbers of agents to be isolated and we see that more agents are not getting sick. What is the benefit of isolating a person? In Figure 6 we show that for each isolated person on average one or two other persons are spared. However, after isolating 80 agents the marginal benefit start to decline.

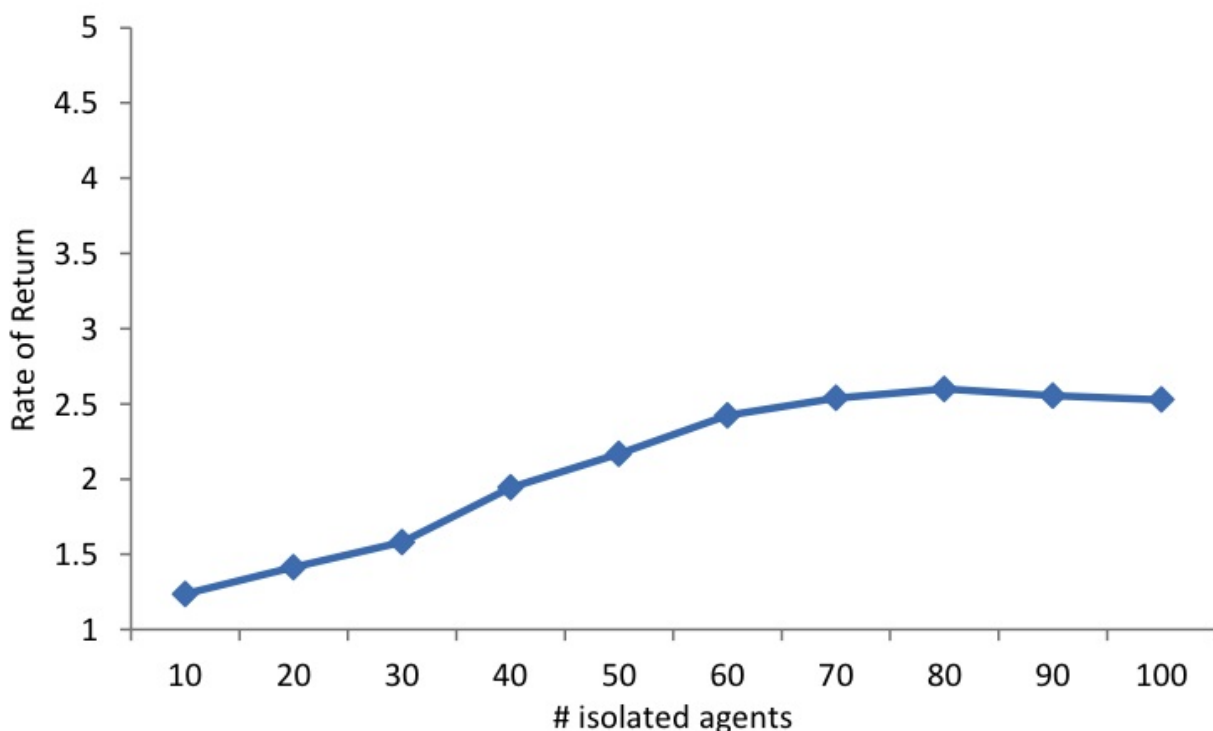


Figure 6: The rate of return (reduced number of infections – isolated agents, divided by the number of isolated agents).

Looking at an example of what the strategy is doing (Figure 7) we see that isolating the agents with the highest degrees is not necessary since it is, in effect, isolating agents far away from the infection. Furthermore, it does not isolate the spread of the disease.

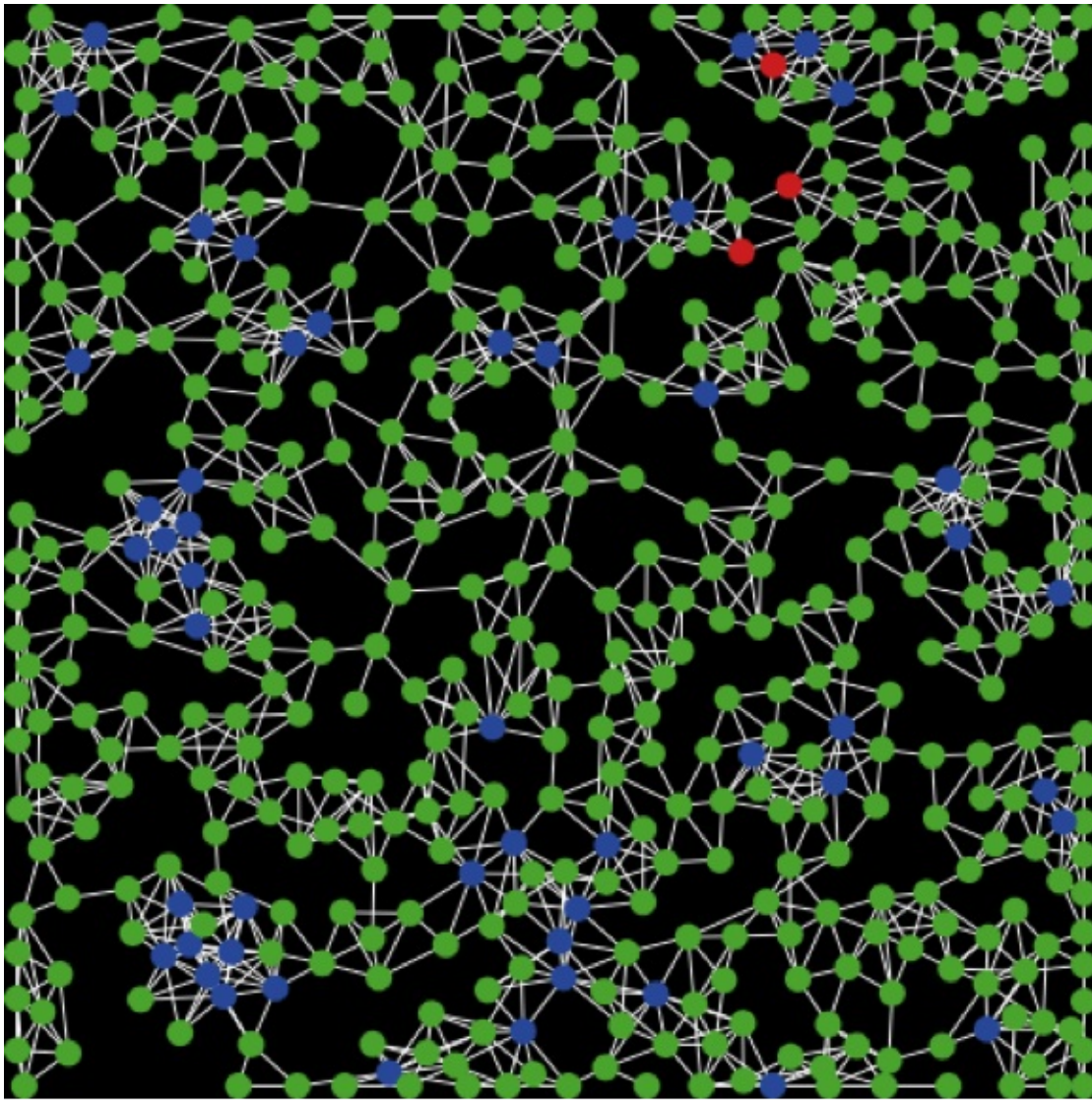


Figure 7. Screenshot of network with 50 isolated agents.

Of course it would be the most effective strategy to isolate the nodes next to the infected agents, but that would not be a fair strategy, since the computer is as fast as the speed of light unlike actual policies.

Suppose that we can isolate one agent each tick, which agents would we want to isolate. We can choose one agent with the highest degree each tick within a radius X of turtles who are infected. A smaller radius would indicate that the controller who is isolating agents has better information on where infected agents are. Not surprisingly, we see that a short distance leads to better performance (Figure 8). But not too close, however, since there might not be a sufficient number of highly connected nodes in a small radius that could stop the spread. Figure 8 also shows that more than 50 isolated agents starts giving less effect per isolated agent.

Although agents are not isolated at the start of the simulation, the performance is much better than isolating agents with a high degree at the start of the simulation.

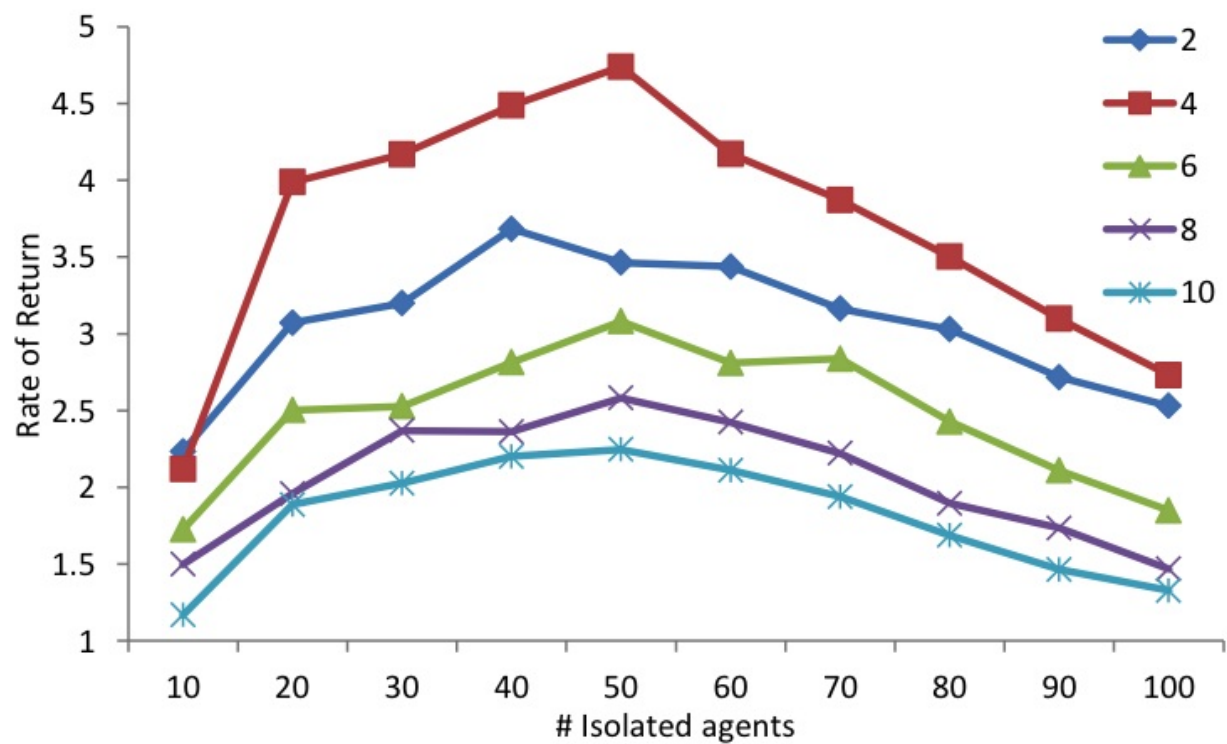


Figure 8. Average rate of return in isolating agents when different radius distances and a different number of isolated agents are used.

Summary and NetLogo Primitives

Summary

In this chapter we discussed the SIR model to simulate the spread of infectious diseases. We see that the spread of the agents is affected by the average degree of agents and probability of infection and recovery. We have seen that isolating highly connected nodes dramatically reduces the spread of the disease. Hence it is advisable to close schools and other places where people interact with many others in order to stop the spread of the disease.

NetLogo Primitives

[max-one-of](#)
[distance](#)
[create-link-with](#)

Fads and Fashion

Key concepts In this chapter we will:

- Introduce a model of consumer behavior
- Define agents who are satisfied and imitate friends when uncertain
- Discuss the consequences of different assumptions of decision making on market dynamics.

Introduction

What is in style? Many people follow trends in the clothes they wear, their haircuts, the kind of TV shows that they watch, the kind of toys children play with, etc. And what is in style changes over time. What is clear is that people like to imitate the behavior of others. Of course, there are also people who do not want to join the majority, and they will create a counter culture.



Figure 1: Fashion in the 1970s.

What affects fads and fashion in our behaviors? Why are some items, like clothing, more fashionable than others, like detergent? Why do fashionable items disappear over time?

These are the questions we will address in this chapter, which is mainly focused on how we, as consumers, make decisions.

Consumer Behavior

In the previous chapter the agents adopted products because a number of other agents used that product. The agents did not evaluate the product itself. In a more realistic model of product choice agents may make decisions based on different attributes of the products affecting different needs of the agents. In this chapter we will include more psychology in modeling the decision making of the agents.

We assume that agents try to satisfy their needs. One activity, such as eating a meal with your family, can satisfy needs of subsistence as well as needs of belongingness. When people make decisions they take into account the different types of needs that are involved. In this chapter we simplify the model to include an individual need and a social need. The individual need refers to the functionality of the product, it is a product that the agent likes. The social need refers to the fact that people like to belong to a group and like to conform to the choices made by their friends.

A Model of Consumer Behavior

We will present a simple model in which agents make decisions on which product to buy from a set of N possible products. Agents are connected to each other in a social-network. Agents buy products according to the decision rules and evaluate their choices relative to what happens in the neighborhood of their social network.

The utility of using a product consists of an individual part and a social effect part. The individual part expresses the degree of fit between the product characteristics and the preferences of the consumer. In the model this individual utility is expressed as the difference between the single personal preference of a consumer and the single product characteristic. The personal preference of actor i , p_i , is expressed by a value between 0 and 1. The utility of the product, based on personal preferences alone, is equal to one minus the absolute difference between personal preference p_i and product dimension δ_j . Hence the smaller the difference between the individual preference and the product characteristic, the closer the individual utility gets to its maximum value of 1.

The social effect means that the utility of a product increases when more friends consume the same product. This effect only affects the social need satisfaction. Hence, this effect qualitatively differs from the positive network effect often discussed in markets such as software, fax machines and the like, because the individual utility is not affected by the number of friends consuming the same product. This social effect relates to the need to conform to others in the agent's network. The variable x_j denotes therefore average distance of choices made by the agents and their neighbors. For each agent we calculate the ratio of products as a fraction of all products consumed among the neighbors of the agent. The total expected utility of consuming product j is equal to:

$$E[U_{ij}] = \beta_i \cdot (1 - |\delta_j - p_i|) + (1 - \beta_i) \cdot x_j \quad (1)$$

The components of the utility function, the individual part and the social part, are weighted with β_i and $1 - \beta_i$, with $\beta_i \in [0, 1]$. A low β_i holds that personal need is weighted less, as is usually the case with less innovative people (Rogers, 1995), whereas a high β_i holds that social needs are weighted less, as is usually the case with more innovative people. The dimension δ_j on which the agents make decisions can represent various product characteristics.

The agents are located in a network. Like the previous chapter we first place agents randomly on the landscape, but instead of connecting nodes to nearest other nodes, we link agents to both nearby and faraway agents. The probability of any two agents being connected to each other is calculated so that the probability of a connection between any two agents decreases as the spatial distance between them increases. The probability that a short-length connection will be made rather than a long-distance connection is determined by a parameter, D , which represents the desired link length in the network.

Given two agents separated by distance d , the probability of a link connecting them is:

$$p = \frac{n}{2\pi D^2} e^{-Nd^2 / 2D^2} \quad (2)$$

where n is the average number of links in the network, d is the average network density, D is the desired length-scale for generating networks, and N the number of agents. When D is small, i.e., $D = 2$, agents will be preferentially connected to agents within their immediate spatial vicinity. When D is larger, i.e., $D = 10$, agents are more connected with individuals at greater distances from themselves, that is, they are more “globally” connected (Figures 2 and 3).

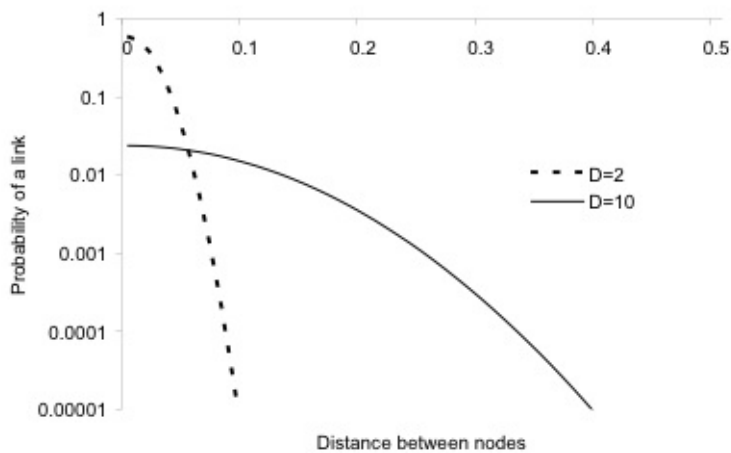


Figure 2: The effect of D on the network structure. With $D=2$ the connections are more local compared to $D=10$.

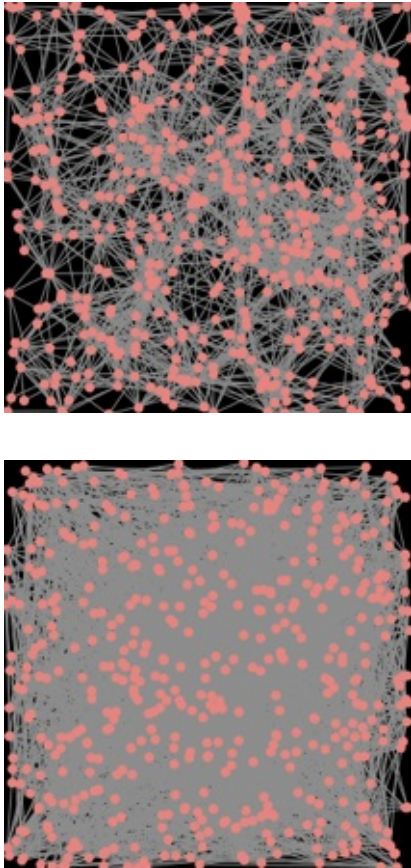


Figure 3. Network of 500 agents using $D=2$ (top) and $D=10$ (bottom).

Agents make a decision to buy a product with probability $p_b (=0.10)$. When an agent makes a decision it will continue the choice of past rounds if the experienced utility is higher than the minimum utility that satisfies the agent u_{min} . If the agent is not satisfied, the agent will calculate the expected utility for all N products and chooses the one which will maximize her utility. Since agents will use less cognitive energy if they repeat decisions from the past, it is common to repeat past decisions if you have been happy with the outcome. Only when it is needed will you evaluate all the options.

We now turn to some exercises. Let's assume all agents have the same values for β and u_{min} . We will vary β and u_{min} and run the model 100 times for each parameter combination. All agents are initialized randomly with one of the available products.

What do we measure from the simulations? We would like to know whether the products are distributed evenly among the agents, which would be the case if they only care about their own individual preferences. Therefore we calculate the [gini coefficient](#) of the products bought by the agents at the end of the simulation. Figure 4 shows that there is a low gini coefficient, equal spread of the products among agents, if agents weight the individual preferences high and when the minimum utility level is low. If β is 0.5 or higher the gini coefficient will rise to above 0.5 especially if the minimum utility is high. In such situations one or two products dominate the market. Agents value conformity and therefore buy similar products as their friends.

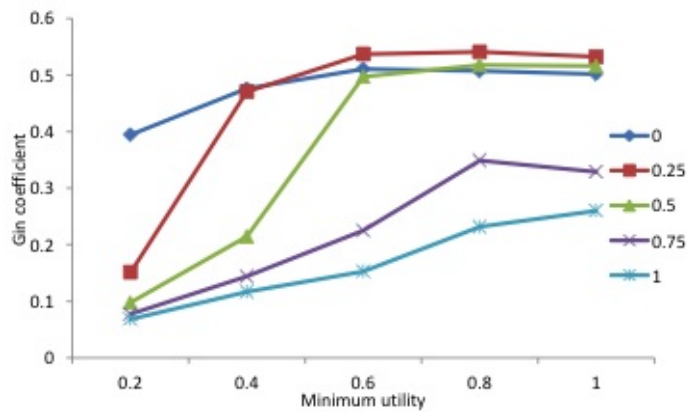


Figure 4. Gini coefficient of the distribution of products in the population for different parameter combinations. Results are averages for 100 runs of 100 ticks.

The other variable we measure is the average utility at the end of the simulation. In general higher utility is derived if the minimum utility is higher (Figure 5). The utility is also higher if they focus more on conformity. The reason for this is the relative small number of choices, 10 products, and the uniform spread of the preferences. Products cannot fully satisfy individual preferences, but if people weigh social preferences more they can get a high utility by buying the same products as their friends do.

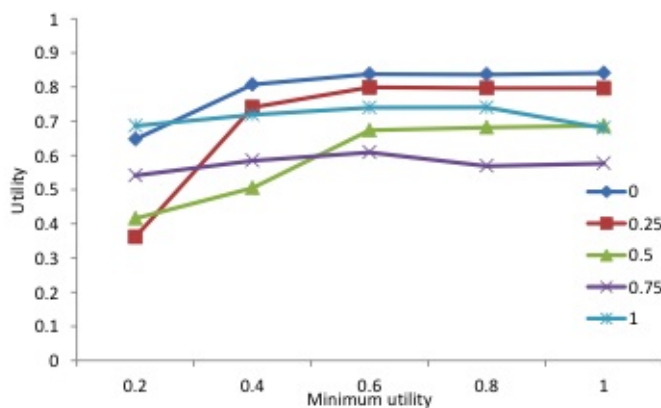


Figure 5. Average utility for different parameter combinations.

In the previous simulations the β value of all agents was equal to 0.5. What would be the consequence of agents having different values of β ? It would mean that some agents are innovators, focusing on products meeting their preferences, while others are behind in adopting new products but care to conform with the majority in their circle of friends. We see that a heterogeneous distribution of β leads to a lower gini coefficient and lower utility if the threshold of the minimum level of utility is high. As we see in Figure 4 a high β leads to lower gini coefficient and variation of β follows this trend.

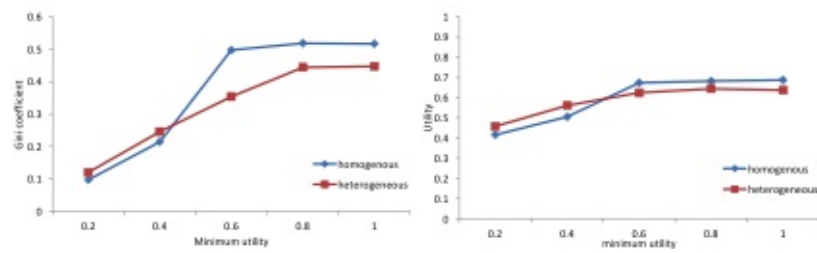


Figure 6. The gini coefficient (top) and average utility (bottom) for different parameter values and where β was 0.5 or on average 0.5 but with a uniform distribution between 0 and 1.

NETLOGO EXAMPLE: CONSUMERS IN

Model: Right click on the Link and Save it. Open Netlogo and Run it with NetLogo.

Including Social Influence

In the model we described above agents were repeating their decisions, or they were deliberating on finding the solution that maximizes the expected utility. In practice people often benefit from the experiences of other people in making a decision. Using information on other people's behavior or experiences may function as a cue to simplify the decision task. Especially when people are uncertain (due to the complexity of the decision, uncertain personality) and the behavior in question is very visible (either by observation or discussion) social heuristics are used.

People are assumed to imitate if they are uncertain. When agents imitate they follow the majority of the people in the community. This does not necessarily lead to a product that satisfies the agent. For example, an agent may imitate the product choice of their friends even though they have different preferences. Another approach is social comparison, where agents look at the products used by agents like them and the agent will evaluate those options.

The expected uncertainty $E [Unc_{ij}]$ reflects how uncertain an agent i is about having made the best choice when choosing product j . The more friends in their social network consume other products, the more uncertain an agent is. Furthermore, the value of β_i determines how sensitive an agent is to not having the same choice as his or her friends. The more important the social need (the lower the value of β_i), the more uncertain an agent becomes when his or her friends consume different products:

$$E [Unc_{ij}] = (1 - \beta)(1 - x_j) \quad (3)$$

with $(1 - x_j)$ represents the fraction of the friends of agent i who consume a different product than agent i . It is essential that agents start engaging in social processing when their uncertainty exceeds a critical level, the maximum uncertainty level.

Agents make a choice between what they really like and whether they select the same product as their neighbors. We specify the different heuristics they use.

Deliberate

Agent evaluate all the options available and will calculate what the expected utility will be if one consumes the product. Based on the expectation she will choose the product with the highest expected utility.

Repeat

Agent will repeat her choice from the previous time.

Imitate

Agent will copy the most popular product in her neighborhood.

Socially compare

Agent will evaluate the most popular product in her neighborhood and will chose this option if it increases her expected utility compared to sticking with her current choice.

Figure 7 shows the chosen products in the social network, and illustrates the clustering of choices in parts of the network. Although initially agents randomly chose a product, within a few time steps we see clusters emerging. Figure 8 shows that in the beginning of the simulation there is a lot of social comparison and imitation, and when the agents start to settle on their choices they continue making repetitive choices.

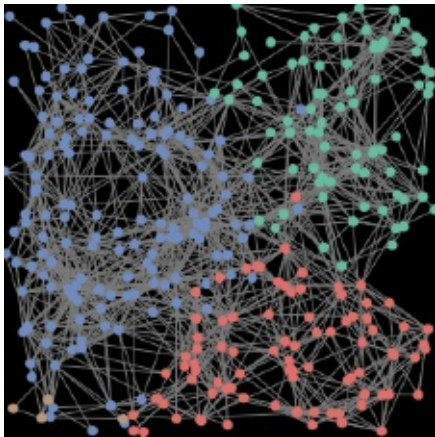


Figure 7: Network of agents where each agent is colored according to the product chosen.

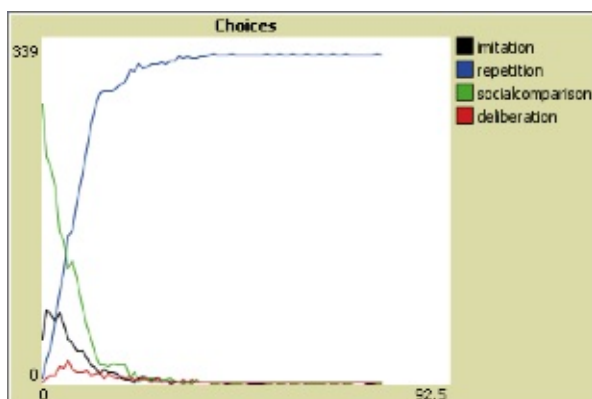


Figure 8. The number of agents who make choices according to a particular heuristic.

Next we will explore the effect of different levels of the maximum uncertainty level when using a heterogeneous β and a minimum utility of 0.5. We ran the model 100 times 100 ticks at different levels of maximum uncertainty toleration. Figure 9 shows that when no uncertainty is tolerated agents only imitate and socially compare. When the uncertainty tolerance is increased we see repetition and deliberation being included. The gini coefficient drops, and was highest when agents only imitated or socially compared. With a higher maximum allowable uncertainty the average utility is increased a bit since imitation does not necessarily lead to solutions that have high utility.

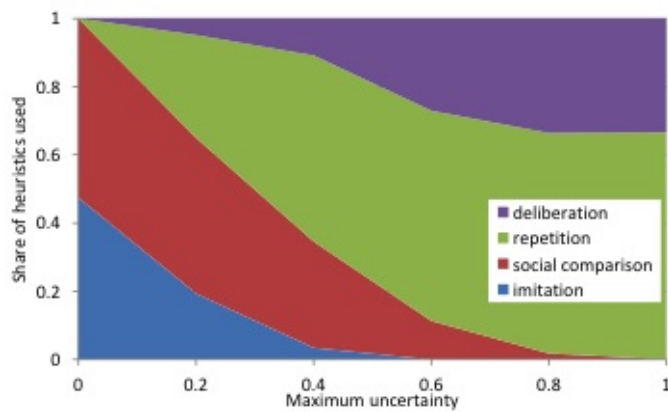


Figure 9. Distribution of heuristics for different levels of maximum uncertainty tolerated.

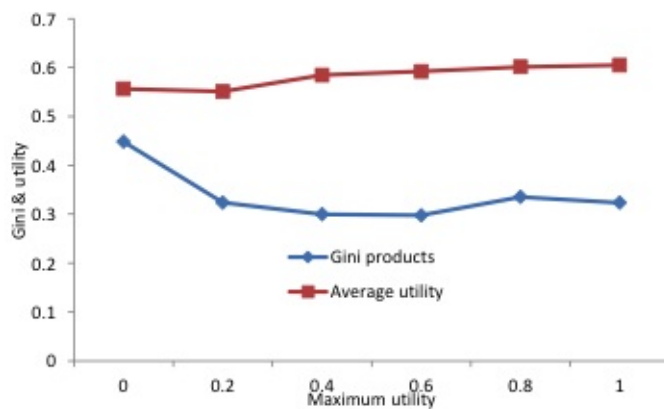


Figure 10. Gini coefficient of products and average utility for different levels of maximum uncertainty tolerated.

We will now explore the consequences if agents do not know all the available products at the start of the simulation. Suppose agents have a memory of the products on the market. In previous simulations we assumed that the agents had full knowledge about the opportunities. Suppose that initially they have only a memory of products they and their neighbors are consuming. Agents build up their memory and thus their repertoire of possible products if they see one of their neighbors consuming a product.

Figure 11 shows that there is more equality if agents do not have full knowledge. Since not all options are known, and agents initially are randomly allocated a product, there remains a spread of many products in the population. When agents know the products available, we see that one or two products start to get a larger market share, especially when they use imitation.

The average utility is generally higher when agents have full knowledge. This is not surprising since they have more options available to increase their utility. However, when agents only care about what others do, and they imitate decisions of others, we see that there will be a higher level of satisfaction.

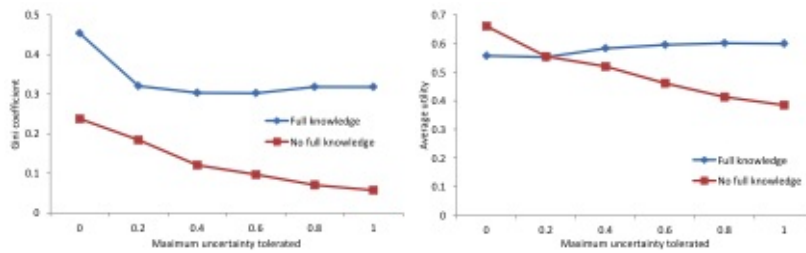


Figure 11. Gini coefficient (top) and average utility (bottom) for simulations with full and incomplete knowledge for different levels of maximum tolerated uncertainty.

Figure 12 shows that when agents have incomplete initial knowledge, they will use more deliberation and less repetition. Their utility is more frequent below the threshold of satisfaction leading agents to deliberate.

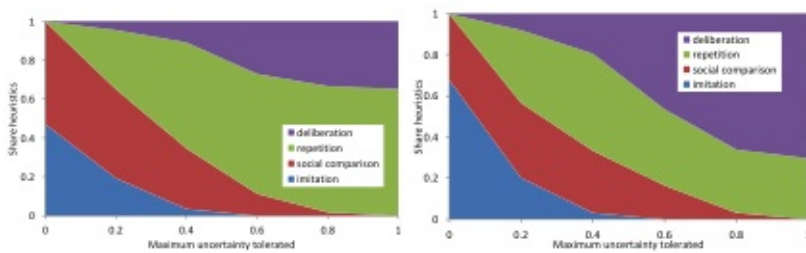


Figure 12. Share of different heuristics with full (top) and incomplete knowledge (bottom) for different levels of maximum tolerated uncertainty.

Summary, Reference and Recommended Readings

Summary

In this chapter we look at consumer behavior in buying products on the market. Agents aim to buy products to satisfy different needs. In making decisions they use less cognitive energy when satisfied and focus their attention to choices of others when uncertain. The agents use a different heuristic of decision making depending on the agent's context at that moment.

References

Janssen, M.A. and W. Jager (2003). Simulating market dynamics: Interactions between consumer psychology and social networks. *Artificial Life* 9: 343-356.

Recommended readings

Gladwell, M. (2005). *Blink: The power of thinking without thinking*. New York: Little and Brown.

Levitt, S.D., and S.J. Dubner (2005) *Freakonomics: A rogue economist explores the hidden side of everything*. New York: William Morrow.

Collective Action

Key concepts

In this chapter we will:

- Discuss collective action problems
- Explore the consequences of conditional cooperation on collective action
- Use models to evaluate the effect of different interaction patterns on collective action

Introduction

Our global society is facing major challenges due to human activities. Human-induced climate change; acidification of land and oceans; loss of biodiversity ([link is external](#)); depletion of ground water, fish stocks, or forests, etc., are all major problems that require collective action in order to limit the magnitude of their effects. Besides the destruction of our environment we are also facing major global health challenges. Due to increased transportation of goods and humans, diseases can spread rapidly around the world making us vulnerable to new infectious diseases. Furthermore, almost 2 billion people lack access to proper sanitation, leading to the deaths of millions of people each year due to [diarrhea](#). Collective action is needed to address these global health problems as well.



Figure 1.

So the solution to many of the major problems of society requires collective action. But will you give up your holiday to reduce the impact on the environment in the future? Would you be willing to clean toilets to improve sanitation quality? Most people would rather free ride on the efforts of others. This is the [social dilemma](#).

Coordinated action at the global level might be needed. But global negotiations such as the [Rio + 20 conference](#) in 2012 did not lead to the kinds of concrete shifts in behavioral patterns that are needed for a more sustainable future.



Figure 2.

What do we know about collective action? In Chapter 3.4 we discussed the work of Elinor Ostrom who found that in the right conditions communities are able to govern their own common resources. Such conditions include the ability to influence the rules community members have to comply with, graduated sanctioning, and inexpensive conflict resolution mechanisms.

From experimental studies we have gained some additional insights. Most people are willing to cooperate but this willingness is conditional based on the actions of others. Most people will cooperate if others do too. Psychologists have done experiments that show that information about the actions of others can increase cooperation. For example, take the case of towels in hotel rooms. You may have recently seen little cards in hotel bathrooms encouraging patrons to reuse their towels to save energy and water and to help the environment. Psychologist [Robert Cialdini](#) and his colleagues did experiments where they changed the message in half of the hotel rooms to say that 70% of the people in the hotel recycle their towels. This was truthful information, but it was in the rooms with the message that the higher levels of recycling occurred.

In this chapter we discuss some models to study collective action and how assumptions of conditional cooperation and the structure of interaction among agents affect the ability of groups to come to collective action.

A Threshold Model of Collective Action

The first model is based on the work of sociologist [Mark Granovetter](#). He describes a situation in which 100 agents have different thresholds that determine whether they are willing to join a riot. Some agents will join the riot if a handful of others participate, while some will only join if most others participate. Suppose the distribution of the thresholds is such that there is one agent with threshold 1, one agent with threshold 2, etc., up to one agent with threshold 99. If one agent who has a threshold 0 initiates the process, we will see that all 100 agents will end up participating in the riot. The reason is as follows. When one agent starts making trouble, the agent with threshold 1 will join next which will lead to the agent with threshold 2 to join the riot.. The process will continue until the thresholds of all 100 agents are met.

Suppose the distribution of the thresholds is a bit different. Instead of the distribution {0 1 2 3 ... 98 99} the distribution is {0 1 2 4 4 5 ... 98 99}. What will happen? Indeed no more than 2 agents will participate in the riot.

A NetLogo model is developed to explore the consequences of different distributions. We will look at the distribution of riot sizes if the distribution is uniform between 0 and 99, and if the distribution is a truncated normal distribution with mean 50 and standard deviation 25.

Figure 3 shows the distribution of riot sizes for 1000 simulations. Although the average threshold is the same for both distributions, the resulting outcomes are very different. The largest riot size for the [Gaussian distribution](#) is 11, while there are dozens of all 100 agents with the uniform distribution. It is important to have sufficient agents who are easily triggered to participate.

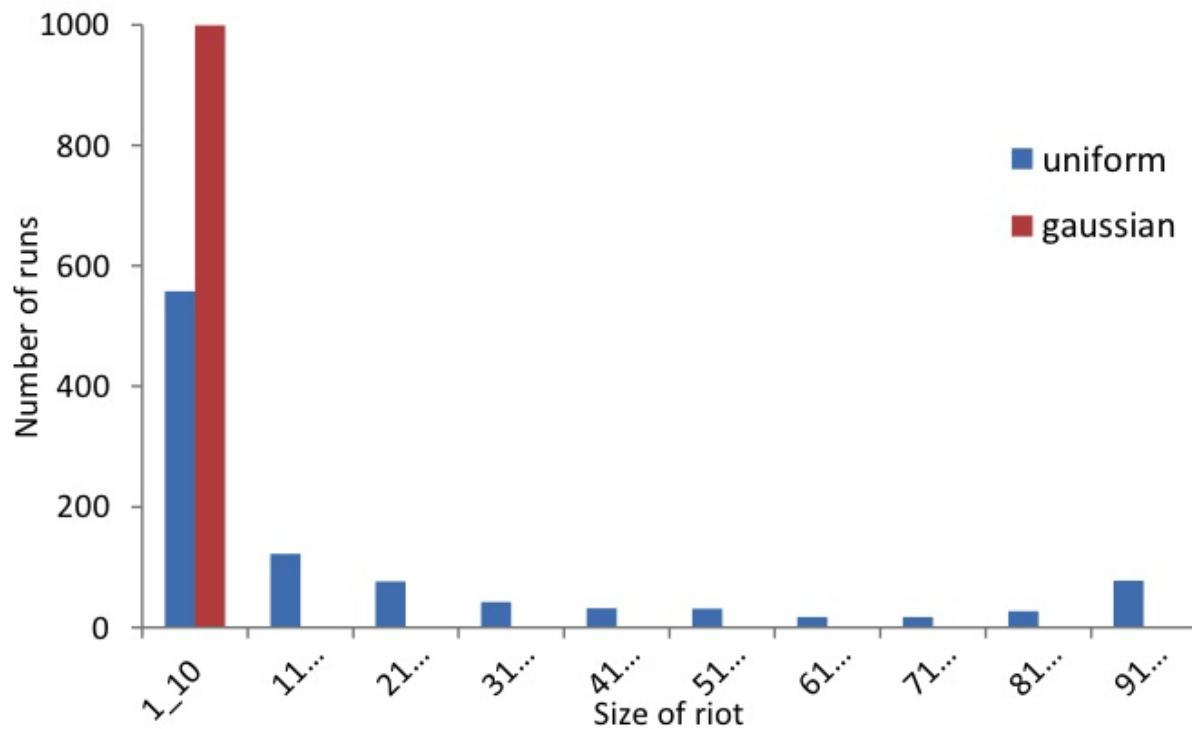


Figure 3: Distribution of riot size for different distributions of thresholds.

The thresholds we have used so far are all or nothing. An alternative way to model the thresholds is to define them as [probability functions](#). We define the probability that an agent will participate as follows

$$P[A] = X\alpha / (T\alpha + X\alpha)$$

Where $P[A]$ is the probability of action, X is the number of agents already participating, T is the threshold of the agent and α is a parameter that defines the shape of the function. Figure 4 shows the probability function for $T = 50$ and $\alpha = 5$ or 10 . Figure 5 shows that the smoother the threshold the more likely that the whole group will join the riot. Still the Gaussian distribution of thresholds do not lead to many population size riots.

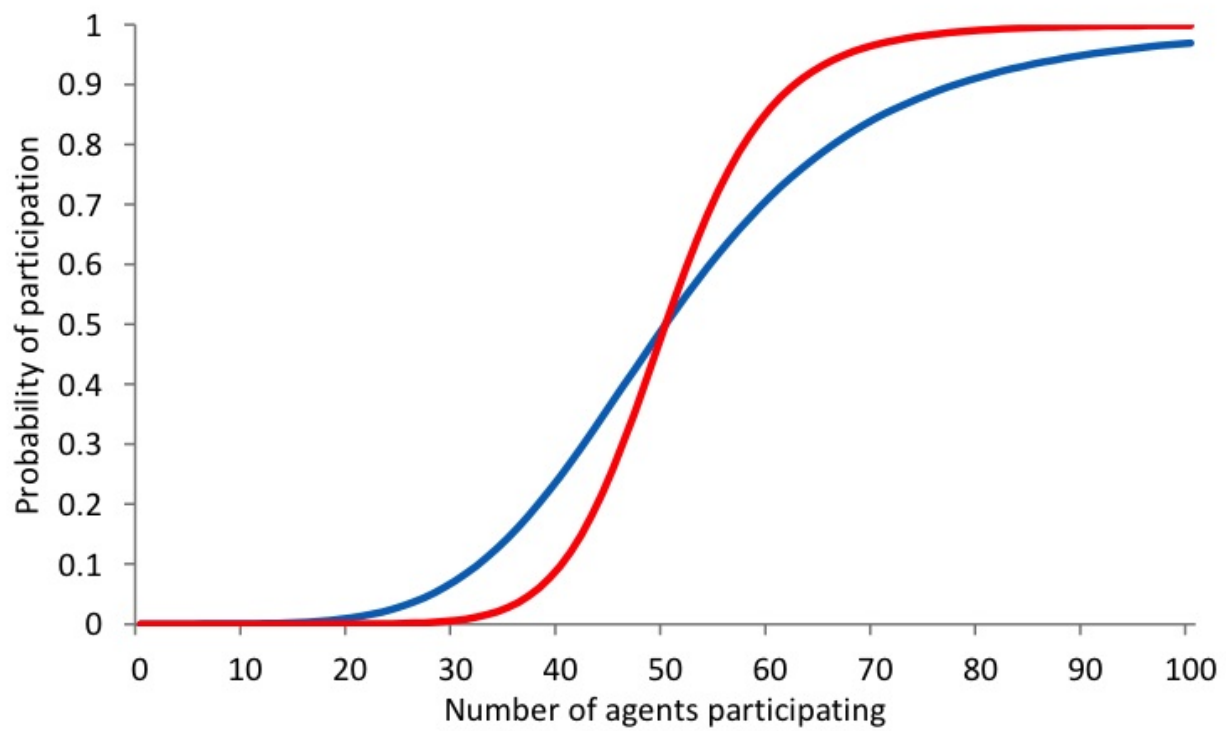


Figure 4. Probability distribution of participation with $T = 50$ for $\alpha = 5$ (blue) and $\alpha = 10$ (red).

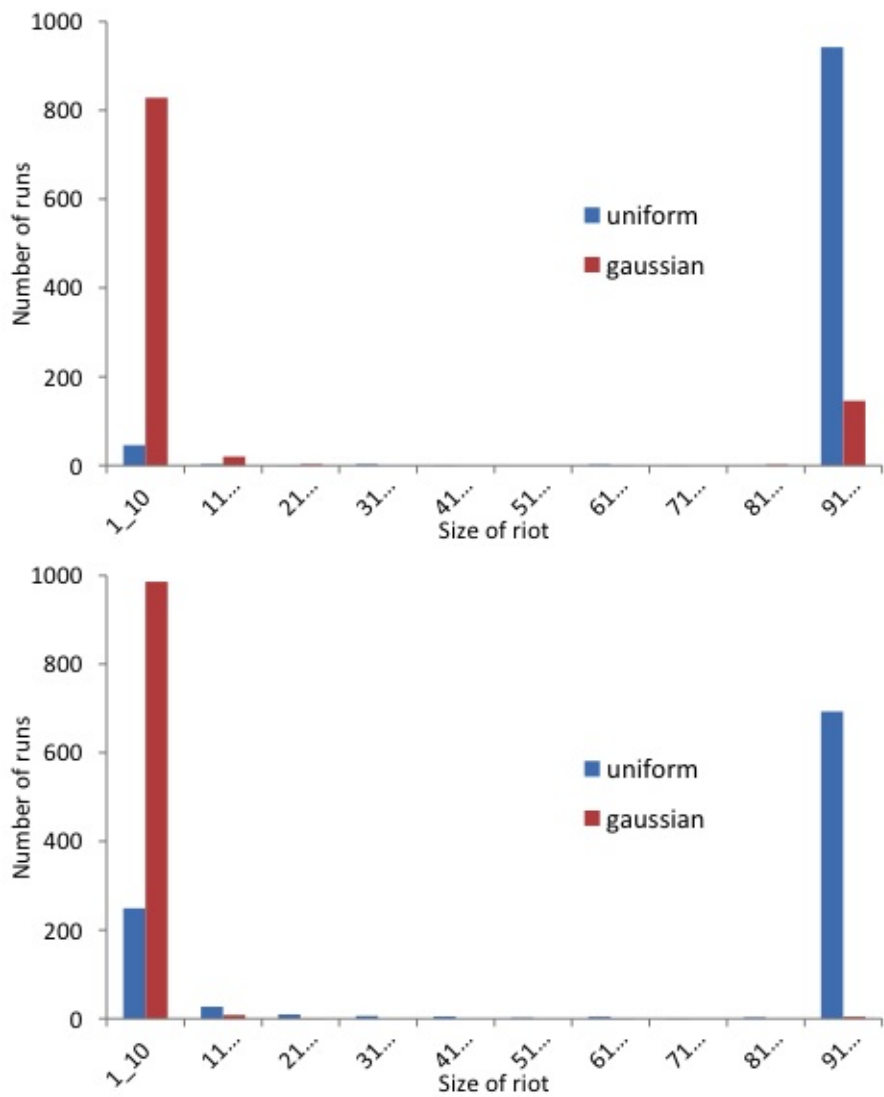


Figure 5. Distribution of riot sizes for different probability functions using $\alpha = 5$ (top) and $\alpha = 10$ (bottom).

NETLOGO EXAMPLE: GRANOVETTER MODEL

Model: Right click on the Link and Save it. Open Netlogo and Run it with NetLogo.

Threshold Model Applied to Networks

People may not make the decision to contribute to the public good based on the contribution rate of society at large, but rather based on whether their friends contribute. Therefore we look at a network perspective of collective action and implement the same threshold logic as above to agents on the network.

We will use a small world network and explore the consequences of different rewiring probabilities. Each agent has a threshold and each tick agents will check whether the share of participation among their friends is above their threshold. If so, they will adopt to contribute to the collective action problem.

At the start of the simulation one of the agents is picked at random to contribute. Figure 6 shows an example of a diffusion of collective action. When all agents were in one population we had, over 1000 runs, an average group size of 1.3 for the Gaussian distribution and 22.8 for the uniform distribution. In a networked environment the Gaussian distribution of thresholds lead to an average group size of 2.5 and 4 for the uniform distribution. It is unlikely to have collective action spread through the whole population since diffusion of cooperative behavior is easily blocked by a few agents who do not want to collaborate, e.g., having a high threshold.

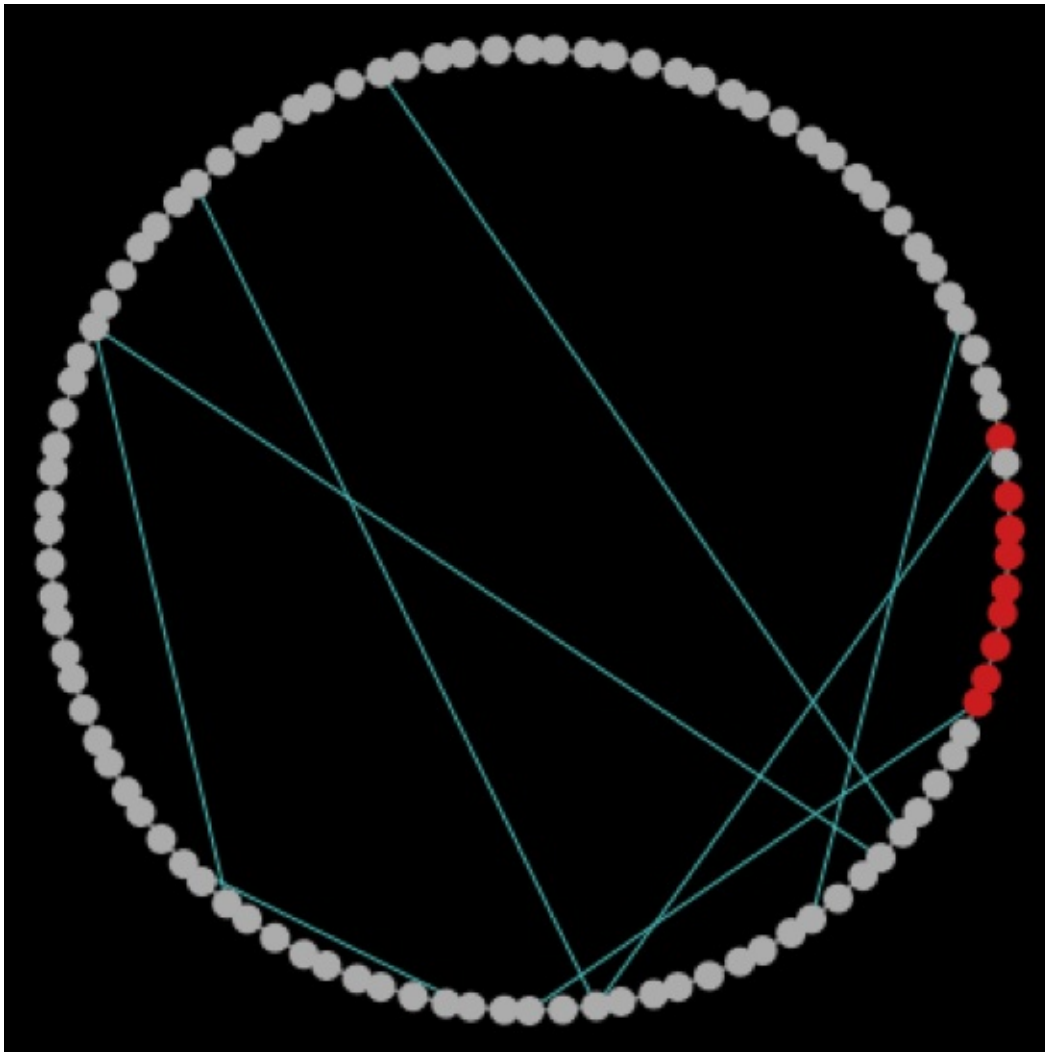


Figure 6. Screenshot of diffusion of collective action in a small world network.

NETLOGO EXAMPLE: THRESHOLD IN NETWORKS IN

Model: Right click on the Link and Save it. Open Netlogo and Run it with NetLogo.

Pick Your Friends

The network approach discussed above is static and agents with different threshold values are connected with each other. Another approach would be for agents to find others like them who want to form a group to implement a public good. In this section we present a simple model of such a community-evolving process.

Agents are randomly allocated on the landscape. The agents make decisions to contribute to a public good or not. At least two agents need to be within a radius D in order to create a public good. The payoff from the public good is equal to

$$y_i = 1 - x_i + r \sum_{j=1}^N x_j$$

Where r is the marginal per capita return, y the payoff and x the investment in the public good. The public good game is played with agents within a radius D of the agents. This means that each agent has a different set of players.

When agents have interacted with the agents around them they will receive feedback on the attitude of the other agents. We assume that agents who are surrounded with agents where the level of cooperation is above threshold will stay around. If the level of cooperation of the other agents is lower than that threshold they will move away from the group. Agents who are tolerant, a low value of threshold, are more likely to stay.

This is modeled in NetLogo as

```
let otheragents other turtles in-radius dist
let numberofagents count other turtles in-radius dist
if numberofagents > 0 [
  let meandecision mean [decision] of otheragents
  set payoff payoff + 1 - decision + 0.6 * (meandecision * numberofagents + decision)
  ifelse meandecision > threshold [
    face one-of other turtles in-radius dist
  ][
    face one-of other turtles in-radius dist
    set heading heading + 180
  ]
]
```

When the threshold values are initially drawn from a uniform distribution we get the distribution of agents as shown in Figures 7 and 8. Agents with low thresholds start cooperating and attract other agents who only cooperate with higher thresholds. This leads to groups expanding. In the end all agents cooperate.



Figure 7. Agents colored according to their thresholds. Green is around 0.5, black around 0 and light green around 1.



Figure 8. Agents colored for their decisions. Black agents contributed, light blue agents did not contribute.

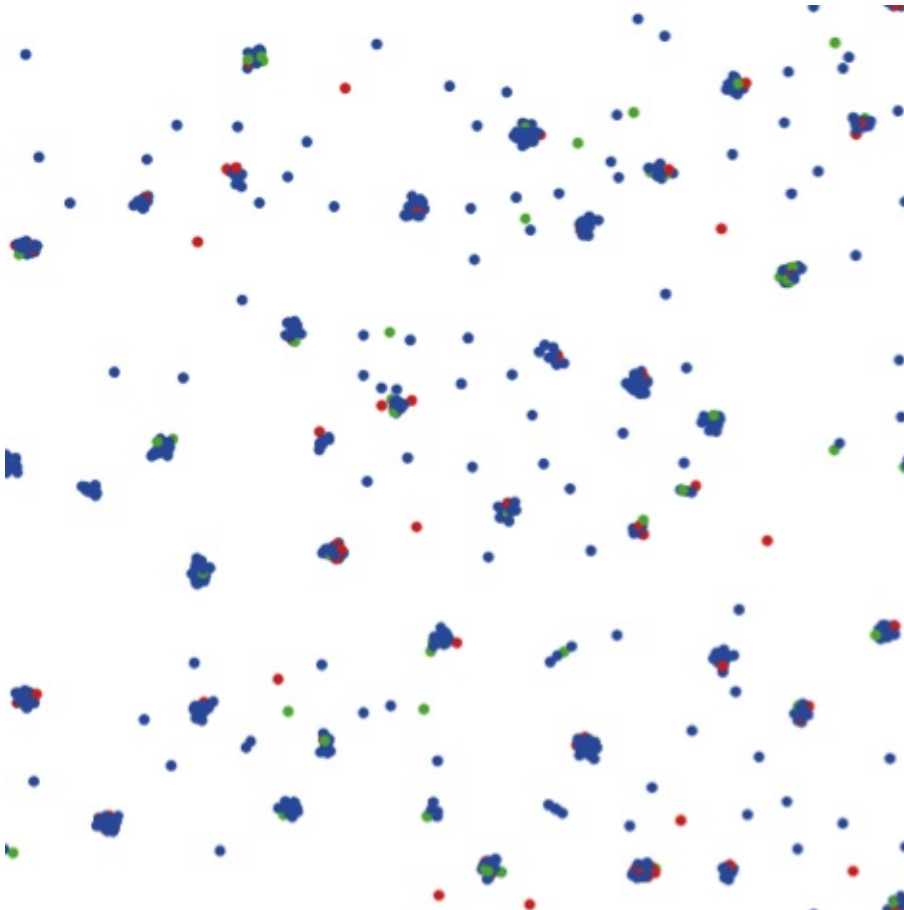
The agents in the example above did not evolve their thresholds. We will now include an evolutionary force. The fitness of an agent is assumed to be the average payoff per tick derived during its lifetimes. Each tick we will pick one agent from the population that has the lowest average payoff and we will replace that agent with an agent that has randomly drawn traits and threshold value and put it randomly on the landscape. We also add noise to the traits each tick. This means that the traits remain varying and do not get locked in into one particular value.

By removing the least successful agent each tick we get a population that becomes more similar and tolerant of differences of trait characteristics. We will get agents evolving to all cooperation again.

We will now add types of agents. The threshold-based agents are conditional cooperators. They only cooperate if $x\%$ of others cooperate too. In empirical research we find 2 other types of agents: the egoists who are always defecting, and the altruists who are always

cooperating.

We start the simulation with 10% egoists and 10% altruists, and make the rest of the agents conditional cooperators. We see groups emerging initially started by some altruists and conditional cooperators with low thresholds (Figure 9). The groups can grow over time but since egoists are also attracted to groups of cooperators, groups can fall apart. Egoists entering the group lead some conditional cooperators to stop cooperating and leaving the group.



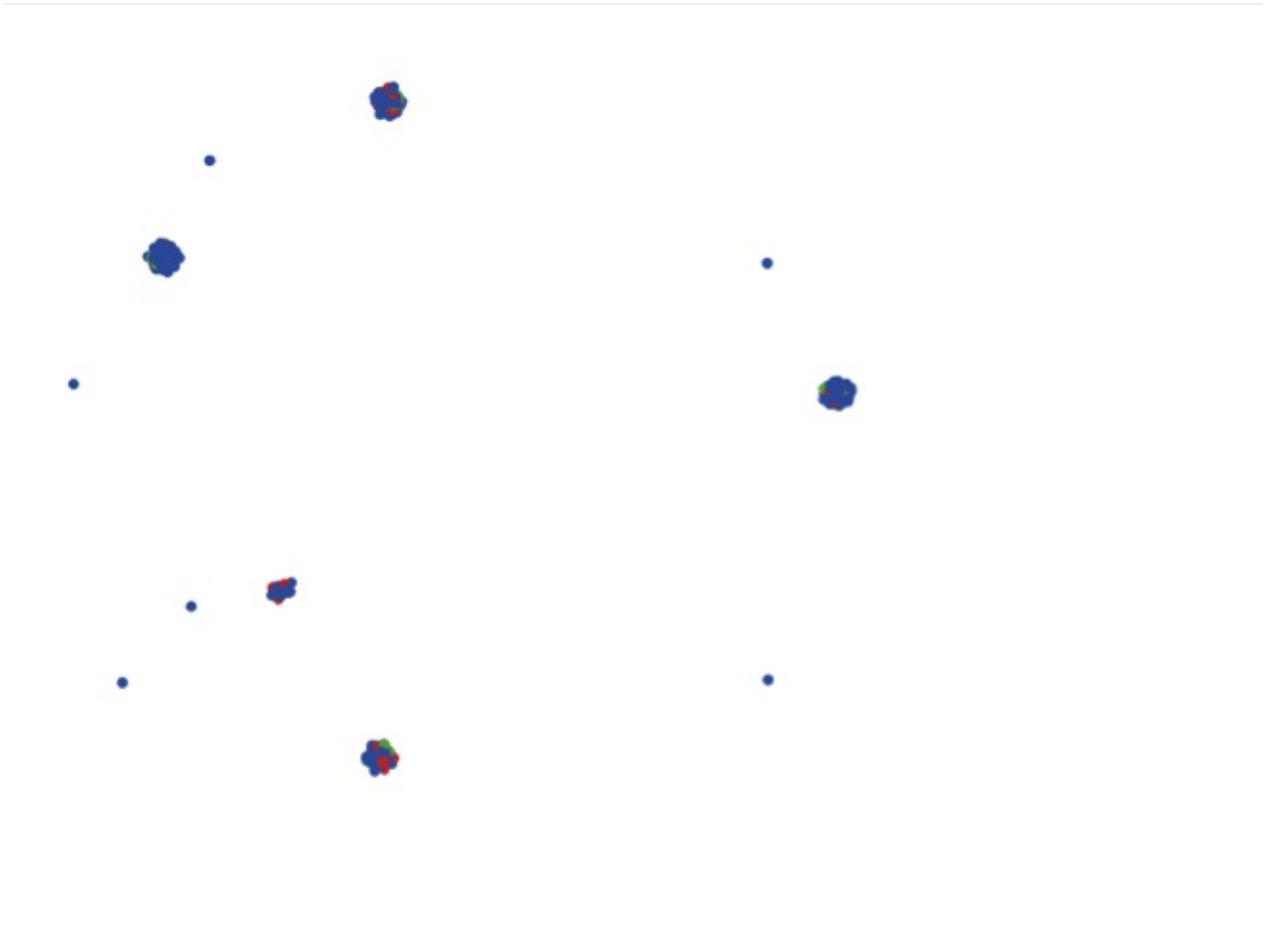


Figure 9: Clusters of agents with `dist` equal to 5 at the beginning of the simulation (top) and after 25,000 ticks (bottom).

Figure 10 shows that the distribution of types of agents remains relatively stable over time, but the distribution of thresholds evolves to more tolerant agents. Conditional cooperators who only cooperate if everybody around them cooperates will not find a group that will lead them to gain a higher fitness. As such agents who are very picky will die out.

Play around with the model and see the different dynamics you get with different values of `dist` and with different initial distributions of the types of agents. You will see that the results are quite sensitive to the initial conditions.

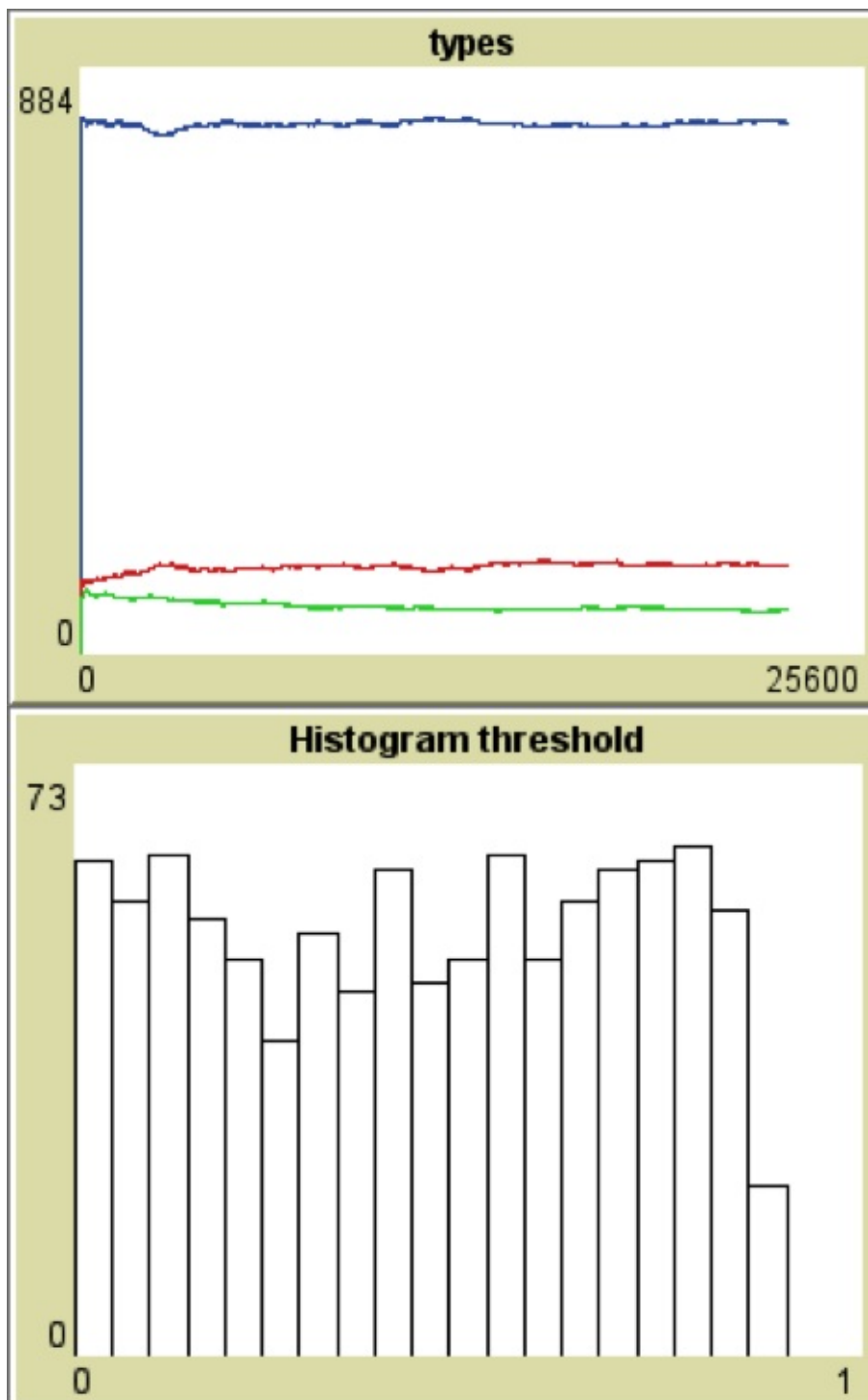


Figure 10: Time series of the share of the types of agents (left) and the distribution of thresholds after 25,000 ticks. The colors in the left figure refer to egoists (red), altruists (green) and conditional cooperators (blue).

The model discussed is very simplistic with respect to the actual social complexity. We could add to the model that cooperative agents would punish freeriders so that egoists will comply with the norms in the community. The model can be extended by rules on when to enter or exit a group. The model simply assumes that everybody can join and leave. In practice it might be difficult to get people to join a group or to kick a defector out of the group. Hence this simple model can be extended in various ways to study questions on collective action.

NETLOGO EXAMPLE: GROUPS

Model: Right click on the Link and Save it. Open Netlogo and Run it with NetLogo.

Summary and Reference

Summary

In this chapter we explored models of collective action where agents make decisions of cooperation conditionally based on the decisions of their neighbors. We explore different spatial configurations and observe that the ability to select partners leads to the highest performance of the society. When groups grow over time with more altruistic agents at the start, a higher level of cooperation can be achieved.

References

Granovetter, Mark (1978). Threshold Models of Collective Behavior. *American Journal of Sociology* 83 (May): 1420-1443.